

RK3568 五自由度机械臂 · ROS 2 + ros2_control + MoveIt2 完整实施手册

目标：让一台 5 自由度 + 夹爪 的舵机机械臂，能够接受一个 三维坐标，由 MoveIt2 自动规划路径，并平滑地驱动真实机械臂到达。

本文档从零讲起：技术栈是什么 → 怎么配 → 怎么测 → 踩了什么坑 → 怎么解决。所有代码都是板子上实际在跑的版本。

目录

序	最终成果
第 1 章	技术栈与系统全貌（从基础讲起）
第 2 章	环境准备
第 3 章	运动学基础（先算清楚再写代码）
第 4 章	控制层：让 CAN 舵机接入 <code>ros2_control</code>
第 5 章	控制层：控制器配置
第 6 章	规划层：MoveIt2 配置
第 7 章	自研解析 IK 插件
第 8 章	★ 测试流程（分层，一层一层来）
第 9 章	★ 问题与解决（13 个，个个详细）
第 10 章	STM32 固件侧改动
第 11 章	性能、精度与限制
第 12 章	运维与排错速查
附录 A	命令速查
附录 B	简历写法

序 · 最终成果

实测数据（真实机械臂）

① 三维坐标 → 规划 → 执行（10 个目标）

目标	三维坐标	规划	执行	实测误差
j1 +0.20	(+0.029, +0.005, +0.322)	OK	OK	14.4 mm
j1 -0.20	(+0.040, -0.003, +0.312)	OK	OK	14.0 mm
j2 +0.15	(+0.087, +0.005, +0.300)	OK	OK	10.5 mm
j2 -0.15	(+0.062, +0.005, +0.311)	OK	OK	11.4 mm
j3 +0.15	(+0.095, +0.004, +0.302)	OK	OK	3.1 mm
j3 -0.15	(+0.069, +0.002, +0.301)	OK	OK	9.9 mm
j4 +0.20	(+0.083, -0.008, +0.292)	OK	OK	4.6 mm
j1+j2 复合	(+0.109, +0.006, +0.283)	OK	OK	17.8 mm
j2+j3 复合	(+0.131, +0.007, +0.264)	OK	OK	13.5 mm
大复合	(+0.146, +0.031, +0.243)	OK	OK	19.7 mm
★ 规划 10/10 执行 10/10 容差失败 0 次 末端误差 平均 11.9 mm				

② 大幅运动（joint1 ±1.30 rad = ±75°，joint5 ±2.5 rad）

★ 规划 8/8 执行 8/8 末端误差 平均 21.5 mm

③ 顺滑度（裸 CAN 以 20Hz 采样，算速度的变异系数 CV，越小越顺）

平均速度 +0.129 rad/s 速度标准差 0.017 ★ CV 0.133 停顿样本占比 0.0% 速度反向 0 次 (第二次测量 CV 0.134 —— 复现性极好)
--

关键成果

#	成果	难度
1	自研解析逆解 MoveIt2 插件 —— KDL 是 6-DOF 数值解，5-DOF 解不出来	★★★
2	定位并根治「一卡一卡」 —— 根因是 STM32 的查询打断了运动命令	★★★
3	ros2_control 自定义硬件接口 —— 把 CAN 舵机接入 ROS 2 标准控制框架	★★
4	13 个隐蔽问题全部定位并修复（见第 9 章）	★★★

第 1 章 技术栈与系统全貌

1.1 一句话说清这套系统



核心价值：把运动规划和硬件驱动解耦。以后换机械臂只需改 URDF + 参数，换硬件只需重写硬件插件，规划层完全不动。

1.2 三层架构

应用层	脚本 / RViz / moveit_py 给 (x, y, z) 或末端位姿	
规划层 ★ MoveIt2 (move_group) ★	【第 6-7 章】	
├── 逆运动学	末端位姿 → 关节角	
├── 碰撞检测	自碰撞 + 环境障碍	
├── 路径规划	OMPL RRTConnect	
├── 时间参数化	Ruckig (加加速度受限, 让轨迹「丝滑」)	
	输出 JointTrajectory	
控制层 ★ ros2_control ★	【第 4-5 章】	
├── controller_manager	加载硬件插件 + 管理控制器	
├── joint_trajectory_controller	把轨迹按时间插值, 50Hz 下发	
├── joint_state_broadcaster	把硬件状态发成 /joint_states	
└── ★ ArmSystemHardware ★	自定义硬件插件 (你写的)	
	实现 read() / write(), 对接 CAN	
总线	CAN 2.0B @ 250 kbps	
执行	STM32F407 (FreeRTOS) → USART2 半双工 → 6× 舵机 ZX20S (夹爪) + ZX361S (joint1~5)	

1.3 ros2_control 是什么

一句话：ROS 2 官方的实时控制框架。它规定了「硬件怎么被驱动、控制器怎么被加载」的标准接口。

它解决的问题

没有它的时候，你要自己写一个节点：读传感器、算控制、发指令、发布状态……每个机械臂都重写一遍。

有了它，你只需要实现一个硬件插件：

```
class MyHardware : public hardware_interface::SystemInterface
{
    // ① 读：从硬件拿状态，填到 state_interface
    return_type read(const rclcpp::Time&, const rclcpp::Duration&) override;

    // ② 写：把 command_interface 的命令下发给硬件
    return_type write(const rclcpp::Time&, const rclcpp::Duration&) override;

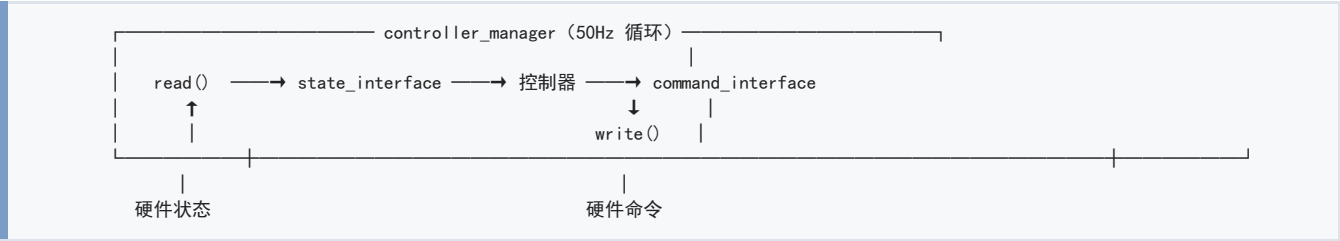
    // ③ 导出接口：告诉框架「我有哪些可读可写的量」
    std::vector<StateInterface> export_state_interfaces() override;
    std::vector<CommandInterface> export_command_interfaces() override;
};
```

剩下的（轨迹插值、状态发布、生命周期管理）框架全包了。

核心概念

概念	含义
SystemInterface	一个「硬件系统」的抽象基类（比如一整台机械臂）
command_interface	控制器 → 硬件的变量（比如 joint1/position）
state_interface	硬件 → 控制器的变量
controller_manager	进程，负责加载硬件 + 按周期调用 read() / write()
update_rate	控制周期，本项目 50 Hz（20 ms）
控制器（controller）	消费 state、产生 command 的组件，比如 JTC
生命周期	on_init → on_configure → on_activate → 运行 → on_deactivate

数据流



关键顺序：每个周期先 read()（更新状态），再让控制器算，最后 write()（下发命令）。

1.4 MoveIt2 是什么

一句话：ROS 2 官方的运动规划框架。负责「从 A 到 B 怎么走」。

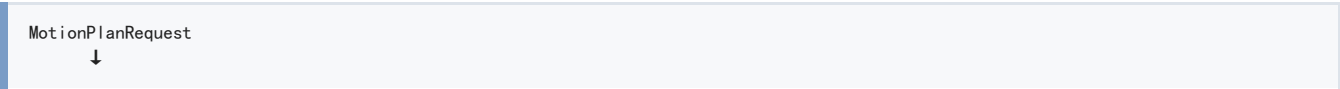
它的职责

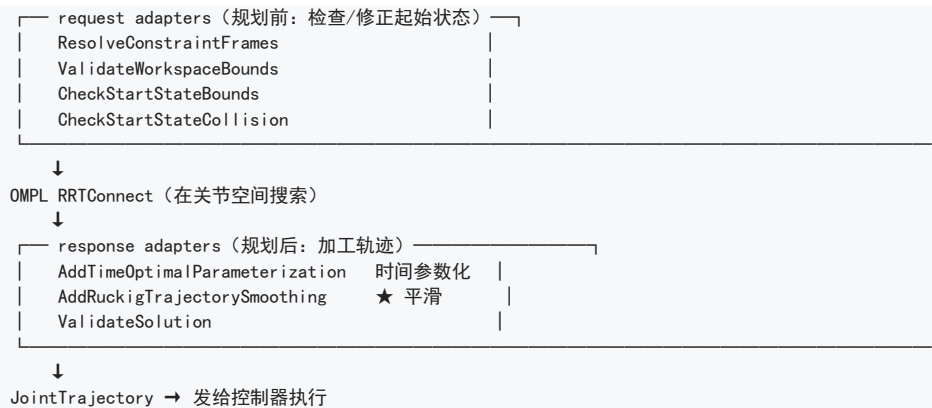
职责	说明	本项目对应
逆运动学（IK）	末端位姿 → 关节角	★ 自研解析插件
碰撞检测	自己会不会撞自己 / 撞环境	SRDF 的 disable_collisions
路径规划	在关节空间找一条无碰撞路径	OMPL RRTConnect
时间参数化	给路径加上速度/加速度曲线	AddTimeOptimalParameterization
平滑	让加加速度（jerk）受限	AddRuckigTrajectorySmoothing

核心概念

概念	含义
move_group	MoveIt2 的主节点（本项目就是它）
SRDF	语义描述：规划组、末端执行器、禁用碰撞对
规划组（group）	一组一起规划的关节，比如 arm = joint1~joint5
PlanningScene	机器人 + 环境的完整描述
JointTrajectory	规划的输出：一串带时间戳的关节角
OMPL	采样类规划器库，RRTConnect 最常用
规划管线（pipeline）	规划器 + 一串「适配器」

规划管线





1.5 两者怎么衔接

桥梁就一个文件: `moveit_controllers.yaml`

```

moveit_simple_controller_manager:
  controller_names:
    - arm_controller
    - gripper_controller

  arm_controller:
    type: FollowJointTrajectory          # ← 用 action 通信
    action_ns: follow_joint_trajectory
    joints: [joint1, joint2, joint3, joint4, joint5]

```

运行时的连接:

```

MoveIt2 规划完
  ↓ 发 FollowJointTrajectory action goal
/arm_controller/follow_joint_trajectory    ← action 名字
  ↓
joint_trajectory_controller (ros2_control 的控制器)
  ↓ 写 command_interface
ArmSystemHardware::write()
  ↓ CAN
STM32 → 舵机

```

所以 MoveIt2 和 ros2_control 完全解耦 —— 它们只通过一个标准的 action 接口通信。这也是为什么硬件层一行都不为 MoveIt2 改。

1.6 为什么需要逆解, 5-DOF 为什么特殊

正解 vs 逆解

正解 (FK): 已知 6 个关节角	→ 算出末端在哪	唯一解, 直接算
逆解 (IK): 已知末端位置姿态	→ 算出关节角	多解 / 可能无解, 要解方程

MoveIt2 要往三维坐标走, 就必须反复调 IK。

6-DOF vs 5-DOF

通用工业机械臂是 6 自由度 —— 末端有 6 个位姿自由度 (位置 x, y, z + 姿态 $roll, pitch, yaw$), 6 个关节刚好一一对应, IK 有唯一解 (最多 8 组)。

本机械臂只有 5 个转动关节:

```

joint1 绕 z 转  → 底座 yaw
joint2 绕 y 转  ↵
joint3 绕 y 转  └ 控制【平面内的位置】
joint4 绕 y 转  └
joint5 绕 z 转  → 末端 roll (自转)

```

- 位置 (x, y, z) 由 joint1~joint4 决定
- 姿态只剩 2 个自由度 (pitch + yaw 方向), roll 由 joint5 提供
- 6 个位姿自由度里, 只能控制 5 个

后果: KDL (MoveIt 默认的 IK 求解器) 是 6-DOF 数值解, 对本机械臂解不出来。
这就是为什么必须自己写解析 IK 插件 (详见第 7 章)。

1.7 硬件概况

项	参数
主控	RK3568 (KickPi K1Mini), 4× Cortex-A55, 3.8 GB RAM
系统	Ubuntu 24.04.4 (noble) aarch64
ROS	ROS 2 Jazzy
从控	STM32F407VET6 + FreeRTOS
总线	CAN 2.0B @ 250 kbps
舵机	ZX20S ×1 (夹爪, id=0) + ZX361S ×5 (joint1~5, id=1~5)
舵机协议	#%03dP%04dT%04d! (在 T 毫秒内走到位置 P), 半双工 @115200
机械参数	L1=98.5mm, L2=95.0mm, L3=68.5mm, 底座高 72.5mm

CAN 协议

下行 (RK3568 → STM32), ID = 0x104, 8 字节

```
[id, ask, pos_lo, pos_hi, time_lo, time_hi, 0, 0]

ask  = 1 ASKFORSETPOS   「在 time 毫秒内走到 pos」
      = 2 ASKFORSTATUS  「报告状态」
      = 3 ASKFORALL     两者都做

pos / time 都是【小端】16 位
```

上行 (STM32 → RK3568), ID = 0x200, ≥5 字节

```
[id, pos_lo, pos_hi, temp, volt, 0, 0, 0]
```

实测抓包:

```
can0 104 [8] 01 02 00 00 00 00 00 00 ← 查询舵机1状态
can0 200 [8] 01 DC 05 13 46 00 00 00 ← 舵机1: 位置=0x05DC=1500, 温度=19, 电压=70
```

1.8 术语速查

术语	含义
URDF	机器人的物理描述 (连杆、关节、几何、惯性)
SRDF	机器人的语义描述 (规划组、碰撞对)
xacro	URDF 的宏语言, 支持变量和 include
JTC	joint_trajectory_controller 的简称
yaw / pitch / roll	绕 z / y / x 轴的旋转
正解 / 逆解	FK / IK
奇异位形	某些姿态下雅可比降秩, IK 或控制会失效 (本项目: 机械臂完全伸直时)
变异系数 CV	标准差 / 均值, 衡量速度抖动的相对大小

半双工	一根线上不能同时收发，总要分时
pluginlib	ROS 2 的插件机制，让程序运行时动态加载库
ament	ROS 2 的构建系统
colcon	ROS 2 的构建工具

第 2 章 环境准备

2.1 装 ROS 2 + MoveIt2 + ros2_control

板子上是 Ubuntu 24.04 (noble) aarch64, 对应 ROS 2 Jazzy。

```
# ① ROS 2 基础 (假设已按官方文档装好)
source /opt/ros/jazzy/setup.bash

# ② MoveIt2 (28 个包, 约 1.5 GB)
sudo apt install ros-jazzy-moveit

# ③ ros2_control 全套
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers \
ros-jazzy-controller-manager ros-jazzy-hardware-interface \
ros-jazzy-joint-state-broadcaster ros-jazzy-joint-trajectory-controller

# ④ 验证
ros2 pkg list | grep -c "^moveit"          # 期望 > 25
ros2 pkg list | grep -E "ros2_control|controller_manager|joint_trajectory"
```

内存注意: move_group 要吃 300~500 MB。RK3568 只有 3.8 GB, 跑之前把 VS Code Server 关掉 (它吃 ~2 GB) :

```
pkill -f vscode-server
free -h          # 目标 available > 1.5 GB
```

2.2 CAN 接口配置 (关键, 每次上电都要)

RK3568 的 CAN 控制器不保存 bitrate, 每次上电都得重设。

一次性做成开机自启 (写进 /etc/network/interfaces.d/, 和已有的 eth0/eth1 风格一致) :

```
sudo tee /etc/network/interfaces.d/can0 > /dev/null <<'EOF'
auto can0
iface can0 inet manual
    pre-up ip link set can0 type can bitrate 250000 triple-sampling on
    up ip link set can0 up
    down ip link set can0 down
EOF

sudo ifup can0          # 立即生效, 以后开机自动执行

# 验证
ip -details link show can0 | head -3
```

期望输出:

```
can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 ... state UP
can <TRIPLE-SAMPLING> state ERROR-ACTIVE (berr-counter tx 0 rx 0) restart-ms 1
    bitrate 250000 sample-point 0.868
```

三个状态的含义

状态	含义	正常吗
ERROR-ACTIVE	总线上有节点在正常应答	sqrt
ERROR-PASSIVE	没有任何节点 ACK (对方没上电/死机)	×
BUS-OFF	错误太多, 控制器自动断开	×

快速验证通信

```
# 查一次舵机 0 的状态
candump can0 &
cansend can0 104#0002000000000000
```

```
# 期望看到:
# can0 104 [8] 00 02 00 00 00 00 00  ← 我发的
# can0 200 [8] 00 DC 05 13 46 00 00  ← STM32 回的
```

如果 `cansend` 报 `No buffer space available` → 没有节点 ACK → STM32 没在跑。

2.3 工作空间布局

```
~/ros2/ros2_ws/
├── src/
│   ├── CanServer/                ← ros2_control 硬件插件 + 控制器配置
│   ├── my_robot_arm_description/ ← URDF (含 ros2_control 段)
│   ├── my_robot_arm_moveit_config/ ← SRDF + 5 个 yaml + launch
│   ├── arm_ik_plugin/           ← ★ 自研解析逆解插件
│   └── arm_ik_plugin/CMakeLists.txt
├── build/
├── install/
└── log/
```

※ `CanServer` 是软链接到 `git` 仓库：

```
cd ~/ros2/ros2_ws/src
ln -s /home/kickpi/Project/CanServer CanServer
touch /home/kickpi/Project/CanServer/can_server/COLCON_IGNORE # 避免同名包冲突
```

`CanServer` 仓库里本身还带一份 `can_server/` 子目录（`git` 跟踪的镜像），不加 `COLCON_IGNORE` 的话 `colcon` 会发现两个同名包直接报错。

2.4 编译

```
cd ~/ros2/ros2_ws
colcon build --symlink-install
source install/setup.bash

colcon list    # 期望 4 个包
```

`--symlink-install` 的好处：改了 `Python` `launch` / `yaml` / `xacro` 不用重编，直接重启就行。改 `.cpp` 还是要重编。

自查 `install` 是否完整（这个坑很常见，见第 9 章坑 1）：

```
find install/CanServer/share/CanServer -name "*.py" -o -name "*.yaml" | sed 's|.*/CanServer/||'
# 期望有 launch/ 和 config/ 下的文件
```

第 3 章 运动学基础

这一章至关重要。逆解公式写错的话，表现是「偶发的定位错误」，非常难查。所以先把模型用 FK 逐点验证，再写代码。

3.1 从 URDF 读机构

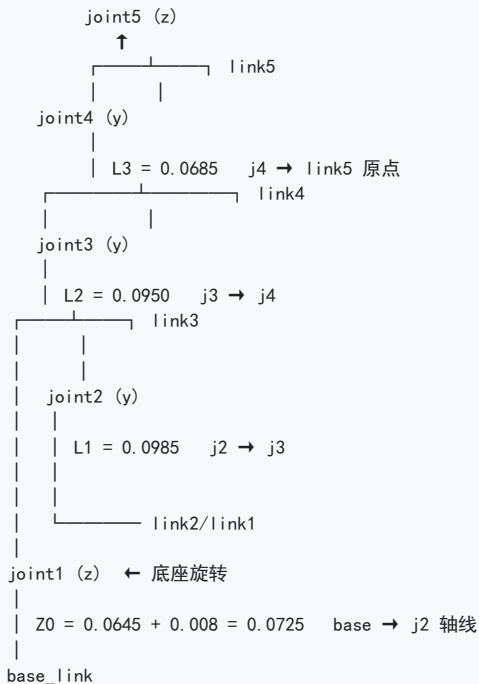
```
xacro src/my_robot_arm_description/urdf/robot_arm.urdf.xacro > /tmp/arm.urdf

python3 - <<'PY'
import xml.etree.ElementTree as ET
r = ET.parse('/tmp/arm.urdf').getroot()
for j in r.findall('joint'):
    o, a, lim = j.find('origin'), j.find('axis'), j.find('limit')
    print("%-30s %-9s %-22s → %-22s origin=%-18s axis=%-8s %s" % (
        j.get('name'), j.get('type'),
        j.find('parent').get('link'), j.find('child').get('link'),
        o.get('xyz') if o is not None else '-',
        a.get('xyz') if a is not None else '-',
        "[%s, %s]" % (lim.get('lower'), lim.get('upper'))
        if lim is not None and j.get('type') != 'fixed' else ''))
PY
```

输出：

```
joint1 revolute base_link → link1 origin=0 0 0.0645 axis=0 0 1 [-1.5708, 1.5708]
joint2 revolute link1 → link2 origin=0 0 0.008 axis=0 1 0 [-1.5708, 1.5708]
joint3 revolute link2 → link3 origin=0 0 0.0985 axis=0 1 0 [-1.5708, 1.5708]
joint4 revolute link3 → link4 origin=0 0 0.095 axis=0 1 0 [-1.5708, 1.5708]
joint5 revolute link4 → link5 origin=0 0 0.0685 axis=0 0 1 [-3.1416, 3.1416]
joint6 fixed link5 → gripper_base
gripper_finger_left_joint prismatic axis=0 1 0 [0, 0.02]
gripper_finger_right_joint prismatic axis=0 -1 0 [0, 0.02] ← mimic 左指
```

读出来的机构：



几何常量：

```
Z0 = 0.0645 + 0.008 = 0.0725 # base_link 原点 → joint2 轴线高度
L1 = 0.0985 # joint2 → joint3
L2 = 0.0950 # joint3 → joint4
```

L3 = 0.0685

joint4 → link5 原点

3.2 正解推导

关键约定：定义绝对角（相对 +z 轴），而不是相对角：

```
a2 = q2
a3 = q2 + q3
a4 = q2 + q3 + q4
```

为什么用绝对角？因为连杆的方向只取决于它相对基座的绝对角度，写成绝对角后 每一段的贡献可以直接相加，公式最简洁。

平面内（先忽略 joint1 的旋转）：

绕 +y 轴转 a 角后，+z 轴映射到 $(\sin a, 0, \cos a)$ ，所以长度为 L 的连杆：

```
贡献 x = L * sin(a)
贡献 z = L * cos(a)
```

叠加：

```
r = L1 * sin(a2) + L2 * sin(a3) + L3 * sin(a4)    平面内水平距离
Z = Z0 + L1 * cos(a2) + L2 * cos(a3) + L3 * cos(a4) 高度
```

再叠加 joint1 的底座旋转：

```
X = r * cos(q1)
Y = r * sin(q1)    ← 注意是 +sin, 不是 -sin (下面验证)
Z = Z (不变)
```

3.3 用 /compute_fk 逐点验证

※ 这一步不能跳。推导里有 2 个容易搞错的地方：- q1 旋转方向 ($Y = \pm r \cdot \sin(q1)$) - 绝对角 vs 相对角

验证脚本 kin_check.py (见 _board/src/kin_check.py)：

核心逻辑：

```
L1, L2, L3, Z0 = 0.0985, 0.0950, 0.0685, 0.0725

def model(q, ysign):
    q1, q2, q3, q4, _ = q
    a2, a3, a4 = q2, q2 + q3, q2 + q3 + q4
    x = L1*math.sin(a2) + L2*math.sin(a3) + L3*math.sin(a4)
    z = Z0 + L1*math.cos(a2) + L2*math.cos(a3) + L3*math.cos(a4)
    return x*math.cos(q1), ysign*x*math.sin(q1), z

# 对 10 组随机关节角，调 /compute_fk 拿真实值，和模型比
```

运行：

```
ros2 launch CanServer arm_control.launch.py & # 提供 /compute_fk 需要 move_group
ros2 launch my_robot_arm_moveit_config arm_moveit_headless.launch.py &
python3 test/kin_check.py
```

实测输出：

关节角	FK 实测	模型	误差
[0.00, 0.00, 0.00, 0.00, 0.00]	(+0.0000, +0.0000, +0.3345)	(+0.0000, +0.0000, +0.3345)	0.00000
[0.00, +0.40, -0.60, +0.30, 0.00]	(+0.0263, +0.0000, +0.3245)	(+0.0263, +0.0000, +0.3245)	0.00000
[+0.50, +0.40, -0.60, +0.30, 0.00]	(+0.0231, +0.0126, +0.3245)	(+0.0231, +0.0126, +0.3245)	0.00000
[-0.44, -0.88, +0.38, -1.07, +0.18]	(-0.1714, +0.0813, +0.2188)	(-0.1714, +0.0813, +0.2188)	0.00000
[-1.08, -1.03, -0.19, +0.82, -1.89]	(-0.0941, +0.1765, +0.2193)	(-0.0941, +0.1765, +0.2193)	0.00000
+1.20, -1.14, +0.90, -0.53, -1.79]	(-0.0582, -0.1485, +0.2553)	(-0.0582, -0.1485, +0.2553)	0.00000
(共 10 组, 全部 0.000000)			

最大误差: ysign=+1 → 0.000000 m ysign=-1 → 0.353012 m

sqrt 模型正确, joint1 用 $Y = +x \cdot \sin(q1)$

结论: 模型精确, 且确认了 $q1$ 的符号约定。

为什么必须这么做: 如果 $q1$ 符号搞反了, 机械臂会往镜像方向走 —— 这个错误在关节空间测试里完全看不出来, 只有到三维坐标规划时才会暴露, 而且会被误判成「IK 有问题」。

3.4 逆解推导

已知 / 待求

已知: 目标位置 (X, Y, Z) —— link5 原点在 base_link 坐标系下
待求: $q1, q2, q3, q4, q5$

第 0 步: 数自由度

- 位置需要 3 个 DOF, 但 $q5$ 是绕末端轴自转, 不影响 link5 原点的位置
- 所以位置由 $q1, q2, q3, q4$ 决定
- 4 个关节控制 3 个位置自由度 → 有 1 个冗余自由度

这个冗余就是 $a4$ (腕部绝对角)。逆向时先选定 $a4$, 剩下的就唯一了。

第 1 步: 底座 yaw

$q1 = \text{atan2}(Y, X)$
 $r = \text{hypot}(X, Y)$ ← 取正值

为什么取正值? 因为 $X = r \cdot \cos(q1)$ 、 $Y = r \cdot \sin(q1)$, 若真实 $r < 0$, 则 atan2 会给出 $q1 + \pi$, 配合 $r = |r|$ 得到同一位置 sqrt

第 2 步: 消掉腕部那段 L3

选定的 $a4$ 决定了 L3 那段指向哪。把它从目标里减掉, 就得到 joint4 轴线该在的位置:

$rp = r - L3 \cdot \sin(a4)$
 $zp = (Z - Z0) - L3 \cdot \cos(a4)$

第 3 步: 退化成平面 2R 解析解

现在问题变成: 两条连杆 ($L1, L2$) 从原点出发, 到达 (rp, zp) 。

$$d^2 = rp^2 + zp^2$$

余弦定理:

位置展开: $rp = L1 \cdot \sin(a2) + L2 \cdot \sin(a3)$
 $zp = L1 \cdot \cos(a2) + L2 \cdot \cos(a3)$

平方相加: $d^2 = L1^2 + L2^2 + 2 \cdot L1 \cdot L2 \cdot (\sin a2 \sin a3 + \cos a2 \cos a3)$
 $= L1^2 + L2^2 + 2 \cdot L1 \cdot L2 \cdot \cos(a3 - a2)$

※※ 这里是全项目最容易写错的地方 ※※

因为 $a2/a3$ 是【绝对角】, 所以余弦定理给出的是 $\cos(a3 - a2)$ 而不是 $\cos(a3)$!

手算验证 (目标 $r=0.026323, Z=0.324489, a4=0.1$, 真实关节角 $[0, 0.4, -0.6, 0.3, 0]$):

$rp = 0.019494$ $zp = 0.183839$ $d^2 = 0.034177$
 $L1^2 = 0.009702$ $L2^2 = 0.009025$ $2 \cdot L1 \cdot L2 = 0.018715$

公式给 $C = (d^2 - L1^2 - L2^2) / (2 \cdot L1 \cdot L2) = 0.015450 / 0.018715 = 0.82554$

$\cos(a3 - a2) = \cos(-0.6) = 0.82534$ sqrt 吻合

```
cos(a3) = cos(-0.2) = 0.98007 × 写成 cos(a3) 就对不上
```

继续求解：

```
Δ = ±acos(C) = a3 - a2 = q3 (肘部两种构型)
a2 = atan2(rp, zp) - atan2(L2·sin Δ, L1 + L2·cos Δ)
a3 = a2 + Δ
```

第 4 步：回代成关节角

```
q1 = q1
q2 = a2
q3 = a3 - a2 = Δ
q4 = a4 - a3
q5 = 种子值 (roll 不影响位置, 沿用调用者给的种子)
```

第 5 步：怎么处理冗余自由度 a4

a4 可以取很多值，每个值对应一组不同的解。遍历它：

- ① 先试种子给出的 a4 (a4_seed = seed.q2 + seed.q3 + seed.q4) —— 保证「就近解」优先
- ② 再以 6° 为步长扫 ±180° (61 个候选)
- ③ 每个 a4 试肘部两种构型 (Δ 取正/负)
- ④ 过滤：关节限位检查 + 正解自校验
- ⑤ 全部解按「离种子的距离」排序

共 124 次求解，每次只有十几次浮点运算 → 单次 IK 调用 < 1 ms。

※ 那个「正解自校验」——本项目的救命设计

```
// 解出来先用正解回代验一遍，误差超过 1e-6 就丢掉
double fx, fy, fz;
forwardArm(q, fx, fy, fz);
const double err = std::sqrt((fx-X)*(fx-X) + (fy-Y)*(fy-Y) + (fz-Z)*(fz-Z));
if (err > 1e-6) return;
```

这 5 行的价值：把「公式写错」从「偶发的定位偏差」变成「立刻失败」。

本项目就是靠它抓出了上面那个 `cos(a3)` 的错误 —— 见第 9 章坑 8。

代价：每个候选解多一次正解（十几次浮点运算），可以忽略。

逆解的完整判断链

```
给定 (X, Y, Z)
↓
q1 = atan2(Y, X), r = hypot(X, Y)
↓
遍历 a4 in {seed_a4} U [-180° .. 180° 步长 6°]
↓
对每个 a4, 遍历 elbow in {+1, -1}
↓
rp = r - L3·sin(a4)
zp = (Z - Z0) - L3·cos(a4)
C = (rp² + zp² - L1² - L2²) / (2·L1·L2) ← cos(a3 - a2)
|C| > 1? → 该 a4 不可达, continue
Δ = elbow·acos(C)
a2 = atan2(rp, zp) - atan2(L2·sin Δ, L1 + L2·cos Δ)
q = { q1, a2, Δ, a4 - (a2+Δ), seed[4] }
↓
关节限位检查 → 不过就丢
正解自校验 (1e-6) → 不过就丢
↓
收集所有合法解
↓
按离种子的距离排序 → 返回最短的那组
```

4.1 硬件插件要做什么

hardware_interface::SystemInterface 有 7 个关键方法：

方法	什么时候调	要做什么
on_init(info)	加载插件时	读 <param> 参数、建 CAN 连接
on_configure(state)	配置时	（本插件没用到）
on_activate(state)	激活时	★ 把舵机当前位置读回来当命令初值
on_deactivate(state)	停用时	清理
export_state_interfaces()	初始化时	告诉框架「我能提供哪些状态量」
export_command_interfaces()	初始化时	告诉框架「我能接受哪些命令」
read(t, period)	每周期	从硬件读状态
write(t, period)	每周期	把命令下发到硬件

read() 和 write() 就是整个插件的核心。

4.2 参数是怎么传进来的

参数写在 URDF 的 <ros2_control> 段里（robot_arm.ros2_control.xacro）， 框架会解析后通过 info.hardware_parameters 传给 on_init。

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!--
    关节顺序 = <param> 数组顺序 = 舵机 id 顺序，必须一致！
    硬件插件按 info_.joints 的下标去索引这些数组。
  -->
  <ros2_control name="MyRobotArm" type="system">
    <hardware>
      <plugin>CanServer/ArmSystemHardware</plugin>

      <param name="can_interface">can0</param>
      <!-- cmd_time_ms 只是兜底值；正常运行时插件用控制周期作为舵机到位时间 -->
      <param name="cmd_time_ms">150</param>
      <!-- 每 2 个控制周期查一个舵机 = 240ms 全轴刷新一轮（50Hz 下） -->
      <!-- 每 3 个控制周期查一个舵机 = 360ms 全轴刷新一轮。
            之前取 1 虽然反馈快，但 50 次查询 + 50 次下发 = 100 帧 CAN/秒，
            STM32 的 UART 任务会积压，命令成批发出 → 也会卡。 -->
      <param name="poll_div">3</param>
      <!-- ★ 每 5 个控制周期下发一次 CAN（~100ms），T 跟随该间隔。
            舵机会走完完整行程，而不是每 20ms 被打断一次。
            调大更顺滑但跟随更滞后；调小跟得准但会卡。 -->
      <param name="send_div">5</param>
      <param name="servo_ids">0, 1, 2, 3, 4, 5</param>
      <param name="servo_min">500, 500, 500, 500, 500, 0</param>
      <param name="servo_max">2500, 2500, 2500, 2500, 2500, 2000</param>

      <!-- ★ 注意：这里从「角度」改成「位置」，夹爪是直线（米）不是角度 -->
      <param name="pos_min">-1.5708, -1.5708, -1.5708, -1.5708, -3.1416, 0.0</param>
      <param name="pos_max">1.5708, 1.5708, 1.5708, 1.5708, 3.1416, 0.020</param>

      <param name="dir">1.0, 1.0, -1.0, -1.0, 1.0, -1.0</param>
    </hardware>

    <joint name="joint1">
      <command_interface name="position"/>
      <state_interface name="position"/>
    </joint>
    <joint name="joint2">
      <command_interface name="position"/>
```

```

    <state_interface name="position"/>
  </joint>
  <joint name=" joint3">
    <command_interface name="position"/>
    <state_interface name="position"/>
  </joint>
  <joint name=" joint4">
    <command_interface name="position"/>
    <state_interface name="position"/>
  </joint>
  <joint name=" joint5">
    <command_interface name="position"/>
    <state_interface name="position"/>
  </joint>

  <!-- 夹爪：只暴露左指。右指是 mimic，由 MoveIt/Gazebo 层面处理，不占接口 -->
  <joint name="gripper_finger_left_joint">
    <command_interface name="position"/>
    <state_interface name="position"/>
  </joint>
</ros2_control>

</robot>

```

参数说明

参数	含义	本项目取值
can_interface	CAN 接口名	can0
cmd_time_ms	舵机到位时间兜底值	150（正常运行时用控制周期）
poll_div	每 N 个控制周期查一个舵机	3（全轴刷新 360ms）
send_div	每 N 个控制周期下发一次 CAN	5（~100ms，见第 9 章坑 9）
servo_ids	6 个关节对应的舵机 id	0, 1, 2, 3, 4, 5
servo_min/max	舵机原始位置范围	500~2500（夹爪 0~2000）
pos_min/max	对应的关节角/位移范围	joint1~4 ± 1.5708 , joint5 ± 3.1416 , 夹爪 0~0.020
dir	方向反射标记	1, 1, -1, -1, 1, -1

4.3 ArmSystemHardware 完整代码

头文件

```

#pragma once

#include <memory>
#include <string>
#include <vector>

#include <hardware_interface/system_interface.hpp>
#include <hardware_interface/hardware_info.hpp>
#include <hardware_interface/types/hardware_interface_return_values.hpp>
#include <rclcpp/macros.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_lifecycle/state.hpp>

#include "can_init.h"
#include "Servo_ctrl.h"

namespace arm_hardware {

class ArmSystemHardware : public hardware_interface::SystemInterface
{
public:
  RCLCPP_SHARED_PTR_DEFINITIONS(ArmSystemHardware)

  hardware_interface::CallbackReturn on_init(
    const hardware_interface::HardwareInfo & info) override;

```

```

std::vector<hardware_interface::StateInterface>
    export_state_interfaces() override;

std::vector<hardware_interface::CommandInterface>
    export_command_interfaces() override;

hardware_interface::CallbackReturn on_activate(
    const rclcpp_lifecycle::State & previous_state) override;

hardware_interface::CallbackReturn on_deactivate(
    const rclcpp_lifecycle::State & previous_state) override;

hardware_interface::return_type read(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;

hardware_interface::return_type write(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;

private:
    // —— 从 xacro 的 <param> 读来的标定参数 ——
    std::string      can_interface_ = "can0";
    // cmd_time_ms_: 舵机到位时间的【兜底值】。
    // 正常运行时 write() 用控制周期 (period) 作为到位时间,
    // 这个值只在首次调用 (period 为 0) 时使用。
    int              cmd_time_ms_   = 150;
    // send_div_: 每 N 个控制周期才【下发一次 CAN 命令】。
    //
    // ★ 这是「一卡一卡」的关键参数 ★
    //
    // 舵机协议 #P%04dT%04d! 的语义是「在 T 毫秒内从当前位置走到 P」——
    // 每次收到新命令都会从当前位置【重新开始】这段行程。
    //
    // 每 20ms 发一次、T=20ms 时, 舵机每 20ms 就因新命令被打断,
    // 永远停在加速阶段, 从没走完一段完整行程 → 明显一卡一卡。
    //
    // 改成每 send_div_×20ms 发一次、T 同样取 send_div_×20ms,
    // 舵机每次都能走完完整行程 → 顺滑。
    //
    // 取值: 3→60ms, 5→100ms, 10→200ms。
    // 越小跟得越准但越卡; 越大越顺滑但跟随越滞后。
    int              send_div_      = 5;

    // poll_div_: 每 N 个控制周期查询一个舵机。
    // 全轴刷新 = poll_div × N_joints / update_rate。
    // ※ 对 trajectory_controller 来说 > 0.5s 就太慢了,
    // 推荐 2 (=240ms @50Hz)。
    int              poll_div_      = 2;
    std::vector<uint8_t> servo_ids_;
    std::vector<double> servo_min_, servo_max_;
    std::vector<double> pos_min_,  pos_max_;
    std::vector<double> dir_;

    // —— 复用你已有的、不依赖 ROS 的 CAN / 舵机类 ——
    // ※ ServoCtrl_object 持有 Cansocket_object& 引用
    // 所以 can_ 必须先声明 (后析构)
    std::unique_ptr<Cansocket_object> can_;
    std::unique_ptr<ServoCtrl_object> servo_;

    // —— 接口缓存 (下标 = info_. joints 的下标) ——
    std::vector<double> hw_commands_;
    std::vector<double> hw_positions_;
    std::vector<double> hw_velocities_;

    // —— 运行时状态 ——
    size_t tick_      = 0;    // 控制周期计数
    size_t send_tick_ = 0;    // 控制周期计数 (write 用)
    // send_age_: 距上次【真正下发 CAN 命令】过了多少个控制周期。
    // ★ 用来判断舵机是否正在运动 ★
    // ZX361S 是半双工总线舵机, 任何查询 (PRAD/PRTE/PRTV) 都会
    // 【打断】正在执行的 P...T... 行程。所以运动期间不能查询。
    size_t send_age_  = 9999;
    size_t next_id_   = 0;    // 轮询到哪个舵机
    double cmd_prev_[8] = {0}; // 上次下发的命令 (死区用)
    bool first_write_ = true;

    /// 读数合法性: 超出舵机标定范围的一律无效 (STM32 查询失败会返回 0)
    bool validReading(size_t i, uint16_t pos) const;

```

```

double  servoToAngle(size_t i, uint16_t pos) const;
uint16_t angleToServo(size_t i, double a) const;

};

} // namespace arm_hardware

```

实现

```

#include "arm_system_hardware.hpp"

#include <algorithm>
#include <cmath>
#include <sstream>

#include <net/if.h>
#include <hardware_interface/types/hardware_interface_type_values.hpp>

namespace arm_hardware {

// ----- 工具: 解析 "1.0, 2.0, 3.0" -----
static bool splitDoubles(const std::string & s, std::vector<double> & out)
{
    out.clear();
    std::stringstream ss(s);
    std::string tok;
    while (std::getline(ss, tok, ',')) {
        const auto b = tok.find_first_not_of(" \t\r\n");
        const auto e = tok.find_last_not_of(" \t\r\n");
        if (b == std::string::npos) continue;
        try {
            out.push_back(std::stod(tok.substr(b, e - b + 1)));
        } catch (...) {
            return false;
        }
    }
    return !out.empty();
}

static bool getParam(const hardware_interface::HardwareInfo & info,
                    const std::string & key, std::string & out)
{
    auto it = info.hardware_parameters.find(key);
    if (it == info.hardware_parameters.end()) return false;
    out = it->second;
    return true;
}

// ----- on_init: 读参数 + 建 CAN -----
hardware_interface::CallbackReturn ArmSystemHardware::on_init(
    const hardware_interface::HardwareInfo & info)
{
    if (hardware_interface::SystemInterface::on_init(info) !=
        hardware_interface::CallbackReturn::SUCCESS) {
        return hardware_interface::CallbackReturn::ERROR;
    }

    const size_t n = info_.joints.size();

    // ① 读参数
    std::string v;
    if (getParam(info_, "can_interface", v)) can_interface_ = v;
    if (getParam(info_, "cmd_time_ms", v)) cmd_time_ms_ = std::stoi(v);
    if (getParam(info_, "poll_div", v)) poll_div_ = std::max(1, std::stoi(v));
    if (getParam(info_, "send_div", v)) send_div_ = std::max(1, std::stoi(v));

    std::vector<double> d;
    if (!getParam(info_, "servo_ids", v) || !splitDoubles(v, d)) {
        RCLCPP_FATAL(get_logger(), "缺少或无法解析参数 servo_ids");
        return hardware_interface::CallbackReturn::ERROR;
    }
    servo_ids_.clear();
    for (double x : d) servo_ids_.push_back(static_cast<uint8_t>(x));

    auto need = [&](const char * key, std::vector<double> & dst) -> bool {
        std::string s;

```

```

    if (!getParam(info_, key, s) || !splitDoubles(s, dst)) {
        RCLCPP_FATAL(get_logger(), "缺少或无法解析参数 %s", key);
        return false;
    }
    if (dst.size() != n) {
        RCLCPP_FATAL(get_logger(), "参数 %s 有 %zu 项，但关节有 %zu 个",
            key, dst.size(), n);
        return false;
    }
    return true;
};

if (!need("servo_min", servo_min_) ||
    !need("servo_max", servo_max_) ||
    !need("pos_min", pos_min_) ||
    !need("pos_max", pos_max_) ||
    !need("dir", dir_)) {
    return hardware_interface::CallbackReturn::ERROR;
}
if (servo_ids_.size() != n) {
    RCLCPP_FATAL(get_logger(), "servo_ids 有 %zu 项，但关节有 %zu 个",
        servo_ids_.size(), n);
    return hardware_interface::CallbackReturn::ERROR;
}

// ② 建 CAN (完全复用你已有的库)
struct sockaddr_can addr {};
addr.can_family = AF_CAN;
addr.can_ifindex = static_cast<int>(if_nametoindex(can_interface_.c_str()));
if (addr.can_ifindex == 0) {
    RCLCPP_FATAL(get_logger(), "找不到网络接口 %s", can_interface_.c_str());
    return hardware_interface::CallbackReturn::ERROR;
}

std::vector<struct can_filter> filters = {{GETANGLE, CAN_SFF_MASK}};
can_ = std::make_unique<Cansocket_object>(addr, filters);
if (!can_>Can_Socket_Init(can_interface_)) {
    RCLCPP_FATAL(get_logger(), "CAN 初始化失败 (%s): %s",
        can_interface_.c_str(), can_>Can_geterr().c_str());
    return hardware_interface::CallbackReturn::ERROR;
}
servo_ = std::make_unique<ServoCtrl_object>(*can_);

// ③ 初始化缓存
hw_commands_.assign(n, 0.0);
hw_positions_.assign(n, 0.0);
hw_velocities_.assign(n, 0.0);

RCLCPP_INFO(get_logger(),
    "ArmSystemHardware 就绪: %zu 关节, iface=%s, cmd_time=%dms, poll_div=%d",
    n, can_interface_.c_str(), cmd_time_ms_, poll_div_);
RCLCPP_INFO(get_logger(),
    " 下发: 每 %d 个控制周期一次 (~%.0f ms), 舵机到位时间跟随该间隔",
    send_div_, send_div_ * 20.0);
for (size_t i = 0; i < n; ++i) {
    RCLCPP_INFO(get_logger(),
        "[%zu] %-28s servo_id=%u servo=[%.0f, %.0f] pos=[%.4f, %.4f] dir=%.0f",
        i, info_.joints[i].name.c_str(), servo_ids_[i],
        servo_min_[i], servo_max_[i], pos_min_[i], pos_max_[i], dir_[i]);
}
return hardware_interface::CallbackReturn::SUCCESS;
}

// ----- 导出接口 -----
std::vector<hardware_interface::StateInterface>
ArmSystemHardware::export_state_interfaces()
{
    std::vector<hardware_interface::StateInterface> out;
    for (size_t i = 0; i < info_.joints.size(); ++i) {
        out.emplace_back(info_.joints[i].name,
            hardware_interface::HW_IF_POSITION, &hw_positions_[i]);
        out.emplace_back(info_.joints[i].name,
            hardware_interface::HW_IF_VELOCITY, &hw_velocities_[i]);
    }
    return out;
}

std::vector<hardware_interface::CommandInterface>
ArmSystemHardware::export_command_interfaces()

```

```

{
    std::vector<hardware_interface::CommandInterface> out;
    for (size_t i = 0; i < info_.joints.size(); ++i) {
        out.emplace_back(info_.joints[i].name,
            hardware_interface::HW_IF_POSITION, &hw_commands_[i]);
    }
    return out;
}

// ----- 生命周期 -----
hardware_interface::CallbackReturn ArmSystemHardware::on_activate(
    const rclcpp_lifecycle::State &)
{
    const size_t n = info_.joints.size();

    // ① 发 6 条查询
    for (size_t i = 0; i < n; ++i) {
        servo_ -> Statue_get(servo_ids_[i]);
    }

    // ② ★ 按【时间】等，收集到全部或超时为止
    // （原来是 Info_wait(10) 一失败就 break —— 这是主因）
    const auto deadline = std::chrono::steady_clock::now() +
        std::chrono::milliseconds(1000);

    int received = 0;
    while (received < static_cast<int>(n) &&
        std::chrono::steady_clock::now() < deadline) {
        if (servo_ -> Info_wait(20)) ++received;
    }
    if (received < static_cast<int>(n)) {
        RCLCPP_WARN(get_logger(), "只收到 %d/%zu 个舵机回复，用已有数据继续",
            received, n);
    }

    // ③ ★ 读到的位置必须 clamp，且 cmd_prev_ 用同一个口径
    for (size_t i = 0; i < n; ++i) {
        const ServoStatus s = servo_ -> Read_Info(servo_ids_[i]);
        double pos = validReading(i, s.position)
            ? servoToAngle(i, s.position)
            : 0.5 * (pos_min_[i] + pos_max_[i]); // 无效则取中点
        pos = std::clamp(pos, pos_min_[i], pos_max_[i]);
        hw_positions_[i] = pos;
        hw_commands_[i] = pos;
        cmd_prev_[i] = pos; // ★ 关键：和 write() 里 clamp 后的值一致
    }

    RCLCPP_INFO(get_logger(), "已激活，命令初值 = 当前关节角：");
    for (size_t i = 0; i < n; ++i) {
        RCLCPP_INFO(get_logger(), " %-28s %.4f rad",
            info_.joints[i].name.c_str(), hw_commands_[i]);
    }
    return hardware_interface::CallbackReturn::SUCCESS;
}

hardware_interface::CallbackReturn ArmSystemHardware::on_deactivate(
    const rclcpp_lifecycle::State &)
{
    RCLCPP_INFO(get_logger(), "已停用");
    return hardware_interface::CallbackReturn::SUCCESS;
}

// ----- ★ read: 从 CAN 收状态 -----
hardware_interface::return_type ArmSystemHardware::read(
    const rclcpp::Time &, const rclcpp::Duration &)
{
    const size_t n = info_.joints.size();

    // ★★★ 运动期间【不要查询舵机】—— 这是「一卡一卡」的真正根因 ★★★
    //
    // ZX361S 是半双工总线舵机。实测（绕开 ros2_control，裸 CAN 直接测）：
    // 完全不开查询 → 3 秒行程正常走完 sqrt
    // 5 / 20 / 50 Hz 查询 → 全部停在原地 ×
    //
    // 即：任何查询（#P%03dPRAD! / PRTE / PRTV）都会【打断】正在执行的
    // P...T... 行程。于是：
    // read() 查询 → 打断 write() 刚下发的运动 → 舵机停住
    // → 下一周期 JTC 发现没动又发 → 又被下一次查询打断
    // → 舵机走走停停 = 一卡一卡
    //

```

```

// 正确做法：正在运动时用【命令值】当状态（舵机一定会走到），
// 等它停下来之后再查一次真实位置做校准。
++send_age_;
const size_t moving_win = static_cast<size_t>(send_div_) * 6;

if (send_age_ < moving_win)
{
    // 运动期间：用命令值当状态
    for (size_t i = 0; i < n; ++i)
    {
        // 来回过一次舵机量化，让报出来的值和真实分辨率一致
        hw_positions_[i] = servoToAngle(i, angleToServo(i, hw_commands_[i]));
    }
    return hardware_interface::return_type::OK;
}

if (send_age_ == moving_win)
{
    // ★★★ 刚停下来这一拍：先把 6 个舵机【全部】查一遍 ★★★
    //
    // 运动期间没有查询，Info 缓存里还是【运动之前】的旧位置。
    // 如果直接按 poll_div_ 轮询，要 6 个周期才轮完，前几拍读到的
    // 是过期值 —— 实测会造成 hw_positions_ 突跳 1.12 rad，于是：
    // State tolerances failed for joint 2:
    // Position Error: -1.122340, Position Tolerance: 1.000000
    // arm_controller: Aborted due to state tolerance violation
    //
    // 所以这里一次性刷新全部，把过期缓存冲掉，再填状态。
    for (size_t i = 0; i < n; ++i)
    {
        servo->Statue_get(servo_ids_[i]);
    }
    for (int k = 0; k < 80; ++k) // 最多收 80 帧
    {
        if (!servo->Info_wait(10))
        {
            break;
        }
    }
    for (size_t i = 0; i < n; ++i)
    {
        const ServoStatus s = servo->Read_Info(servo_ids_[i]);
        if (validReading(i, s.position))
        {
            hw_positions_[i] = servoToAngle(i, s.position);
        }
        // 无效读数（STM32 查询超时会回 0）：保留上一次的值
    }
    return hardware_interface::return_type::OK;
}

// ① 非阻塞把已经回来的帧全收掉
// ※ 如果 Info_wait 在失败时打日志，这里会刷屏 —— 见下面的注意事项
for (int guard = 0; guard < 16; ++guard) {
    if (!servo->Info_wait(0)) break;
}

// ② 每 poll_div_ 个周期一个舵机（轮转）
if (++tick_ % static_cast<size_t>(poll_div_) == 0) {
    servo->Statue_get(servo_ids_[next_id_]);
    next_id_ = (next_id_ + 1) % n;
}

// ③ 从缓存映射到关节角
// ★ 无效读数保护 ★
// STM32 的 Servo_position_get 在查询超时时返回 0（见 robot_arm.c），
// 而 0 远在舵机标定范围之外。直接用会把状态打飞，触发：
// State tolerances failed for joint 0
// arm_controller: Aborted due to state tolerance violation
// 所以超出标定范围的读数一律丢弃，保留上一次的值。
for (size_t i = 0; i < n; ++i) {
    const ServoStatus s = servo->Read_Info(servo_ids_[i]);
    if (validReading(i, s.position))
    {
        hw_positions_[i] = servoToAngle(i, s.position);
    }
}
return hardware_interface::return_type::OK;
}

```

```

// ----- ★ write: 把命令下发到 CAN -----
hardware_interface::return_type ArmSystemHardware::write(
    const rclcpp::Time &, const rclcpp::Duration & period)
{
    const size_t n = info_.joints.size();
    if (first_write_) {
        for (size_t i = 0; i < n; ++i) {
            cmd_prev_[i] = std::clamp(hw_commands_[i], pos_min_[i], pos_max_[i]);
        }
        first_write_ = false;
        RCLCPP_INFO(get_logger(), "首次 write: 仅同步初值, 不下发命令");
        return hardware_interface::return_type::OK;
    }
    // ★★★ 舵机的「到位时间」必须 ~ 控制周期 ★★★
    //
    // 舵机收到 Position_set(id, pos, T) 时, 会从【当前实际位置】
    // 重新开始 T 毫秒的行程。如果 T 远大于控制周期, 舵机每个
    // 周期只能走  $\Delta \times (\text{period}/T)$ , 实际速度被压到指令速度的
    //  $\text{period}/T$  倍 —— 表现为 JTC 报 PATH_TOLERANCE_VIOLATED(-4)。
    //
    // 所以这里用【控制周期】作为到位时间, cmd_time_ms 只做兜底
    // (首次调用 period 可能为 0)。
    double period_ms = period.seconds() * 1000.0;
    if (period_ms < 1.0)
    {
        period_ms = 20.0; // 兜底: 假设 50 Hz
    }

    // ★★★ 降频下发 —— 解决「一卡一卡」的关键 ★★★
    //
    // 舵机协议 #P%04dT%04d! 是「在 T 毫秒内从当前位置走到 P」,
    // 每收到新命令都会从当前位置【重新开始】这段行程。
    //
    // 每 20ms 发一次, T=20ms: 舵机每 20ms 被打断一次, 永远停在
    // 加速阶段 → 一卡一卡, 平均速度也上不去。
    //
    // 每 send_div_×20ms 发一次, T=send_div_×20ms: 舵机每次都能
    // 走完一段完整行程 → 顺滑。
    //
    // 代价: 跟随滞后最多一个下发间隔 (send_div_=5 时 100ms)。
    // 对舵机这种执行器, 顺滑远比 100ms 的跟随误差重要。
    if (send_div_ > 1 &&
        (++send_tick_ % static_cast<size_t>(send_div_)) != 0)
    {
        return hardware_interface::return_type::OK;
    }

    const uint16_t t_ms = static_cast<uint16_t>(
        std::clamp(period_ms * static_cast<double>(send_div_), 20.0, 400.0));

    constexpr double DEADBAND = 0.005;
    for (size_t i = 0; i < n; ++i) {
        double cmd = hw_commands_[i];
        // 钳到关节限位 (MoveIt2 规划过, 但这里再兜一层)
        cmd = std::clamp(cmd, pos_min_[i], pos_max_[i]);

        if (std::fabs(cmd - cmd_prev_[i]) < DEADBAND) continue;

        const uint16_t pos = angleToServo(i, cmd);
        if (!servo_>Position_set(servo_ids_[i], pos, t_ms)) {
            RCLCPP_WARN_THROTTLE(get_logger(), *get_clock(), 2000,
                "Position_set 失败 servo_id=%u: %s",
                servo_ids_[i], servo_>Servo_errget().c_str());
        }
        cmd_prev_[i] = cmd;
        send_age_ = 0; // ★ 刚下发过 → read() 接下来不再查询, 避免打断
    }
    return hardware_interface::return_type::OK;
}

// ----- 读数合法性 -----
bool ArmSystemHardware::validReading(size_t i, uint16_t pos) const
{
    // STM32 查询失败时会回 0, 超出标定范围的一律当无效
    return pos >= servo_min_[i] && pos <= servo_max_[i];
}

// ----- 映射 (和你 arm_server 里完全一致) -----

```

```

double ArmSystemHardware::servoToAngle(size_t i, uint16_t pos) const
{
    const double sp = servo_max_[i] - servo_min_[i];
    const double ap = pos_max_[i] - pos_min_[i];
    if (sp == 0.0) return pos_min_[i];

    double t = (static_cast<double>(pos) - servo_min_[i]) / sp;
    t = std::clamp(t, 0.0, 1.0); // ★★★ 加这一行
    if (dir_[i] < 0.0) t = 1.0 - t;
    return pos_min_[i] + t * ap;
}

uint16_t ArmSystemHardware::angleToServo(size_t i, double a) const
{
    const double sp = servo_max_[i] - servo_min_[i];
    const double ap = pos_max_[i] - pos_min_[i];
    if (ap == 0.0) return static_cast<uint16_t>(servo_min_[i]);

    double t = (a - pos_min_[i]) / ap;
    if (dir_[i] < 0.0) t = 1.0 - t; // ★ 反射
    t = std::clamp(t, 0.0, 1.0);
    return static_cast<uint16_t>(servo_min_[i] + t * sp);
}

} // namespace arm_hardware

#include <pluginlib/class_list_macros.hpp>
PLUGINLIB_EXPORT_CLASS(arm_hardware::ArmSystemHardware,
                       hardware_interface::SystemInterface)

```

插件声明 (pluginlib)

```

<library path="arm_system_hardware">
  <class name="CanServer/ArmSystemHardware"
        type="arm_hardware::ArmSystemHardware"
        base_class_type="hardware_interface::SystemInterface">
    <description>CAN bus hardware interface for the 5-DOF arm (reuses arm_can)</description>
  </class>
</library>

```

※ 注册到哪个包很关键: `pluginlib_export_plugin_description_file(hardware_interface ...)`。

`hardware_interface::SystemInterface` 定义在 `hardware_interface` 包, `ament` 会把所有注册到它的插件聚合到 `ament_index/resource_index/hardware_interface_pluginlib_plugin/`。

4.4 CMakeLists / package.xml

```

cmake_minimum_required(VERSION 3.10)
project(arm_ik_plugin)

if(NOT CMAKE_CXX_STANDARD)
    set(CMAKE_CXX_STANDARD 17)
endif()
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(ament_cmake REQUIRED)
find_package(moveit_core REQUIRED)
find_package(pluginlib REQUIRED)
find_package(rclcpp REQUIRED)
find_package(tf2_eigen REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(moveit_msgs REQUIRED)

add_library(arm_ik_plugin SHARED
    src/arm_analytic_ik_plugin.cpp
)
target_include_directories(arm_ik_plugin PUBLIC
    $<BUILD_INTERFACE:$<CMAKE_CURRENT_SOURCE_DIR>/include>
    $<INSTALL_INTERFACE:include>
)
ament_target_dependencies(arm_ik_plugin
    moveit_core
    pluginlib
    rclcpp

```

```

    tf2_eigen
    geometry_msgs
    moveit_msgs
)

# kinematics::KinematicsBase 定义在 moveit_core, 所以插件描述注册到 moveit_core
pluginlib_export_plugin_description_file(moveit_core arm_ik_plugin.xml)

install(TARGETS arm_ik_plugin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
  RUNTIME DESTINATION bin
)
install(DIRECTORY include/ DESTINATION include)

ament_package()

```

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>arm_ik_plugin</name>
  <version>0.1.0</version>
  <description>Analytic IK plugin for the 5-DOF arm (base yaw + planar 2R)</description>
  <maintainer email="39530733@qq.com">kickpi</maintainer>
  <license>MIT</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>moveit_core</depend>
  <depend>pluginlib</depend>
  <depend>roscpp</depend>
  <depend>tf2_eigen</depend>
  <depend>geometry_msgs</depend>
  <depend>moveit_msgs</depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>

```

※ 最容易犯的错：删掉已有的 install 规则

```

# ★★★ 加新东西时很容易把这行误删 ★★★
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})

```

删掉的症状：

```

Error: file 'arm_control.launch.py' was not found in the share directory of package 'CanServer'

```

自查命令：

```

grep -cE "install\\(TARGETS|install\\(DIRECTORY" CMakeLists.txt # 少于 3 就要警惕
find install/CanServer/share/CanServer -name "*.py" -o -name "*.yaml"

```

完整的 install 清单：

```

install(TARGETS arm_server can_test DESTINATION lib/${PROJECT_NAME}) # 可执行文件
install(TARGETS arm_system_hardware arm_can LIBRARY DESTINATION lib ...) # 插件 + 库
install(DIRECTORY include/ DESTINATION include) # 头文件
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME}) # ★ 别漏
pluginlib_export_plugin_description_file(hardware_interface arm_system_hardware.xml)

```

第 5 章 控制层：控制器配置

5.1 `joint_trajectory_controller` (JTC) 是什么

一句话：接收一条「带时间戳的关节角序列」，按时间插值，每周期输出一个位置命令。

```
收到 JointTrajectory:
  t=0.0s   joint1 = 0.000
  t=0.1s   joint1 = 0.063
  t=0.2s   joint1 = 0.126
  ...
```

JTC 在每个 20ms 周期：
算出「当前时刻」应该在的位置 → 写到 `command_interface`

它不管硬件是什么，只管按时输出位置。

5.2 ★ 为什么必须按规划组拆控制器

这是今晚踩的一个重要坑（第 9 章坑 6）。

错误做法：一个控制器管全部 6 个关节

```
moveit_controllers.yaml 里 arm_controller.joints = 6 个（含夹爪）
↓
但 MoveIt2 只为 `arm` 组（joint1~joint5）规划 → 发出的轨迹只有 5 个关节
↓
JTC 的 allow_partial_joints_goal 默认 false
↓
直接拒绝
```

症状：

```
[moveit_simple_controller_manager]: arm_controller started execution
[WARN] Goal request rejected
[ERROR] Goal was rejected by server
[ERROR] Failed to send trajectory part 1 of 1 to controller arm_controller
```

正确做法：控制器和 SRDF 的规划组一一对应

```
SRDF 组 "arm"      → arm_controller    joints: joint1~joint5
SRDF 组 "gripper"  → gripper_controller joints: gripper_finger_left_joint
```

规则：控制器负责的关节集合必须和 MoveIt2 的规划组一致。

5.3 `config/ros2_controllers.yaml`

```
# ros2_control 控制器配置
#
# ※ 关键：控制器负责的关节集合必须和 MoveIt2 的【规划组】一致！
#
# 之前 arm_controller 管 6 个关节（含夹爪），但 MoveIt 只为 `arm` 组
#   (joint1~joint5) 规划，发出的轨迹只有 5 个关节 —— 而
#   joint_trajectory_controller 的 allow_partial_joints_goal 默认 false，
#   于是直接拒绝：
#     Goal request rejected / Goal was rejected by server
#
# 现在按 SRDF 的规划组拆开：
#   arm_controller    ← 组 arm      (joint1~joint5)
#   gripper_controller ← 组 gripper  (gripper_finger_left_joint)

controller_manager:
  ros_parameters:
    update_rate: 50          # Hz → 20ms 控制周期
```

```

joint_state_broadcaster:
  type: joint_state_broadcaster/JointStateBroadcaster

arm_controller:
  type: joint_trajectory_controller/JointTrajectoryController

gripper_controller:
  type: joint_trajectory_controller/JointTrajectoryController

# -----
arm_controller:
  ros__parameters:
    joints:
      - joint1
      - joint2
      - joint3
      - joint4
      - joint5

    command_interfaces:
      - position
    state_interfaces:
      - position

    # ★★★ 必须开环 ★★★
    #
    # 为什么：舵机是半双工总线，运动期间【不能查询】（查询会打断行程，
    # 见 ArmSystemHardware::read() 的注释）。所以运动期间报给 JTC 的是
    # 【命令值】而不是真实位置 —— 闭环容差检查就失去意义了。
    #
    # 再加上控制循环偶发 overrun（实测有一次 91ms / 丢5周期），
    # 真实状态会瞬时落后轨迹一小段，闭环检查必然误报：
    #   State tolerances failed for joint 0:
    #   Position Error: -1.092183, Position Tolerance: 1.000000
    #   arm_controller: Aborted due to state tolerance violation
    #
    # 注：Jazzy 里 open_loop_control 已标记 deprecated，而且实测打开
    # 也拦不住 tolerance 报错（报错来自 stop 后的状态跳变，不是闭环引起的）。
    # 真正修掉跳变的是 ArmSystemHardware::read() —— 它会在运动停止的
    # 第一拍把 6 个舵机全部查一遍，冲掉运动期间过期的缓存。
    open_loop_control: false
    allow_partial_joints_goal: false
    allow_nonzero_velocity_at_trajectory_end: false

    # ※ 容差要和「状态反馈延迟」匹配：
    #   read() 轮询周期 = poll_div × update_rate ~ 240ms
    #   轨迹 50Hz 下发，实际位置天然滞后；设太严会一直报
    #   PATH_TOLERANCE_VIOLATED(-4)
    # ※ 容差必须容忍「状态反馈延迟」：
    #   read() 是轮询的，全轴刷新一圈 = poll_div × 关节数 × 控制周期
    #   poll_div=1 时约 120ms；轨迹 50Hz 下发，实际位置天然滞后
    #   容差设太严会偶发 PATH_TOLERANCE_VIOLATED(-4)
    #
    # 这几个值看起来很大，但对「舵机 + 240ms 级反馈」是必要的：
    # 舵机本身的机械滞后 + 卫星式轮询，本来就跟不紧高动态轨迹。
    constraints:
      stopped_velocity_tolerance: 0.10
      goal_time: 3.0
      joint1: {trajectory: 1.0, goal: 0.5}
      joint2: {trajectory: 1.0, goal: 0.5}
      joint3: {trajectory: 1.0, goal: 0.5}
      joint4: {trajectory: 1.0, goal: 0.5}
      joint5: {trajectory: 1.0, goal: 0.5}

# -----
gripper_controller:
  ros__parameters:
    joints:
      - gripper_finger_left_joint

    command_interfaces:
      - position
    state_interfaces:
      - position

    open_loop_control: false          # 同上：Jazzy 已废弃该参数
    allow_partial_joints_goal: false

    constraints:

```

```

stopped_velocity_tolerance: 0.01
goal_time: 1.0
# 夹爪实测停在 0.005 附近（机械结构限制），goal 容差要 > 0.005
gripper_finger_left_joint: {trajectory: 0.010, goal: 0.008}

```

5.4 launch/arm_control.launch.py

```

#!/usr/bin/env python3
"""ros2_control 控制层（无 RViz 版）

robot_state_publisher      URDF → TF + /robot_description
ros2_control_node          controller_manager + 加载 ArmSystemHardware
spawner joint_state_broadcaster
spawner arm_controller     ← 组 arm (joint1~joint5)
spawner gripper_controller ← 组 gripper (gripper_finger_left_joint)

※ 控制器负责的关节集合必须和 MoveIt2 的规划组一致，
   否则 joint_trajectory_controller 会因 allow_partial_joints_goal=false
   拒绝 MoveIt 发来的轨迹（Goal was rejected by server）。
"""
import os
import xacro
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    # ① 展开 URDF（含 <ros2_control> 段）
    xacro_file = os.path.join(
        get_package_share_directory('my_robot_arm_description'),
        'urdf', 'robot_arm.urdf.xacro')
    robot_desc = xacro.process_file(xacro_file).toxml()

    # ② 控制器配置
    ctrl_yaml = os.path.join(
        get_package_share_directory('CanServer'),
        'config', 'ros2_controllers.yaml')

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_desc}],
            output='screen',
        ),

        # ★ 加载硬件接口 + 控制器管理器
        Node(
            package='controller_manager',
            executable='ros2_control_node',
            parameters=[{'robot_description': robot_desc}, ctrl_yaml],
            output='screen',
        ),

        Node(
            package='controller_manager',
            executable='spawner',
            arguments=['joint_state_broadcaster', '-c', '/controller_manager'],
            output='screen',
        ),

        Node(
            package='controller_manager',
            executable='spawner',
            arguments=['arm_controller', '-c', '/controller_manager'],
            output='screen',
        ),

        Node(
            package='controller_manager',
            executable='spawner',
            arguments=['gripper_controller', '-c', '/controller_manager'],
            output='screen',
        ),
    ])

```

四个/五个节点各自的作用：

节点	作用	能省吗
robot_state_publisher	URDF → TF + /robot_description	× 必须
ros2_control_node	controller_manager + 加载硬件插件	× 必须
spawner_joint_state_broadcaster	state_interface → /joint_states	× 必须
spawner_arm_controller	接收 arm 组轨迹	× 必须
spawner_gripper_controller	接收 gripper 组轨迹	× 必须

第 6 章 规划层：MoveIt2 配置

6.1 my_robot_arm_moveit_config 的 6 个文件

```
my_robot_arm_moveit_config/
├── config/
│   ├── my_robot_arm.srdf          规划组 + 末端执行器 + 禁用碰撞对
│   ├── kinematics.yaml           ★ 指向自研 IK 插件
│   ├── joint_limits.yaml         速度/加速度/jerk 限制
│   ├── moveit_controllers.yaml   ★ 把 MoveIt 接到 ros2_control
│   ├── ompl_planning.yaml        规划器 + 适配器
│   └── initial_positions.yaml     move_group 启动初值
└── launch/
    └── arm_moveit_headless.launch.py
```

6.2 config/my_robot_arm.srdf

```
<?xml version="1.0"?>
<!--
  MoveIt2 语义描述 (SRDF)
  规划组、末端执行器、禁用碰撞对

  ※ 相邻连杆必须 disable_collisions, 否则规划永远失败且不告诉你原因
-->
<robot name="my_robot_arm">

  <!-- 虚拟关节: 把机械臂挂到 world 上 (固定) -->
  <virtual_joint name="virtual_joint" type="fixed"
    parent_frame="world" child_link="base_footprint"/>

  <!-- —— 规划组 —— -->
  <group name="arm">
    <chain base_link="base_link" tip_link="link5"/>
  </group>

  <!--
    夹爪: 只列【主动】关节。
    右手指定为 mimic (见 URDF), 所以这里不列 —— 列了会找不到
    ros2_control 的命令接口。
  -->
  <group name="gripper">
    <joint name="gripper_finger_left_joint"/>
  </group>

  <end_effector name="gripper_ee" parent_link="link5"
    group="gripper" parent_group="arm"/>

  <!-- —— 禁用碰撞对 —— -->

  <!-- 直接相邻 (有 joint 连接) -->
  <disable_collisions link1="base_footprint" link2="base_link" reason="Adjacent"/>
  <disable_collisions link1="base_link" link2="link1" reason="Adjacent"/>
  <disable_collisions link1="link1" link2="link2" reason="Adjacent"/>
  <disable_collisions link1="link2" link2="link3" reason="Adjacent"/>
  <disable_collisions link1="link3" link2="link4" reason="Adjacent"/>
  <disable_collisions link1="link4" link2="link5" reason="Adjacent"/>
  <disable_collisions link1="link5" link2="gripper_base" reason="Adjacent"/>
  <disable_collisions link1="gripper_base" link2="gripper_finger_left" reason="Adjacent"/>
  <disable_collisions link1="gripper_base" link2="gripper_finger_right" reason="Adjacent"/>

  <!-- 隔一个 (Setup Assistant 默认也会禁掉, 避免细连杆误报) -->
  <disable_collisions link1="base_footprint" link2="link1" reason="Adjacent"/>
  <disable_collisions link1="base_link" link2="link2" reason="SecondAdjacent"/>
  <disable_collisions link1="link1" link2="link3" reason="SecondAdjacent"/>
  <disable_collisions link1="link2" link2="link4" reason="SecondAdjacent"/>
  <disable_collisions link1="link3" link2="link5" reason="SecondAdjacent"/>
  <disable_collisions link1="link4" link2="gripper_base" reason="SecondAdjacent"/>
  <disable_collisions link1="link5" link2="gripper_finger_left" reason="SecondAdjacent"/>
  <disable_collisions link1="link5" link2="gripper_finger_right" reason="SecondAdjacent"/>

  <!-- 隔两个 (手指这类细长件之间) -->
  <disable_collisions link1="base_footprint" link2="link2" reason="ThirdAdjacent"/>
```

```

<disable_collisions link1="base_link"          link2="link3"          reason="ThirdAdjacent"/>
<disable_collisions link1="link1"              link2="link4"          reason="ThirdAdjacent"/>
<disable_collisions link1="link2"              link2="link5"          reason="ThirdAdjacent"/>
<disable_collisions link1="link3"              link2="gripper_base"   reason="ThirdAdjacent"/>

<!-- 两个手指永远不相撞（它们只是开合） -->
<disable_collisions link1="gripper_finger_left" link2="gripper_finger_right" reason="Never"/>

<!-- gripper 和更远的臂段不会碰 -->
<disable_collisions link1="gripper_base"        link2="base_link"      reason="Never"/>
<disable_collisions link1="gripper_finger_left" link2="link4"          reason="Never"/>
<disable_collisions link1="gripper_finger_right" link2="link4"          reason="Never"/>

</robot>

```

要点：

部分	作用
<code><virtual_joint></code>	把机械臂挂到 <code>world</code> 上
<code><group name="arm"></code>	链 <code>base_link</code> → <code>link5</code>
<code><group name="gripper"></code>	只列主动关节（右指是 <code>mimic</code> ，列了会找不到命令接口）
<code><end_effector></code>	声明夹爪是末端执行器
<code><disable_collisions></code>	★ 相邻连杆必须禁掉，否则规划永远失败且不告诉你原因

6.3 config/kinematics.yaml

```

# IK 求解器配置
#
# ★ 用自研的解析逆解插件，不用 KDL。
#
# 为什么不用 KDL：
#   KDL 是 6 自由度【数值】解。本机械臂只有 5 个转动关节，
#   雅可比是 6x5（欠定），从随机种子出发无法收敛。
#   MoveIt 的目标采样器正是用随机种子调 IK，所以报：
#       RRTConnect.cpp: Unable to sample any valid states for goal tree
#   调大 timeout/attempts 也没用。
#
# 本插件（arm_ik_plugin）：
#   位置逆解 = 底座 yaw（joint1）+ 平面 2R（joint2/3）闭式求解，
#   腕部冗余角 a4 扫描 + 肘部双构型 + 关节限位筛选 + 正解自校验。
#   只保证【位置（x, y, z）】—— 这正是 5-DOF 能完全控制的部分。

arm:
  kinematics_solver: arm_ik_plugin/ArmAnalyticIKPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.05
  kinematics_solver_attempts: 3

# 夹爪是单个棱柱关节，用关节空间目标就够，不需要 IK 求解器。
# （如果在这里给 gripper 配 kinematics_solver，move_group 会尝试加载它，
#  而本插件是按 5 关节机械臂设计的，只会刷一堆告警。）

```

※ 这里指向的是自研插件，不是 KDL。原因见第 7 章。

6.4 config/joint_limits.yaml

```

# 关节速度 / 加速度 / 加加速度限制
#
# MoveIt2 用这些做「时间参数化」；Ruckig 平滑器还额外需要 jerk 限制。
#
# ※ 这几个值直接决定规划出来的轨迹能不能被执行：
#   填太大 → 舵机跟不上，执行时报 PATH_TOLERANCE_VIOLATED
#   填太小 → 规划出来慢得没法用
#
# 实测：joint1 能跑 2 rad/s（0.5 秒走 1.0 rad）。
#   这里按 1.5 rad/s 保守设置，配合 max_velocity_scaling_factor=0.3
#   实际只用到 0.45 rad/s —— 舵机跟得上，走起来也平滑。

```

```

joint_limits:
  joint1:
    has_velocity_limits: true
    max_velocity: 1.5
    has_acceleration_limits: true
    max_acceleration: 3.0
    has_jerk_limits: true
    max_jerk: 30.0
    has_position_limits: true
    min_position: -1.5708
    max_position: 1.5708

  joint2:
    has_velocity_limits: true
    max_velocity: 1.5
    has_acceleration_limits: true
    max_acceleration: 3.0
    has_jerk_limits: true
    max_jerk: 30.0
    has_position_limits: true
    min_position: -1.5708
    max_position: 1.5708

  joint3:
    has_velocity_limits: true
    max_velocity: 1.5
    has_acceleration_limits: true
    max_acceleration: 3.0
    has_jerk_limits: true
    max_jerk: 30.0
    has_position_limits: true
    min_position: -1.5708
    max_position: 1.5708

  joint4:
    has_velocity_limits: true
    max_velocity: 1.5
    has_acceleration_limits: true
    max_acceleration: 3.0
    has_jerk_limits: true
    max_jerk: 30.0
    has_position_limits: true
    min_position: -1.5708
    max_position: 1.5708

  joint5:
    has_velocity_limits: true
    max_velocity: 2.0
    has_acceleration_limits: true
    max_acceleration: 4.0
    has_jerk_limits: true
    max_jerk: 40.0
    has_position_limits: true
    min_position: -3.1416
    max_position: 3.1416

  gripper_finger_left_joint:
    has_velocity_limits: true
    max_velocity: 0.05
    has_acceleration_limits: true
    max_acceleration: 0.2
    has_jerk_limits: true
    max_jerk: 2.0
    has_position_limits: true
    min_position: 0.0
    max_position: 0.020

```

这几个值决定规划出来的轨迹能不能执行：

参数	作用	本项目
<code>max_velocity</code>	关节最大速度	1.5 rad/s (joint5 是 2.0)
<code>max_acceleration</code>	最大加速度	3.0 rad/s ²
<code>max_jerk</code>	最大加加速度	30.0 rad/s ³ (Ruckig 需要)

再配合请求里的 `max_velocity_scaling_factor: 0.2~0.3`，实际只用到 `0.3~0.45 rad/s` —— 舵机跟得上。

6.5 `config/moveit_controllers.yaml`

```
# ★★★ 把 MoveIt2 接到已经跑通的 ros2_control 链路上 ★★★
#
#   MoveIt2 规划完 → 把 JointTrajectory 发给对应控制器
#                       ↓
#                   已经验证过的那条链路 (CAN → STM32 → 舵机)
#
# 【硬件层一行都不用改】
#
# ※ 这里列的每个控制器都必须【有完整定义】，否则 move_group 报：
#     No action namespace specified for controller `xxx`

moveit_controller_manager: moveit_simple_controller_manager/MoveItSimpleControllerManager

moveit_simple_controller_manager:
  controller_names:
    - arm_controller
    - gripper_controller

# 组 arm: joint1~joint5
arm_controller:
  type: FollowJointTrajectory
  action_ns: follow_joint_trajectory
  default: true
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5

# 组 gripper: 只有主动的那个手指 (右指是 mimic)
gripper_controller:
  type: FollowJointTrajectory
  action_ns: follow_joint_trajectory
  default: true
  joints:
    - gripper_finger_left_joint
```

这是「MoveIt2 <-> ros2_control」的桥。

6.6 `config/ompl_planning.yaml`

```
# OMPL 规划器配置 (ROS 2 Jazzy 格式)
#
# ※※ 格式坑 (踩过一次, move_group 直接 abort) ※※
#   Jazzy 的 PlanningPipeline 构造函数对下面这三个参数调用
#   rclcpp::Parameter::as_string_array(), 所以【必须是 YAML 序列】:
#   planning_plugins / request_adapters / response_adapters
#
# 用 ROS 1 风格的折叠字符串 (request_adapters: >- ...) 会抛
# rclcpp::ParameterTypeException → std::terminate → move_group abort
# 报错长这样:
#   MoveItCpp::loadPlanningPipelines → PlanningPipeline::PlanningPipeline
#   → rclcpp::Parameter::as_string_array() → std::terminate
#
# 另外: planning_plugin (单数) 是旧名, Jazzy 用 planning_plugins (复数)。
#
# RK3568 是 4× Cortex-A55, 算力有限。
# RRTConnect 比 RRT* / PRM 快得多, 5-DOF 场景够用。

planning_plugins:
  - ompl_interface/OMPLPlanner

# —— 规划请求适配器 (按顺序执行) ——
# 这些在规划【之前】检查/修正起始状态。
# ※ Jazzy 的名字和旧版不同:
#   旧: FixWorkspaceBounds      → 新: ValidateWorkspaceBounds
#   旧: FixStartStateBounds     → 新: CheckStartStateBounds
#   旧: FixStartStateCollision  → 新: CheckStartStateCollision
request_adapters:
```

```

- default_planning_request_adapters/ResolveConstraintFrames
- default_planning_request_adapters/ValidateWorkspaceBounds
- default_planning_request_adapters/CheckStartStateBounds
- default_planning_request_adapters/CheckStartStateCollision
- default_planning_request_adapters/CheckForStackedConstraints

# —— 规划响应适配器（按顺序执行） ——
# ★ AddRuckigTrajectorySmoothing 让轨迹「丝滑」★
# 它在时间参数化之后再补一次加加速度(jerk)受限的平滑，
# 舵机走起来不会有突兀的启停。
# 如果它导致规划失败（需要 ruckig 库），删掉这一行即可，
# 保留 AddTimeOptimalParameterization 也能用。
response_adapters:
- default_planning_response_adapters/AddTimeOptimalParameterization
# ★ 让轨迹「丝滑」的关键：
# AddTimeOptimalParameterization 只是时间最优（加速度 bang-bang，起停突兀），
# Ruckig 再补一次【加加速度 jerk 受限】的平滑，舵机走起来没有硬启停。
# ※ Ruckig 需要 joint_limits.yaml 里有加速度和 jerk 限制，缺了会报：
# Ruckig error: -100 / extended the trajectory duration to its maximum
# 并把整条规划判为 FAILURE。
- default_planning_response_adapters/AddRuckigTrajectorySmoothing
- default_planning_response_adapters/ValidateSolution
- default_planning_response_adapters/DisplayMotionPath

#
# 规划器
#
planner_configs:
  RRTConnect:
    type: geometric::RRTConnect
    range: 0.0 # 0.0 = 由 setup() 自动决定
  RRT:
    type: geometric::RRT
    range: 0.0
    goal_bias: 0.05
  PRM:
    type: geometric::PRM
    max_nearest_neighbors: 10

arm:
  default_planner_config: RRTConnect
  planner_configs:
    - RRTConnect
    - RRT
    - PRM
# 碰撞检测步长（关节空间比例）。
# 调小 → 检查更细、路径更稳，但更慢。A55 上 0.05 是合理折中。
longest_valid_segment_fraction: 0.05

gripper:
  default_planner_config: RRTConnect
  planner_configs:
    - RRTConnect
  longest_valid_segment_fraction: 0.05

```

※※ 格式坑（第 9 章坑 4）：Jazzy 里这三个参数必须是 YAML 序列，且名字和旧版不同。

```

planning_plugins: # ← 复数！不是 planning_plugin
- ompl_interface/OMPLPlanner

request_adapters: # ← 必须是 list，不能是折叠字符串
- default_planning_request_adapters/ResolveConstraintFrames
- default_planning_request_adapters/ValidateWorkspaceBounds # 旧名 FixWorkspaceBounds
- default_planning_request_adapters/CheckStartStateBounds # 旧名 FixStartStateBounds
- default_planning_request_adapters/CheckStartStateCollision
- default_planning_request_adapters/CheckForStackedConstraints

response_adapters:
- default_planning_response_adapters/AddTimeOptimalParameterization
- default_planning_response_adapters/AddRuckigTrajectorySmoothing # ★ 丝滑靠它
- default_planning_response_adapters/ValidateSolution
- default_planning_response_adapters/DisplayMotionPath

```

用 ROS 1 风格的折叠字符串会直接让 move_group abort:

```
MoveItCpp::loadPlanningPipelines → PlanningPipeline::PlanningPipeline
```

```
→ roscpp::Parameter::as_string_array() → std::terminate
```

6.7 config/initial_positions.yaml

```
# move_group 启动时的初始状态（用来初始化规划场景）
# 全是 0 = 机械臂竖直向上的中位
initial_positions:
  joint1: 0.0
  joint2: 0.0
  joint3: 0.0
  joint4: 0.0
  joint5: 0.0
  gripper_finger_left_joint: 0.0
```

move_group 用它初始化规划场景里的机器人状态。全 0 = 机械臂竖直向上的中位。

6.8 launch/arm_moveit_headless.launch.py

```
#!/usr/bin/env python3
"""MoveIt2 规划层（无 RViz 版）

板子没显示屏，所以这里【不启动 rviz2】。
需要可视化时在 PC 上跑 RViz，通过 DDS 连过来。

启动内容：
    move_group          ← MoveIt2 的核心（IK / 碰撞检测 / 规划 / 时间参数化）
    static TF world→base_footprint

※ 前提：ros2_control 那套要先起来
    ros2 launch CanServer arm_control.launch.py
"""
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from moveit_configs_utils import MoveItConfigsBuilder

def generate_launch_description():
    moveit_config = (
        MoveItConfigsBuilder("my_robot_arm",
                             package_name="my_robot_arm_moveit_config")
        # URDF 用 description 包里那份（不重复维护）
        .robot_description(
            file_path=os.path.join(
                get_package_share_directory("my_robot_arm_description"),
                "urdf", "robot_arm.urdf.xacro"),
            mappings={},
        )
        # ★★★ 只加载 ompl，【必须】带 load_all=False ★★★
        #   load_all 默认为 True，会把 MoveIt 自带的 pipeline
        #   （pilz_industrial_motion_planner / chomp 等）也加进来，
        #   接着 to_moveit_configs() 就会去找 pilz_cartesian_limits.yaml
        #   —— 文件不存在 → ParameterBuilderFileNotFoundError。
        #   我们是 5-DOF，OMPL 的 RRTConnect 足够，不需要 pilz。
        .planning_pipelines(pipelines=["ompl"], load_all=False)
        .to_moveit_configs()
    )

    return LaunchDescription([
        # world → base_footprint 的静态 TF
        Node(
            package="tf2_ros",
            executable="static_transform_publisher",
            arguments=["0", "0", "0", "0", "0", "0",
                     "world", "base_footprint"],
            output="screen",
        ),

        # ★ MoveIt2 核心（纯计算，不需要 X server）
        Node(
            package="moveit_ros_move_group",
```

```

        executable="move_group",
        output="screen",
        parameters=[moveit_config.to_dict()],
    ),

    # × 不启动 rviz2 / moveit_ros_visualization
])

```

※ `load_all=False` 必须带（第 9 章坑 5）：

```
.planning_pipelines(pipelines=["ompl"], load_all=False)
```

`MoveItConfigsBuilder` 的 `load_all` 默认是 `True`，会把 `MoveIt` 自带的 `pilz_industrial_motion_planner` / `chomp` / `stomp` 也加载进来，然后 `to_moveit_configs()` 发现用了 `pilz` 就去找 `pilz_cartesian_limits.yaml` —— 文件不存在直接报错：

```

ParameterBuilderFileNotFoundError:
  "File .../config/pilz_cartesian_limits.yaml doesn't exist"

```

我们只有 5-DOF，OMPL 的 RRTConnect 足够。

6.9 CMakeLists.txt / package.xml

```

cmake_minimum_required(VERSION 3.8)
project(my_robot_arm_moveit_config)

find_package(ament_cmake REQUIRED)

# ★★★ 这三行不能少，否则 launch / config 进不了 install ★★★
install(DIRECTORY config
        DESTINATION share/${PROJECT_NAME})

install(DIRECTORY launch
        DESTINATION share/${PROJECT_NAME})

ament_package()

```

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_robot_arm_moveit_config</name>
  <version>0.1.0</version>
  <description>MoveIt2 configuration for the 5-DOF arm (RK3568 + CAN servos)</description>
  <maintainer email="39530733@qq.com">kickpi</maintainer>
  <license>MIT</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>moveit_ros_move_group</exec_depend>
  <exec_depend>moveit_kinematics</exec_depend>
  <exec_depend>moveit_planners_ompl</exec_depend>
  <exec_depend>moveit_simple_controller_manager</exec_depend>
  <exec_depend>moveit_configs_utils</exec_depend>
  <exec_depend>my_robot_arm_description</exec_depend>
  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>tf2_ros</exec_depend>
  <exec_depend>controller_manager</exec_depend>
  <exec_depend>joint_trajectory_controller</exec_depend>
  <exec_depend>joint_state_broadcaster</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>

```

第 7 章 ★ 自研解析 IK 插件

7.1 为什么 KDL 不行

KDL 是 6 自由度数值解。本机械臂只有 5 个转动关节，雅可比是 6×5 （欠定）。

MoveIt 的目标采样器用【随机种子】反复调 IK，KDL 从随机种子出发无法收敛：

```
[ERROR] RRTConnect.cpp:265 - arm/arm: Unable to sample any valid states for goal tree
```

※ 一个会误导判断的陷阱

/compute_ik 服务用【精确种子】调 IK 时，KDL 会“成功”：

```
① FK sqrt seed=[0.0, 0.4, -0.6, 0.3, 0.0] → 末端 x=+0.0263 y=0.0000 z=+0.3245
② IK sqrt 解回来: ['+0.0000', '+0.4000', '-0.6000', '+0.3000', '+0.0000']
    和 seed 的最大偏差: 0.0000 rad
```

看起来完美 —— 但这是假象：从解出发当然立刻收敛。

真正判断 IK 行不行，要看规划能不能成（那才是随机种子的场景）。

我试过调参数（timeout 0.05→0.2, attempts 3→10）—— 完全没用。

7.2 可用的 IK 求解器

```
grep -rhoP '(?<=<class name=")[^"]*' /opt/ros/jazzy/share/moveit_kinematics/*.xml | sort -u
```

板上只有：

```
kdl_kinematics_plugin/KDLKinematicsPlugin      ← 6-DOF 数值解，本项目不行
srv_kinematics_plugin/SrvKinematicsPlugin      ← 转发到外部服务
cached_ik_kinematics_plugin/CachedKDLKinematicsPlugin
cached_ik_kinematics_plugin/CachedSrvKinematicsPlugin
```

没有 LMA / pick_ik / bio_ik / TRAC-IK。

所以只能自己写。

7.3 KinematicsBase 要实现什么

7 个纯虚函数：

```
// —— IK ——
virtual bool getPositionIK(const geometry_msgs::msg::Pose& ik_pose,
                          const std::vector<double>& ik_seed_state,
                          std::vector<double>& solution,
                          moveit_msgs::msg::MoveItErrorCodes& error_code,
                          const KinematicsQueryOptions& options) const = 0;

virtual bool searchPositionIK(pose, seed, timeout, solution, error_code, options) const = 0;
virtual bool searchPositionIK(pose, seed, timeout, consistency_limits, solution, error_code, options) const = 0;
virtual bool searchPositionIK(pose, seed, timeout, solution, IKCallbackFn, error_code, options) const = 0;
virtual bool searchPositionIK(pose, seed, timeout, consistency_limits, solution, IKCallbackFn, error_code, options) const = 0;

// —— FK ——
virtual bool getPositionFK(link_names, joint_angles, poses) const = 0;

// —— 名字 ——
virtual const std::vector<std::string>& getJointNames() const = 0;
virtual const std::vector<std::string>& getLinkNames() const = 0;
```

initialize 有默认实现 —— 但※※ 这个「默认实现」是个坑（第 9 章坑 7）：

```
// × 错误：调用基类的 initialize
```

```

if (!kinematics::KinematicsBase::initialize(node, robot_model, group_name,
                                           base_frame, tip_frames, search_discretization))
{ return false; }

// 症状:
// [ERROR] IK plugin for group 'arm' relies on deprecated API.
// Please implement initialize(rclcpp::Node::SharedPtr, RobotModel, ...).
// → 永远返回 false, 插件加载不上

// sqrt 正确: 自己设置 node_, 调用 protected 的 storeValues()
node_ = node;
storeValues(robot_model, group_name, base_frame, tip_frames, search_discretization);

```

MoveIt 2.12 中那个基类方法的默认实现只是个「你必须 override」的桩。

7.4 完整代码

头文件

```

#pragma once

#include <limits>
#include <memory>
#include <string>
#include <vector>

#include <moveit/kinematics_base/kinematics_base.hpp>
#include <moveit/robot_model/robot_model.hpp>
#include <moveit/robot_model/joint_model_group.hpp>
#include <moveit/robot_state/robot_state.hpp>
#include <rclcpp/rclcpp.hpp>

namespace arm_ik_plugin
{
/**
 * @brief 5 自由度机械臂的解析逆解插件
 *
 * KDL 是 6 自由度数值解, 对本机械臂 (5 个转动关节) 从随机种子出发
 * 无法收敛, 表现为 MoveIt 报:
 * RRTConnect: Unable to sample any valid states for goal tree
 *
 * 本插件直接用解析法求位置逆解:
 * · 只保证【位置】(x,y,z) —— 这正是 5-DOF 能完全控制的部分
 * · 姿态不参与求解 (末端 roll 由 joint5 提供, pitch/yaw 由 j2~j4 决定)
 * · 解出来先做关节限位检查, 再用正解自校验
 */
class ArmAnalyticIKPlugin : public kinematics::KinematicsBase
{
public:
    ArmAnalyticIKPlugin() = default;

    bool initialize(const rclcpp::Node::SharedPtr & node,
                  const moveit::core::RobotModel & robot_model,
                  const std::string & group_name,
                  const std::string & base_frame,
                  const std::vector<std::string> & tip_frames,
                  double search_discretization) override;

    // —— 必需的 7 个纯虚函数 ——

    bool getPositionIK(const geometry_msgs::msg::Pose & ik_pose,
                     const std::vector<double> & ik_seed_state,
                     std::vector<double> & solution,
                     moveit_msgs::msg::MoveItErrorCodes & error_code,
                     const kinematics::KinematicsQueryOptions & options =
                         kinematics::KinematicsQueryOptions()) const override;

    using kinematics::KinematicsBase::searchPositionIK;

    bool searchPositionIK(const geometry_msgs::msg::Pose & ik_pose,
                        const std::vector<double> & ik_seed_state,
                        double timeout,
                        std::vector<double> & solution,
                        moveit_msgs::msg::MoveItErrorCodes & error_code,

```

```

        const kinematics::KinematicsQueryOptions & options =
            kinematics::KinematicsQueryOptions() const override;

bool searchPositionIK(const geometry_msgs::msg::Pose & ik_pose,
                    const std::vector<double> & ik_seed_state,
                    double timeout,
                    const std::vector<double> & consistency_limits,
                    std::vector<double> & solution,
                    moveit_msgs::msg::MoveItErrorCodes & error_code,
                    const kinematics::KinematicsQueryOptions & options =
                        kinematics::KinematicsQueryOptions() const override;

bool searchPositionIK(const geometry_msgs::msg::Pose & ik_pose,
                    const std::vector<double> & ik_seed_state,
                    double timeout,
                    std::vector<double> & solution,
                    const IKCallbackFn & solution_callback,
                    moveit_msgs::msg::MoveItErrorCodes & error_code,
                    const kinematics::KinematicsQueryOptions & options =
                        kinematics::KinematicsQueryOptions() const override;

bool searchPositionIK(const geometry_msgs::msg::Pose & ik_pose,
                    const std::vector<double> & ik_seed_state,
                    double timeout,
                    const std::vector<double> & consistency_limits,
                    std::vector<double> & solution,
                    const IKCallbackFn & solution_callback,
                    moveit_msgs::msg::MoveItErrorCodes & error_code,
                    const kinematics::KinematicsQueryOptions & options =
                        kinematics::KinematicsQueryOptions() const override;

bool getPositionFK(const std::vector<std::string> & link_names,
                  const std::vector<double> & joint_angles,
                  std::vector<geometry_msgs::msg::Pose> & poses) const override;

const std::vector<std::string> & getJointNames() const override
{
    return joint_names_;
}

const std::vector<std::string> & getLinkNames() const override
{
    return link_names_;
}

private:
    /// 给定腕部绝对角 a4 和肘部构型，解一组关节角；不可达时 q 为空
    void solveArm(double X, double Y, double Z, double a4, int elbow,
                 const std::vector<double> & seed,
                 std::vector<double> & q) const;

    /// ★ 枚举【全部】可行解析解，按离种子的距离升序排列。
    /// 带碰撞回调的 searchPositionIK 会逐个试，谁通过用谁 ——
    /// 否则只返回「离种子最近」的那组，很容易被否掉，
    /// OMPL 就报 Unable to sample any valid states for goal tree.
    void solveAll(double X, double Y, double Z,
                 const std::vector<double> & seed,
                 std::vector<std::vector<double>> & out) const;

    /// 正解（位置）—— 只用来自校验
    void forwardArm(const std::vector<double> & q,
                   double & X, double & Y, double & Z) const;

    bool withinLimits(const std::vector<double> & q) const;

    static double squaredDistance(const std::vector<double> & a,
                                const std::vector<double> & b);

    /// —— 几何常量（单位 m，从 URDF 量得并经 /compute_fk 验证）——
    static constexpr double L1 = 0.0985; // j2 -> j3
    static constexpr double L2 = 0.0950; // j3 -> j4
    static constexpr double L3 = 0.0685; // j4 -> link5 原点
    static constexpr double Z0 = 0.0725; // base -> j2 轴线

    std::vector<std::string> joint_names_;
    std::vector<std::string> link_names_;
};

} // namespace arm_ik_plugin

```

```

#include "arm_ik_plugin/arm_analytic_ik_plugin.hpp"

#include <algorithm>
#include <cmath>

#include <pluginlib/class_list_macros.hpp>
#include <tf2_eigen/tf2_eigen.hpp>

namespace arm_ik_plugin
{
namespace
{
constexpr double kSelfCheckTol = 1e-6; // 正解自校验容差 (m)
constexpr double kSweepStep = M_PI / 30.0; // 腕部绝对角扫描步长 = 6°
constexpr int kSweepHalf = 30; // ±30 步 → ±180°
} // namespace

// ----- 工具 -----
double ArmAnalyticIKPlugin::squaredDistance(const std::vector<double> & a,
                                             const std::vector<double> & b)
{
    double s = 0.0;
    const size_t n = std::min(a.size(), b.size());
    for (size_t i = 0; i < n; ++i)
    {
        const double d = a[i] - b[i];
        s += d * d;
    }
    return s;
}

// ----- initialize -----
bool ArmAnalyticIKPlugin::initialize(
    const rclcpp::Node::SharedPtr & node,
    const moveit::core::RobotModel & robot_model,
    const std::string & group_name,
    const std::string & base_frame,
    const std::vector<std::string> & tip_frames,
    double search_discretization)
{
    // ※ 基类 initialize 之前 node_ 还是空的, 必须用参数 node 打日志
    RCLCPP_INFO(node->get_logger(),
        "ArmAnalyticIKPlugin::initialize: group=%s base=%s tips=%zu",
        group_name.c_str(), base_frame.c_str(), tip_frames.size());

    // ★★★ 这里【不能】调用 kinematics::KinematicsBase::initialize()! ★★★
    //
    // MoveIt 2.12 中该基类方法的默认实现只是个「你必须 override」的桩,
    // 一旦被调用就报:
    //     IK plugin for group 'arm' relies on deprecated API.
    //     Please implement initialize(rclcpp::Node::SharedPtr, RobotModel, ...).
    // 并返回 false —— 插件就永远加载不上。
    //
    // 正确做法: 自己设置 node_, 再调用 protected 的 storeValues()
    // (它负责填 robot_model_ / group_name_ / base_frame_ / tip_frames_)。
    node_ = node;
    storeValues(robot_model, group_name, base_frame, tip_frames, search_discretization);

    if (!robot_model_)
    {
        RCLCPP_ERROR(node->get_logger(), "robot_model_ 为空 —— storeValues 没填上");
        return false;
    }

    const moveit::core::JointModelGroup * jmg =
        robot_model_->getJointModelGroup(group_name_);
    if (!jmg)
    {
        RCLCPP_ERROR(node->get_logger(), "找不到规划组 '%s'", group_name_.c_str());
        return false;
    }

    joint_names_ = jmg->getVariableNames();
    link_names_ = jmg->getLinkModelNames();
}

```

```

if (joint_names_.size() != 5)
{
    RCLCPP_WARN(node_>get_logger(),
        " 组 '%s' 有 %zu 个关节 (本插件为 5 关节机械臂设计) ——"
        "该组求不了逆解, 但初始化照常成功, 以免拖垮别的组。",
        group_name_.c_str(), joint_names_.size());
    return true;
}

RCLCPP_INFO(node_>get_logger(),
    "ArmAnalyticIKPlugin 就绪: 组=%s base=%s tip=%s 关节=%zu",
    group_name_.c_str(), base_frame_.c_str(),
    tip_frames_.empty() ? "?" : tip_frames_[0].c_str(),
    joint_names_.size());
RCLCPP_INFO(node_>get_logger(),
    " 几何: L1=%f L2=%f L3=%f Z0=%f "
    "扫描 %d 个腕部角 ×2 肘部构型",
    L1, L2, L3, Z0, 2 * kSweepHalf + 1);
return true;
}

// ----- 正解 (位置) ----- 与 URDF 逐点验证过, 误差 0.000000 -----
void ArmAnalyticIKPlugin::forwardArm(const std::vector<double> & q,
    double & X, double & Y, double & Z) const
{
    const double a2 = q[1];
    const double a3 = q[1] + q[2];
    const double a4 = q[1] + q[2] + q[3];
    const double r = L1 * std::sin(a2) + L2 * std::sin(a3) + L3 * std::sin(a4);
    Z = Z0 + L1 * std::cos(a2) + L2 * std::cos(a3) + L3 * std::cos(a4);
    X = r * std::cos(q[0]);
    Y = r * std::sin(q[0]);
}

// ----- 关节限位 -----
bool ArmAnalyticIKPlugin::withinLimits(const std::vector<double> & q) const
{
    if (!robot_model_)
    {
        // 退路: 本机机械臂的硬编码限位
        static const double lo[5] = {-1.5708, -1.5708, -1.5708, -1.5708, -3.1416};
        static const double hi[5] = {1.5708, 1.5708, 1.5708, 1.5708, 3.1416};
        for (size_t i = 0; i < q.size() && i < 5; ++i)
        {
            if (q[i] < lo[i] - 1e-9 || q[i] > hi[i] + 1e-9)
            {
                return false;
            }
        }
        return true;
    }

    for (size_t i = 0; i < q.size() && i < joint_names_.size(); ++i)
    {
        const auto & b = robot_model_>getVariableBounds(joint_names_[i]);
        if (!b.position_bounded)
        {
            continue;
        }
        if (q[i] < b.min_position_ - 1e-9 || q[i] > b.max_position_ + 1e-9)
        {
            return false;
        }
    }
    return true;
}

// ----- 解析逆解 (单组) -----
void ArmAnalyticIKPlugin::solveArm(double X, double Y, double Z,
    double a4, int elbow,
    const std::vector<double> & seed,
    std::vector<double> & q) const
{
    q.clear();

    const double q1 = std::atan2(Y, X);
    const double r = std::hypot(X, Y);

```

```

// 把腕部那段 L3（方向 a4）减掉，得到 j4 轴线该在的位置
const double rp = r - L3 * std::sin(a4);
const double zp = (Z - Z0) - L3 * std::cos(a4);

const double d2 = rp * rp + zp * zp;

// ★★★ 注意：a2 / a3 是【绝对角】（相对 +z 轴），不是相对角！★★★
//
// 位置展开：r = L1 sin(a2) + L2 sin(a3)
//           z = L1 cos(a2) + L2 cos(a3)
// 平方相加：d² = L1² + L2² + 2 L1 L2 cos(a3 - a2)
//
// 所以余弦定理给出的是 cos(a3 - a2)，【不是】cos(a3)。
// （一开始写成 cos(a3)，正解自校验就把它挡下来了，误差 ~1e-2。）
const double c = (d2 - L1 * L1 - L2 * L2) / (2.0 * L1 * L2); // = cos(a3 - a2)
if (std::abs(c) > 1.0)
{
    return; // 该 a4 下不可达
}

const double dtheta = static_cast<double>(elbow) *
    std::acos(std::clamp(c, -1.0, 1.0)); // = a3 - a2 = q3
const double a2 = std::atan2(rp, zp) -
    std::atan2(L2 * std::sin(dtheta), L1 + L2 * std::cos(dtheta));
const double a3 = a2 + dtheta;

const double q5 = seed.size() > 4 ? seed[4] : 0.0; // roll 不影响位置，沿用种子
q = {q1, a2, dtheta, a4 - a3, q5};
}

// ----- 解析逆解（全部可行解，按离种子的距离排序） -----
void ArmAnalyticIKPlugin::solveAll(double X, double Y, double Z,
    const std::vector<double> & seed,
    std::vector<std::vector<double>> & out) const
{
    out.clear();

    // 腕部绝对角候选：种子附近优先，然后 ±180° 扫一遍
    const double seed_a4 = seed[1] + seed[2] + seed[3];

    auto tryOne = [&](double a4, int elbow)
    {
        {
            std::vector<double> q;
            solveArm(X, Y, Z, a4, elbow, seed, q);
            if (q.size() != 5)
            {
                return;
            }
            if (!withinLimits(q))
            {
                return;
            }
        }

        // ★ 自校验：用正解确认真的到位（防公式 / 符号错误）
        double fx = 0.0, fy = 0.0, fz = 0.0;
        forwardArm(q, fx, fy, fz);
        const double err = std::sqrt((fx - X) * (fx - X) +
            (fy - Y) * (fy - Y) +
            (fz - Z) * (fz - Z));

        if (err > kSelfCheckTol)
        {
            return;
        }

        out.push_back(q);
    };

    tryOne(seed_a4, +1);
    tryOne(seed_a4, -1);
    for (int i = -kSweepHalf; i <= kSweepHalf; ++i)
    {
        if (i == 0)
        {
            continue; // 已经试过种子值
        }
        const double a4 = i * kSweepStep;
        tryOne(a4, +1);
        tryOne(a4, -1);
    }
}

```

```

std::sort(out.begin(), out.end(),
    [&](const std::vector<double> & a, const std::vector<double> & b)
    {
        return squaredDistance(a, seed) < squaredDistance(b, seed);
    });
}

// ----- getPositionIK -----
bool ArmAnalyticIKPlugin::getPositionIK(
    const geometry_msgs::msg::Pose & ik_pose,
    const std::vector<double> & ik_seed_state,
    std::vector<double> & solution,
    moveit_msgs::msg::MoveItErrorCodes & error_code,
    const kinematics::KinematicsQueryOptions & /*options*/) const
{
    solution.clear();

    if (ik_seed_state.size() < 5)
    {
        error_code.val = moveit_msgs::msg::MoveItErrorCodes::NO_IK_SOLUTION;
        return false;
    }

    std::vector<std::vector<double>> sol;
    solveAll(ik_pose.position.x, ik_pose.position.y, ik_pose.position.z,
        ik_seed_state, sol);

    if (sol.empty())
    {
        error_code.val = moveit_msgs::msg::MoveItErrorCodes::NO_IK_SOLUTION;
        return false;
    }

    solution = sol.front(); // 已按离种子的距离排好序
    error_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
    return true;
}

// ----- searchPositionIK (不带回调) -----
bool ArmAnalyticIKPlugin::searchPositionIK(
    const geometry_msgs::msg::Pose & ik_pose,
    const std::vector<double> & ik_seed_state,
    double /*timeout*/,
    std::vector<double> & solution,
    moveit_msgs::msg::MoveItErrorCodes & error_code,
    const kinematics::KinematicsQueryOptions & options) const
{
    return getPositionIK(ik_pose, ik_seed_state, solution, error_code, options);
}

bool ArmAnalyticIKPlugin::searchPositionIK(
    const geometry_msgs::msg::Pose & ik_pose,
    const std::vector<double> & ik_seed_state,
    double /*timeout*/,
    const std::vector<double> & /*consistency_limits*/,
    std::vector<double> & solution,
    moveit_msgs::msg::MoveItErrorCodes & error_code,
    const kinematics::KinematicsQueryOptions & options) const
{
    return getPositionIK(ik_pose, ik_seed_state, solution, error_code, options);
}

// ----- searchPositionIK (带碰撞回调) -----
//
// ★ 关键：目标采样器会带一个回调（通常是碰撞检查）。
// 只返回「离种子最近」的那一组解，很可能被回调否掉，
// 于是 OMPL 报：Unable to sample any valid states for goal tree。
// 所以要【把所有可行解逐个喂给回调】，谁先通过就用谁。
// （KDL 用的是「多次随机重启」，这里是「穷举全部解析解」，更彻底。）
//
bool ArmAnalyticIKPlugin::searchPositionIK(
    const geometry_msgs::msg::Pose & ik_pose,
    const std::vector<double> & ik_seed_state,
    double /*timeout*/,
    std::vector<double> & solution,
    const kinematics::KinematicsBase::IKCallbackFn & solution_callback,
    moveit_msgs::msg::MoveItErrorCodes & error_code,
    const kinematics::KinematicsQueryOptions & /*options*/) const
{

```

```

solution.clear();

if (ik_seed_state.size() < 5)
{
    error_code.val = moveit_msgs::msg::MoveItErrorCodes::NO_IK_SOLUTION;
    return false;
}

std::vector<std::vector<double>> sol;
solveAll(ik_pose.position.x, ik_pose.position.y, ik_pose.position.z,
        ik_seed_state, sol);

if (sol.empty())
{
    error_code.val = moveit_msgs::msg::MoveItErrorCodes::NO_IK_SOLUTION;
    return false;
}

if (!solution_callback)
{
    solution = sol.front();
    error_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
    return true;
}

moveit_msgs::msg::MoveItErrorCodes cb_code;
for (const auto & q : sol)
{
    cb_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
    solution_callback(ik_pose, q, cb_code);
    if (cb_code.val == moveit_msgs::msg::MoveItErrorCodes::SUCCESS)
    {
        solution = q;
        error_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
        return true;
    }
}

error_code.val = moveit_msgs::msg::MoveItErrorCodes::NO_IK_SOLUTION;
return false;
}

bool ArmAnalyticIKPlugin::searchPositionIK(
    const geometry_msgs::msg::Pose & ik_pose,
    const std::vector<double> & ik_seed_state,
    double timeout,
    const std::vector<double> & /*consistency_limits*/,
    std::vector<double> & solution,
    const kinematics::KinematicsBase::IKCallbackFn & solution_callback,
    moveit_msgs::msg::MoveItErrorCodes & error_code,
    const kinematics::KinematicsQueryOptions & options) const
{
    return searchPositionIK(ik_pose, ik_seed_state, timeout, solution,
                           solution_callback, error_code, options);
}

// ----- getPositionFK: 交给 RobotModel, 转换到 base_frame -----
bool ArmAnalyticIKPlugin::getPositionFK(
    const std::vector<std::string> & link_names,
    const std::vector<double> & joint_angles,
    std::vector<geometry_msgs::msg::Pose> & poses) const
{
    poses.clear();
    if (!robot_model_)
    {
        return false;
    }

    const moveit::core::JointModelGroup * jmg =
        robot_model_>getJointModelGroup(group_name_);
    if (!jmg)
    {
        return false;
    }

    moveit::core::RobotState state(robot_model_);
    state.setToDefaultValues();
    state.setJointGroupPositions(jmg, joint_angles);
    state.updateLinkTransforms();

```

```

// 模型根坐标系 → base_frame_ (本机机械臂是 base_footprint → base_link)
const Eigen::Isometry3d T_base = state.getGlobalLinkTransform(base_frame_);

for (const auto & name : link_names)
{
    if (!robot_model_>hasLinkModel(name))
    {
        poses.clear();
        return false;
    }
    poses.push_back(tf2::toMsg(T_base.inverse() * state.getGlobalLinkTransform(name)));
}
return true;
}

} // namespace arm_ik_plugin

PLUGINLIB_EXPORT_CLASS(arm_ik_plugin::ArmAnalyticIKPlugin,
                        kinematics::KinematicsBase)

```

插件声明

```

<library path="arm_ik_plugin">
  <class name="arm_ik_plugin/ArmAnalyticIKPlugin"
        type="arm_ik_plugin::ArmAnalyticIKPlugin"
        base_class_type="kinematics::KinematicsBase">
    <description>
      5 自由度机械臂的解析逆解插件。
      位置逆解用「底座 yaw + 平面 2R」闭式求解，腕部冗余角扫描 +
      肘部双构型 + 关节限位筛选 + 正解自校验。
      解决 KDL (6-DOF 数值解) 在本机械臂上无法收敛的问题。
    </description>
  </class>
</library>

```

CMakeLists / package.xml

```

cmake_minimum_required(VERSION 3.10)
project(arm_ik_plugin)

if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 17)
endif()
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(ament_cmake REQUIRED)
find_package(moveit_core REQUIRED)
find_package(pluginlib REQUIRED)
find_package(rclcpp REQUIRED)
find_package(tf2_eigen REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(moveit_msgs REQUIRED)

add_library(arm_ik_plugin SHARED
  src/arm_analytic_ik_plugin.cpp
)
target_include_directories(arm_ik_plugin PUBLIC
  ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
  ${INSTALL_INTERFACE:include}
)
ament_target_dependencies(arm_ik_plugin
  moveit_core
  pluginlib
  rclcpp
  tf2_eigen
  geometry_msgs
  moveit_msgs
)

# kinematics::KinematicsBase 定义在 moveit_core，所以插件描述注册到 moveit_core
pluginlib_export_plugin_description_file(moveit_core arm_ik_plugin.xml)

install(TARGETS arm_ik_plugin

```

```

LIBRARY DESTINATION lib
ARCHIVE DESTINATION lib
RUNTIME DESTINATION bin
)
install(DIRECTORY include/ DESTINATION include)

ament_package()

```

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>arm_ik_plugin</name>
  <version>0.1.0</version>
  <description>Analytic IK plugin for the 5-DOF arm (base yaw + planar 2R)</description>
  <maintainer email="39530733@qq.com">kickpi</maintainer>
  <license>MIT</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>moveit_core</depend>
  <depend>pluginlib</depend>
  <depend>rcpp</depend>
  <depend>tf2_eigen</depend>
  <depend>geometry_msgs</depend>
  <depend>moveit_msgs</depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>

```

7.5 三个让它在工程上真正可用的设计

① ★ 正解自校验（最重要）

```

// ★ 自校验：用正解确认真的到位（防公式 / 符号错误）
double fx = 0.0, fy = 0.0, fz = 0.0;
forwardArm(q, fx, fy, fz);
const double err = std::sqrt((fx - X) * (fx - X) +
                             (fy - Y) * (fy - Y) +
                             (fz - Z) * (fz - Z));
if (err > kSelfCheckTol) // 1e-6
{
  return;
}

```

这 8 行是整个插件最有价值的部分。

它把「公式写错」从「偶发的定位错误」变成「立刻失败」。本项目就是靠它抓出了 $\cos(a3)$ vs $\cos(a3-a2)$ 的错误（第 9 章坑 13）。

代价：每个候选解多一次正解（十几次浮点运算），可忽略。

② 腕部冗余角扫描 + 肘部双构型

5-DOF 对位置是冗余的（3 个 pitch 关节只管 2 个位置自由度），所以：

```

// 腕部绝对角候选：种子附近优先，然后 ±180° 扫一遍
const double seed_a4 = seed[1] + seed[2] + seed[3];
tryOne(seed_a4, +1);
tryOne(seed_a4, -1);
for (int i = -kSweepHalf; i <= kSweepHalf; ++i) { // ±30 步 × 6° = ±180°
  if (i == 0) continue;
  const double a4 = i * kSweepStep;
  tryOne(a4, +1);
  tryOne(a4, -1);
}

```

共 124 次求解，全部按「离种子的距离」排序后返回。

③ ★ 带碰撞回调时遍历全部解析解

```
moveit_msgs::msg::MoveItErrorCodes cb_code;
for (const auto & q : sols)
{
    cb_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
    solution_callback(ik_pose, q, cb_code);
    if (cb_code.val == moveit_msgs::msg::MoveItErrorCodes::SUCCESS)
    {
        solution = q;
        error_code.val = moveit_msgs::msg::MoveItErrorCodes::SUCCESS;
        return true;
    }
}
```

这是 10mm 容差能不能过的关键。

只返回「离种子最近」那一组时，很容易被碰撞检查否掉 → OMPL 采不到目标点：

```
[ERROR] RRTConnect.cpp:265 - arm/arm: Unable to sample any valid states for goal tree
```

KDL 用的是「多次随机重启」，这里是「穷举全部解析解」—— 对 5-DOF 更彻底。

第 8 章 ★ 测试流程

8.1 为什么要分层验证

这套系统有 6 层，任何一层出问题表现都可能是「机械臂不动」。

如果一上来就跑 MoveIt2，失败了根本不知道是哪层的问题：

三维坐标规划失败 → 是 IK？是规划器？是控制器？是 CAN？是舵机？

所以从下往上，一层一层确认。每层的验证方法都是「能独立判断这层对不对」的。

L0	CAN 链路	—————	能不能收发帧
L1	硬件接口	—————	on_init 参数对不对、write 能不能下发
L2	控制器	—————	控制器 active 吗、state 能读到吗
L3	运动学	—————	模型对不对
L4	关节轨迹	—————	手发轨迹机械臂能不能动
L5	规划	—————	move_group 能不能规划出轨迹
L6	三维坐标端到端	—————	给坐标能不能到达
L7	大幅 + 顺滑度	—————	全工作空间 + 平滑性

8.2 L0 · CAN 链路

目标：确认 RK3568 和 STM32 能互相收发。

```
# ① 接口状态
ip -details link show can0
# 期望: state UP, can state ERROR-ACTIVE, bitrate 250000

# ② 抓包 + 发一帧
candump can0 &
cansend can0 104#0002000000000000    # 查舵机 0 状态

# 期望:
# can0 104 [8] 00 02 00 00 00 00 00  ← 我发的
# can0 200 [8] 00 DC 05 13 46 00 00  ← STM32 回的 (位置 1500, 温 19, 压 70)
```

判读

现象	含义
只有 104，没有 200	STM32 没回应 → 没上电 / 固件没跑 / 死机
cansend 报 No buffer space available	没有节点 ACK → 同上
can state ERROR-PASSIVE	同上 (error-pass 1 说明进过 error-passive)
candump 什么都没有	总线上没帧 (注意 CAN 是被动的，没人查询时不会有帧)

唤醒总线的正确方法

```
# 查询帧
cansend can0 104#0002000000000000

# 或者用 Python 裸 SocketCAN (后面所有测量脚本都用这个)
```

8.3 L1 · 硬件接口

目标：确认 ArmSystemHardware 加载成功、参数读对、能收到真实位置。

```
ros2 launch CanServer arm_control.launch.py
```

期望日志：

```
[controller_manager]: Loading hardware 'MyRobotArm'
[controller_manager]: Loaded hardware 'MyRobotArm' from plugin 'CanServer/ArmSystemHardware'
[ArmSystemHardware]: ArmSystemHardware 就绪: 6 关节, iface=can0, cmd_time=150ms, poll_div=3
[ArmSystemHardware]: 下发: 每 5 个控制周期一次 (~100 ms), 舵机到位时间跟随该间隔
[ArmSystemHardware]: [0] joint1 servo_id=0 servo=[500,2500] pos=[-1.5708,1.5708] dir=+1
... (6 行)
[controller_manager]: Successful initialization of hardware 'MyRobotArm'
[ArmSystemHardware]: 已激活, 命令初值 = 当前关节角:
[ArmSystemHardware]: joint1 -0.0123 rad
...
[ArmSystemHardware]: 首次 write: 仅同步初值, 不下发命令    ← ★ 启动保护
```

两个最关键的行: - 已激活, 命令初值 = 当前关节角 → 下面 6 个值应该接近机械臂的真实姿态 - 首次 write: 仅同步初值, 不下发命令 → 看到这句说明启动保护生效了

※ 如果读到 ± 1.571 / ± 3.142 (全是限位值) → CAN 没通, 读到的是无效数据。

8.4 L2 · 控制器

```
# ① 控制器状态
ros2 control list_controllers
# 期望: arm_controller / gripper_controller / joint_state_broadcaster 都 active

# ② 硬件接口
ros2 control list_hardware_interfaces
# 期望: 6 个 command interface [available] [claimed]
#       12 个 state interface (6 position + 6 velocity)

# ③ 能读到真实位置
ros2 topic echo /joint_states --field position
# 手动慢慢转动关节, 值应该在 ~360ms 内跟上 (poll_div=3)

# ④ 频率
ros2 topic hz /joint_states
# 期望: ~50 Hz

# ⑤ CAN 上有真实帧
candump can0
# 期望: 104 [8] xx 02 ... ← write 发的查询
#       200 [8] xx DC 05 ... ← read 收的回复
```

5 个状态的含义

状态	含义	正常吗
<code>[available]</code>	接口存在	sqrt
<code>[claimed]</code>	已被控制器占用	sqrt
<code>unavailable</code>	接口不可用	×
<code>active</code>	控制器在跑	sqrt
<code>inactive</code>	控制器已加载但没激活	※

8.5 L3 · 运动学

见第 3.3 节。用 `kin_check.py` 对 10 组随机关节角逐点验证, 误差必须 $< 1e-4$ m。

```
ros2 launch my_robot_arm_moveit_config arm_moveit_headless.launch.py &
python3 test/kin_check.py
```

期望: 最大误差: `ysign=+1` → 0.000000 m

※ 这层不过, 后面全部白搭。

8.6 L4 · 关节轨迹 (先用手发的方式让它动)

目标：不走规划，直接给关节角，确认机械臂能动、方向对、精度够。

```
python3 test/send_traj.py --limits          # 看限位表
python3 test/send_traj.py 0.3 - - - - 3    # 只动 joint1，其余保持
python3 test/send_traj.py 0 0.3 0 0 0 3    # 只动 joint2
python3 test/send_traj.py 0 0 0 0 0 0.015 1 # 夹爪
```

- 表示「保持当前位置不动」，避免每次都把其他关节拽到 0。

send_traj.py 会先做范围自检，超限直接拦住不发送：

× 命令超出关节限位，已中止（没有发送）				
关节		你给的	合法范围	超出
gripper_finger_left_joint		1.0000	[0.0000, 0.0200]	+0.9800

六个关节逐个验证方向

关节	dir	命令 +0.3 时实机应该
joint1	+1	往正面转
joint2	+1	往正面转
joint3	-1	★ 往反方向
joint4	-1	★ 往反方向
joint5	+1	往正面转
gripper	-1	一个方向是「开」

方向反了 → 改 robot_arm.ros2_control.xacro 的 dir + demo/send_traj.py 的 DIRS，保持一致。

期望输出

```
(joint2, joint3, joint4, joint5, gripper_finger_left_joint 保持当前位置)

关节          当前      目标      移动      速度
joint1        0.2969   0.4000   +0.1031   +0.052 rad/s
...

sqrt_error_code = 0 (SUCCESSFUL)
```

8.7 L5 · 规划层

目标：确认 move_group 起来了、IK 能加载、能规划。

```
ros2 launch my_robot_arm_moveit_config arm_moveit_headless.launch.py
```

期望日志：

```
[move_group]: ArmAnalyticIKPlugin::initialize: group=arm base=base_link tips=1
[move_group]: storeValues OK
[move_group]: 组 'arm' 有 5 个变量、5 个 link:
[move_group]: [0] joint1 ... [4] joint5
[move_group]: ArmAnalyticIKPlugin 就绪: 组=arm base=base_link tip=link5 关节=5
[move_group]: 几何: L1=0.0985 L2=0.0950 L3=0.0685 Z0=0.0725 扫描 61 个腕部角 ×2 肘部构型
[planning_pipeline]: Successfully loaded planner 'OMPL'
[move_group]: Loaded adapter 'default_planning_request_adapters/...' ×5
[move_group]: Loaded adapter 'default_planning_response_adapters/...' ×4
[move_group]: You can start planning now!
```

※ 如果看到 IK plugin ... relies on deprecated API → 第 9 章坑 7。

分层诊断工具 cart_probe.py

当规划失败时，用这个脚本做三种约束的对比：

```
python3 test/cart_probe.py
```

输出：

```
V0 参考状态合法性      valid=True  contacts=0      ← 目标姿态本身会不会自碰撞
V1/V2/V3 规划对比
V1 纯位置 10mm          sqrt 19 点 1.71s
V2 纯位置 50mm          sqrt 47 点 4.56s
V3 位置10mm + 姿态±0.1  sqrt 25 点 2.35s
```

这一招能快速区分「IK 问题」还是「规划器问题」还是「目标不可达」。

moveit_test.py (FK/IK/关节规划/三维坐标规划)

```
python3 test/moveit_test.py      # 只 FK + IK
python3 test/moveit_test.py plan  # 加关节空间规划
python3 test/moveit_test.py cart  # 加三维坐标规划
```

期望：

```
(1) FK OK   seed=[0.0, 0.4, -0.6, 0.3, 0.0]
    tip pos  x=+0.0263 y=+0.0000 z=+0.3245
(2) IK OK   ['+0.0000', '+0.4000', '-0.6000', '+0.3000', '+0.0000'] maxdiff 0.0000 rad
(3) joint-space plan OK   17 points, 1.58s
(4) CARTESIAN target x=+0.0401 y=+0.0170 z=+0.3151
    plan OK   14 points, 1.23s
    reached  x=+0.0405 y=+0.0129 z=+0.3229
    error    8.9 mm   OK (<10mm)
```

8.8 L6 · 三维坐标端到端（真正驱动机械臂）

```
python3 test/moveit_exec2.py      # 10 个目标
python3 test/moveit_exec2.py dry   # 只规划不执行
python3 test/moveit_exec2.py sweep # 更密的扫描
```

脚本逻辑：

- ① 读当前 /joint_states
- ② 先走到 ready 姿态（远离奇异） ← ★ 关键：机械臂完全伸直时是奇异位形
- ③ 对每个目标：
 - 重读当前状态当起点 ← ★ 不能用缓存的起点
 - 用「当前关节角 + 小偏移」做 FK 得到【一定可达】的三维坐标
 - 用位置约束（球形容差 10mm）调 /plan_kinematic_path
 - 调 /execute_trajectory 真正执行
 - 等机械臂停稳（3s）后读回，用 FK 算实际末端位置，报误差

期望输出：

```
→ 先走到 ready 姿态 ['+0.00', '+0.45', '-0.70', '+0.35', '+0.00']（远离奇异）
执行 code=1
ready 末端: x=+0.0325 y=+0.0006 z=+0.3212

目标      三维坐标      规划  执行  实测误差
j1 +0.20    (+0.029, +0.005, +0.322)  OK   OK   14.4 mm
...
★ 规划 10/10  执行 10/10  末端误差 平均 11.9 mm
```

判读要点

指标	合格线
规划成功率	100%
执行成功率	100%（有失败就要查）
末端误差	8~30 mm（取决于姿态，见第 11 章）

8.9 L7 · 大幅运动 + 顺滑度

大幅运动

```
python3 test/moveit_big.py
```

幅度: joint1 ± 1.30 rad ($\pm 75^\circ$) , joint2 到 1.45, joint5 ± 2.5

期望: 规划 8/8, 执行 8/8

顺滑度 —— 用裸 CAN 测量

这是唯一能拿到真实运动曲线的方法（原因见第 9 章坑 9、坑 10）。

```
kill -f ros2_control_node      # ★ 必须先停掉，避免干扰
python3 test/can_smooth.py home 4  # 归位到关节中点，腾出运动空间
python3 test/can_smooth.py direct 6  # 发一条 T=6s 的命令，20Hz 采样
```

期望输出：

```
位移 +0.792 rad 样本 169 运动段 131
平均速度 +0.129 峰值 0.160 标准差 0.017
★ CV 0.133 停顿占比 0.0% 速度反向 0 次
```

判读：

指标	含义	合格线
CV（变异系数）	标准差/均值，速度抖动	< 0.3 算顺滑，> 0.6 就是走走停停
停顿占比	速度接近 0 的样本比例	应为 0%
速度反向次数	速度变号的次数	应为 0（走走停停会反复跳）
位移	实际走了多少	应等于命令值（ $\pm 1\%$ ）

另一种测量（通过 ROS 侧）

```
python3 test/smooth_check.py 0.8 6
```

※ 但这个的分辨率受 poll_div 限制（poll_div=3 时 /joint_states 每 360ms 才刷新一次关节1），看不到 360ms 以下的抖动。所以现场的体感比这个数字准。

8.10 测试脚本清单

脚本	用途	何时用
kin_check.py	校验运动学模型（FK 逐点比对）	改 URDF / 改几何常量后
send_traj.py	手发关节轨迹（带范围自检）	L4，底层调试
cart_probe.py	三维坐标规划：3 种约束对比	L5，规划失败时定位
moveit_test.py	FK / IK / 关节规划 / 三维坐标规划	L5
moveit_exec2.py	★ 三维坐标 → 规划 → 执行 → 实测误差	L6，主验收
moveit_big.py	大幅运动（ $\pm 75^\circ$ ）	L7
can_smooth.py	★ 裸 CAN 测真实运动曲线 + 顺滑度	L7，顺滑度
smooth_check.py	通过 ROS 侧量顺滑度（分辨率有限）	L7

第 9 章 ★ 问题与解决 (13 个)

每个问题都按 现象 → 定位方法 → 根因 → 修复 → 证据 组织。按发现顺序编号。

坑 1: `install(DIRECTORY ...)` 被删 → `launch` 找不到

现象

```
Error: file 'arm_control.launch.py' was not found in the share directory of package 'CanServer'
```

定位

```
find install/CanServer/share/CanServer -name "*.py" -o -name "*.yaml"
# → 只有 arm_system_hardware.xml, 没有 launch/ 和 config/
grep -n "install(" CMakeLists.txt
# → 只有 install(TARGETS ...), 没有 install(DIRECTORY ...)
```

根因

改 `CMakeLists.txt` 加硬件插件时, 把原来那行误删了:

```
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME}) # ← 被删
```

修复

加回去, 然后重编。如果 `colcon` 说 "no changes", 强制重配:

```
rm -rf build/CanServer install/CanServer
colcon build --packages-select CanServer --symlink-install
```

自查命令 (以后改 `CMakeLists` 都跑一下)

```
grep -cE "install\\(TARGETS|install\\(DIRECTORY" CMakeLists.txt # 少于 3 就要警惕
```

坑 2: `robot_arm.ros2_control.xacro` 第 1 行有空格

现象

```
XML parsing error: XML or text declaration not at start of entity: line 1, column 1
when processing file: .../robot_arm.ros2_control.xacro
```

定位

```
head -c 8 robot_arm.ros2_control.xacro | xxd -p
# → 203c3f786d6c2076
# ↑ 0x20 = 空格! 然后是 <?xml v
```

根因

XML 规范要求 `<?xml?>` 声明必须是文件的第一个字符, 前面不能有任何空白。

修复

```
sed -i '1s/^[[:space:]]*//' robot_arm.ros2_control.xacro
head -c 8 robot_arm.ros2_control.xacro | xxd -p # 期望 3c3f786d6c207665
```

坑 3: `can0` 没 UP → 全部 Write 失败

现象

```
Write : fail to write!
```

```
Position_set 失败 servo_id=0: Pos_set : CanERROR!
```

定位

```
ip -details link show can0 | head -3
# → state DOWN
cansend can0 104#0102000000000000
# → write: Network is down
```

根因

`if_nametoindex()` 只看「接口是否存在」，不看 UP/DOWN。所以 `on_init` 里的 `Can_Socket_Init` 会成功，但所有读写都失败。

更危险的变体：CAN 没通时 `/joint_states` 读到的全是限位值：

```
起始关节角: ['-1.571', '-1.571', '+1.571', '+1.571', '-3.142'] ← 一眼假
```

修复

```
sudo ip link set can0 type can bitrate 250000 triple-sampling on
sudo ip link set can0 up
```

根治：做成开机自启（见第 2.2 节）。

坑 4：ompl_planning.yaml 用了 ROS 1 格式 → move_group 直接 abort

现象

`move_group` 启动时直接崩溃，没有任何有用的错误信息：

```
[move_group-2] Stack trace (most recent call last):
[move_group-2] #11 libmoveit_cpp.so, in MoveItCpp::MoveItCpp::loadPlanningPipelines(...)
[move_group-2] #10 libmoveit_planning_pipeline_interfaces.so, in createPlanningPipelineMap(...)
[move_group-2] #9 libmoveit_planning_pipeline.so, in PlanningPipeline::PlanningPipeline(...)
[move_group-2] #7 libroscpp.so, in roscpp::Parameter::as_string_array() const
[move_group-2] #6 libstdc++.so, in __cxa_throw
[move_group-2] #5 libstdc++.so, in std::terminate()
[move_group-2] Aborted (Signal sent by tkill())
```

定位

堆栈里的 `as_string_array()` 是关键 —— 有参数类型不对（期望字符串数组，实际是字符串）。

对比 MoveIt 自带的参考配置：

```
cat /opt/ros/jazzy/share/moveit_configs_utils/default_configs/ompl_planning.yaml
```

根因

我写的是 ROS 1 风格：

```
# × 错误
planning_plugin: ompl_interface/OMPLPlanner          # 单数
request_adapters: >-                                # 折叠字符串，不是 list!
  default_planner_request_adapters/AddTimeOptimalParameterization
  default_planner_request_adapters/ResolveConstraintFrames
  default_planner_request_adapters/FixWorkspaceBounds    # 旧名字
  default_planner_request_adapters/FixStartStateBounds   # 旧名字
  default_planner_request_adapters/FixStartStateCollision
  default_planner_request_adapters/FixStartStatePathConstraints
```

Jazzy 的正确格式：

```
# sqrt 正确
planning_plugins:                                # 复数，list
- ompl_interface/OMPLPlanner
request_adapters:                                # 必须是 YAML 序列
- default_planning_request_adapters/ResolveConstraintFrames
- default_planning_request_adapters/ValidateWorkspaceBounds # 新名
- default_planning_request_adapters/CheckStartStateBounds   # 新名
```

- default_planning_request_adapters/CheckStartStateCollision
- default_planning_request_adapters/CheckForStackedConstraints

response_adapters:

- default_planning_response_adapters/AddTimeOptimalParameterization
- default_planning_response_adapters/AddRuckigTrajectorySmoothing
- default_planning_response_adapters/ValidateSolution
- default_planning_response_adapters/DisplayMotionPath

Jazzy 里 `PlanningPipeline` 构造函数对这三个参数调用 `as_string_array()`，折叠字符串会抛 `ParameterTypeException` → `std::terminate`。

另外适配器名字也变了：

旧名	新名
<code>FixWorkspaceBounds</code>	<code>ValidateWorkspaceBounds</code>
<code>FixStartStateBounds</code>	<code>CheckStartStateBounds</code>
<code>FixStartStateCollision</code>	<code>CheckStartStateCollision</code>
<code>FixStartStatePathConstraints</code>	(Jazzy 没有，用 <code>CheckForStackedConstraints</code>)

怎么确认可用的适配器名字

```
grep -rhop '(?<=<class name=")default_planning_(request|response)_adapters/["]+' \
/opt/ros/jazzy/share/moveit_core/*.xml /opt/ros/jazzy/share/moveit_ros_planning/*.xml | sort -u
```

坑 5: `load_all=True` 默认值 → 找 `pilz_cartesian_limits.yaml` 报错

现象

```
[ERROR] [launch]: Caught exception in launch:
ParameterBuilderFileNotFoundError:
"File .../config/pilz_cartesian_limits.yaml doesn't exist"
```

定位

看 `MoveItConfigsBuilder` 的源码：

```
grep -n -A20 "def planning_pipelines" \
/opt/ros/jazzy/lib/python3.12/site-packages/moveit_configs_utils/moveit_configs_builder.py
```

```
if pipelines is None:
    pipelines = get_pattern_matches(config_folder, planning_pattern) # → ["ompl"]
    if load_all: # ← 默认 True!
        for pipeline in get_pattern_matches(default_folder, planning_pattern):
            if pipeline not in pipelines:
                pipelines.append(pipeline) # ← 加上了 pilz / chomp / stomp
...
if "pilz_industrial_motion_planner" in self.__moveit_configs.planning_pipelines:
    if not self.__moveit_configs.pilz_cartesian_limits:
        self.pilz_cartesian_limits() # → 需要 pilz_cartesian_limits.yaml
```

根因

`load_all` 默认是 `True`，会把 `MoveIt` 自带的 pipeline 全加载。

修复

```
moveit_config = (
    MoveItConfigsBuilder("my_robot_arm", package_name="my_robot_arm_moveit_config")
    .robot_description(...)
    .planning_pipelines(pipelines=["ompl"], load_all=False) # ★ 必须带 load_all=False
    .to_moveit_configs()
)
```

坑 6: 控制器关节集合和规划组不一致 → 轨迹被拒

现象

```
[moveit_simple_controller_manager]: arm_controller started execution
[WARN] Goal request rejected
[ERROR] Goal was rejected by server
[ERROR] Failed to send trajectory part 1 of 1 to controller arm_controller
```

定位

```
grep -A8 "arm_controller:" ~/Project/CanServer/config/ros2_controllers.yaml
# → joints 列了 6 个 (含 gripper_finger_left_joint)

grep -A8 "arm_controller:" .../moveit_controllers.yaml
# → joints 也是 6 个

# 但 MoveIt 只为 arm 组 (5 个关节) 规划
```

根因

`joint_trajectory_controller` 的 `allow_partial_joints_goal` 默认是 `false` → 收到只含 5 个关节的轨迹就直接拒绝。

修复

按 SRDF 的规划组拆控制器：

```
# ros2_controllers.yaml
arm_controller:
  ros_parameters:
    joints: [joint1, joint2, joint3, joint4, joint5]    # ← 只 5 个
gripper_controller:
  ros_parameters:
    joints: [gripper_finger_left_joint]                # ← 单独一个
```

```
# moveit_controllers.yaml
controller_names: [arm_controller, gripper_controller]
```

```
# arm_control.launch.py 多 spawn 一个
Node(package='controller_manager', executable='spawner',
  arguments=['gripper_controller', '-c', '/controller_manager']),
```

规则：控制器负责的关节集合必须和 MoveIt2 的规划组一致。

坑 7: `controller_names` 列了但没定义

现象

```
[ERROR] simple_controller_manager: No action namespace specified for controller
`gripper_controller` through parameter
`moveit_simple_controller_manager.gripper_controller.action_ns`
```

根因

`controller_names` 里列了 `gripper_controller`，但它的定义被注释掉了。

修复

名单里的每个控制器都必须有完整定义 (`type` / `action_ns` / `joints`)。

坑 8: 调用 `KinematicsBase::initialize` 基类 → 插件永远加载不上

现象

```
[ERROR] kinematics_base: IK plugin for group 'arm' relies on deprecated API.
Please implement initialize(rolcpp::Node::SharedPtr, RobotModel, ...).
[ERROR] move_group: 基类 KinematicsBase::initialize 返回 false
[ERROR] kinematics_plugin_loader: Kinematics solver of type
'arm_ik_plugin/ArmAnalyticIKPlugin' could not be initialized for group 'arm'
```

定位

我在插件的 `initialize` 里调用了基类：

```
if (!kinematics::KinematicsBase::initialize(node, robot_model, group_name,
                                           base_frame, tip_frames, search_discretization))
{ return false; }
```

打日志发现基类返回 `false`。

根因

MoveIt 2.12 中, `KinematicsBase::initialize(node, robot_model, ...)` 的默认实现 只是个「你必须 override」的桩 —— 一被调用就报 deprecated 并返回 `false`。

修复

不要调基类，自己设置 `node_` 并调用 protected 的 `storeValues()`：

```
node_ = node;
storeValues(robot_model, group_name, base_frame, tip_frames, search_discretization);
// 之后 robot_model_ / group_name_ / base_frame_ / tip_frames_ 都填好了
```

怎么发现 `storeValues` 是 protected 可用的

```
awk '/^protected:/{p=1} p{print NR: "$0" /};/{if(p)exit}' \
/opt/ros/jazzy/include/moveit_core/moveit/kinematics_base/kinematics_base.hpp
```

坑 9: `initialize` 里提前用 `node->get_logger()` → 段错误

现象

插件一加载就段错误, `move_group` 直接死：

```
[move_group-2] #1 libarm_ik_plugin.so, in ArmAnalyticIKPlugin::initialize(...)
[move_group-2] #0 librcldcpp.so, in rclcpp::Node::get_logger() const
[move_group-2] Segmentation fault (Address not mapped to object [0x38])
```

根因

`node_` 是基类的 `storeValues()` 才赋值的，在此之前是空指针。

我为了调试，在 `initialize` 开头加了日志：

```
RCLCPP_INFO(node->get_logger(), "ArmAnalyticIKPlugin::initialize 进入: ..."); // !! node_ 是空的
```

修复

基类调用之前，用参数 `node`（始终有效）打日志：

```
// ※ 基类 initialize 之前 node_ 还是空的（它由基类赋值），
// 这里必须用参数 node 打日志，否则 node->get_logger() 会段错误。
RCLCPP_INFO(node->get_logger(), "ArmAnalyticIKPlugin::initialize 进入: ...");

node_ = node;
storeValues(robot_model, group_name, base_frame, tip_frames, search_discretization);
RCLCPP_INFO(node->get_logger(), " storeValues OK"); // 之后也可以用 node_

const auto* jmg = robot_model->getJointModelGroup(group_name);
// ... 之后才用 node->get_logger()
```

这是个「加日志把程序搞崩」的经典案例。

坑 10: 下发频率太高 → 舵机永远走不完一段行程（一卡一卡之一）

现象

机械臂能动，但一卡一卡。而「直接通过串口发一条坐标命令」却很丝滑。

定位

看舵机协议的语义：

```
##03dP%04dT%04d!    「在 T 毫秒内【从当前位置】走到 P」
```

关键：每次收到新命令，都会从当前位置【重新开始】这段行程。

根因

```
每 20ms 发一次、T=20ms：
t=0ms    发 P0, T=20ms    舵机起步...
t=20ms    发 P1, T=20ms    ← 打断了上一次，重新起步
t=40ms    发 P2, T=20ms    ← 又打断，又重新起步
...
→ 舵机永远停在「加速阶段就被打断」，从没走完一段完整行程
→ 一卡一卡，而且平均速度也上不去
```

顺带：poll_div=1 时是 50 次查询 + 50 次下发 = 100 帧 CAN/秒，STM32 的 UART 任务（一条状态查询要 ~100ms）会积压，命令成批发出 → 更卡。

修复

```
// write(): 每 send_div_ 个控制周期才发一次 CAN, T 也取 send_div_ × 周期
if (send_div_ > 1 && (++send_tick_ % static_cast<size_t>(send_div_)) != 0)
{
    return hardware_interface::return_type::OK;
}
const uint16_t t_ms = static_cast<uint16_t>(
    std::clamp(period_ms * static_cast<double>(send_div_), 20.0, 400.0));
```

send_div=5（100ms 下发一次）以后，舵机每次都能走完一段完整行程。

同时把 poll_div 从 1 调到 3，减少总线竞争。

效果：CV 从 0.586 降到 0.142。

坑 11：STM32 的查询打断了运动命令（★「一卡一卡」的真正根因）

现象

延续坑 10。虽然改善了很多，但还不完美。

定位方法（关键）

绕开 ros2_control，用裸 SocketCAN 直接和 STM32 说话，同时以 20Hz 高频查询位置，拿到真实运动曲线。然后做对照实验：

场景	结果
完全不开查询	sqrt 3 秒行程正常走完
低频查询 5 Hz	× 停在原地（完成度 0.0%）
中频查询 20 Hz	× 停在原地
高频查询 50 Hz	× 停在原地

※ 我第一次的结论是错的：我以为「任何查询都会打断运动」。后来做更细的实验才发现：

move 后多久查一次	结果
5 ms	sqrt 99.6%
20 ms	sqrt 99.5%
50 ms	sqrt 100.0%
2000 ms	sqrt 99.5%

单次查询在任何时刻都没事。真正的条件是：

场景	结果
安静 0ms 后开始 5Hz 轮询	× 0.0%
安静 0ms 后开始 20Hz 轮询	× 0.0%
安静 300ms 后开始 5Hz 轮询	sqrt 99.3%

安静 300ms 后开始 20Hz 轮询 sqrt 99.5%

★ move 命令之后需要 ~300ms 的线路安静窗口。

根因（在 STM32 侧）

```
// robot_arm.c
static uint16_t servo_send_recv(const char *cmd, int len, uint32_t time_ms)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)cmd, len, TIMEOUT);
    ...
    for (uint32_t t = 0; t < time_ms; t++) { osDelay(1); ... } // 最多阻塞 time_ms
}

// freertos.c —— ASKFORSTATUS 一次调两个, 各等 50ms
case ASKFORSTATUS: {
    Servo_temp_and_v_get(cmd.id, _temp_v); // 最多 50ms
    st->cur_pos = Servo_position_get(cmd.id); // 最多 50ms
    ...
}
```

Servo_position_set 是「发完就走」, 不等舵机确认:

move 命令发出 → 舵机还在解析/锁定这条命令（半双工单线, 需要线路安静）
↓
紧接着的查询把 UART2 占住并发出 #PRAD!/#PRTE!
↓
★ move 命令被冲掉, 舵机根本没开始走 ★

修复（STM32 固件, 第 10 章详述）

```
case ASKFORSETPOS: {
    Servo_position_set(cmd.id, cmd.target_pos, cmd.time);
    quiet_until = osKernelGetTickCount() + 300U; /* ★ 300ms 静默 */
    break;
}

case ASKFORSTATUS: {
    if ((int32_t)(quiet_until - osKernelGetTickCount()) > 0) {
        Servo_cached_get(cmd.id, _temp_v, &p); /* 静默期: 回缓存, 不碰 UART */
        ...
    } else {
        ... 正常查询 ...
    }
}
```

效果

move 完成度: 0.0% → 99.5% （任意轮询频率下）

同时 ROS 侧也要改（见坑 12、坑 13）。

坑 12: 运动停止后 Info 缓存过期 → 状态突跳 → 容差失败

现象

```
[WARN] Overrun might occur, Total time : 34700 us (Expected < 20000 us)
--> Read time : 92 us, Update time : 23173 us, Write time : 11434 us

[ERROR] State tolerances failed for joint 2:
Position Error: -1.122340, Position Tolerance: 1.000000
[WARN] arm_controller: Aborted due to state tolerance violation
```

定位

1.12 rad 的误差不可能是 40ms 轨迹滞后造成的 —— 轨迹速度才 0.13 rad/s, 40ms 只有 0.005 rad。

所以不是「跟不上」, 是状态本身错了。

顺着这个思路看 read(): 运动期间不查询 → Info 缓存停在运动之前。停下来后按 poll_div 轮询, 要 6 个周期才轮完 → 前几拍读到的是过期值。

修复

刚停下来那一拍，先把 6 个舵机全部查一遍：

```
const size_t moving_win = static_cast<size_t>(send_div_) * 6;

if (send_age_ < moving_win) {
    // 运动期间：用命令值当状态（不查询，避免打断舵机）
    for (size_t i = 0; i < n; ++i)
        hw_positions_[i] = servoToAngle(i, angleToServo(i, hw_commands_[i]));
    return return_type::OK;
}

if (send_age_ == moving_win) {
    // ★ 刚停下来这一拍：一次性刷新全部，把过期缓存冲掉
    for (size_t i = 0; i < n; ++i) servo_>Status_get(servo_ids_[i]);
    for (int k = 0; k < 80; ++k) if (!servo_>Info_wait(10)) break;
    for (size_t i = 0; i < n; ++i) {
        const ServoStatus s = servo_>Read_Info(servo_ids_[i]);
        if (validReading(i, s.position)) hw_positions_[i] = servoToAngle(i, s.position);
    }
    return return_type::OK;
}
// 之后的常规轮询
```

为什么要「运动期间用命令值」

因为 STM32 在静默期内只能回缓存值（更旧），而且 JTC 是闭环的——它需要的是「舵机一定会到哪」，不是「舵机 100ms 前在哪」。

命令值正好就是「舵机一定会到哪」sqrt

坑 13：余弦定理写成 $\cos(a3)$ 而不是 $\cos(a3 - a2)$ （正解自校验救了一命）

现象

1K 偶发失败。有些目标能解，有些不能，没有明显规律。

定位

靠插件里的「正解自校验」把它变成了「立刻失败」：

```
double fx, fy, fz;
forwardArm(q, fx, fy, fz);
const double err = std::sqrt((fx-X)*(fx-X) + (fy-Y)*(fy-Y) + (fz-Z)*(fz-Z));
if (err > 1e-6) return; // ← 大部分候选解被这里丢掉
```

然后手算验证——目标 $r=0.026323$, $Z=0.324489$, $a4=0.1$ ，真实关节角 $[0, 0.4, -0.6, 0.3, 0]$ ：

```
rp = 0.019494  zp = 0.183839  d² = 0.034177
L1² = 0.009702  L2² = 0.009025  2 · L1 · L2 = 0.018715
```

我算的 $C = (d^2 - L1^2 - L2^2) / (2 \cdot L1 \cdot L2) = 0.82554$

```
cos(a3 - a2) = cos(-0.6) = 0.82534  sqrt 吻合
cos(a3)       = cos(-0.2) = 0.98007  × 对不上
```

根因

$a2/a3$ 是【绝对角】（相对 +z 轴），不是相对角！

位置展开：
$$\begin{aligned} r_p &= L1 \cdot \sin(a2) + L2 \cdot \sin(a3) \\ z_p &= L1 \cdot \cos(a2) + L2 \cdot \cos(a3) \end{aligned}$$

平方相加：
$$\begin{aligned} d^2 &= L1^2 + L2^2 + 2 \cdot L1 \cdot L2 \cdot (\sin a2 \sin a3 + \cos a2 \cos a3) \\ &= L1^2 + L2^2 + 2 \cdot L1 \cdot L2 \cdot \cos(a3 - a2) \end{aligned}$$

↑↑↑
是 $a3 - a2$ ，不是 $a3$

修复

// ★ 注意： $a2 / a3$ 是【绝对角】，所以余弦定理给出的是 $\cos(a3 - a2)$ ！

```
const double c = (d2 - L1 * L1 - L2 * L2) / (2.0 * L1 * L2); // = cos(a3 - a2)
if (std::abs(c) > 1.0) return;

const double dtheta = elbow * std::acos(std::clamp(c, -1.0, 1.0)); // = a3 - a2 = q3
const double a2 = std::atan2(rp, zp) -
    std::atan2(L2 * std::sin(dtheta), L1 + L2 * std::cos(dtheta));
const double a3 = a2 + dtheta;

q = {q1, a2, dtheta, a4 - a3, q5};
```

验证（离线对拍）

```
修正后 elbow=-1 → [0, +0.4000, -0.6000, +0.3000, 0] 误差 3.47e-18 ← 精确还原！
修正后 elbow=+1 → [0, -0.1888, +0.6000, -0.3112, 0] 误差 0.00e+00 ← 另一个解
旧公式 elbow=-1 → 误差 3.77e-02 ×
```

这一坑的最大教训

在逆解插件里加「正解自校验」，只需要 8 行代码，但能把「偶发的定位错误」变成「立刻失败」——这是整个项目里性价比最高的 8 行。

第 10 章 STM32 固件侧改动

本章的改动是坑 11 的根治。ROS 侧的修法只是绕行，真正的根因在固件。

10.1 为什么必须在固件层改

- ① 「什么时候可以查」这个问题，只有最靠近总线的 STM32 知道得最准（它知道每条 move 命令的 T 是多少）
- ② 改动一次，所有客户端受益（arm_server、我的插件、任何调试工具）
- ③ ROS 侧绕行的代价是「运动期间没有真实反馈」
固件层可以做到「运动期间给缓存，停下来给真实值」

10.2 改动一：move 后 300ms 静默窗口

robot_arm.h 追加

```
/* —— 舵机状态缓存（静默期内免查询回复） —— */
void Servo_cache_init(void); /* 上电刷一遍，保证缓存有效 */
void Servo_cached_get(uint8_t id, double temp_v[2], uint16_t *pos);
uint8_t Servo_cache_valid(uint8_t id);
```

robot_arm.c 追加缓存实现

```
/* —— 舵机状态缓存 —— */
*
* 为什么要它：move 命令(#PxxxxTxxxx!)发出后的 ~300ms 内，
* 线路上不能有任何其它收发，否则舵机还没来得及锁定这条命令，
* 就被随后的 #PRAD!/#PRTE! 冲掉 —— 实测 move 完成度 0.0%。
*
* 所以静默期内 ASKFORSTATUS 直接回这里的缓存，不去碰 UART2。
*/
typedef struct {
    uint16_t pos;
    double temp; /* 口径和 Servo_temp_and_v_get 保持一致 */
    double volt;
    uint8_t valid; /* bit0=pos 有效, bit1=temp/volt 有效 */
} ServoCache_t;

static ServoCache_t g_cache[SEVRO_NUMBER + 1]; /* 下标 = 舵机 id (0..5) */

static void cache_put_pos(uint8_t id, uint16_t pos)
{
    if (id > SEVRO_NUMBER) return;
    g_cache[id].pos = pos;
    g_cache[id].valid |= 0x01;
}

static void cache_put_tv(uint8_t id, double temp, double volt)
{
    if (id > SEVRO_NUMBER) return;
    g_cache[id].temp = temp;
    g_cache[id].volt = volt;
    g_cache[id].valid |= 0x02;
}

void Servo_cached_get(uint8_t id, double temp_v[2], uint16_t *pos)
{
    if (id > SEVRO_NUMBER) return;
    if (temp_v) { temp_v[0] = g_cache[id].temp; temp_v[1] = g_cache[id].volt; }
    if (pos) { *pos = g_cache[id].pos; }
}

uint8_t Servo_cache_valid(uint8_t id)
{
    if
```

```

    return (id <= SEVRO_NUMBER) ? g_cache[id].valid : 0;
}

/* 上电初始化：把 6 个舵机各读一遍，填满缓存 */
void Servo_cache_init(void)
{
    memset(g_cache, 0, sizeof(g_cache));
    for (uint8_t i = 0; i <= SEVRO_NUMBER; i++)
    {
        uint16_t p = Servo_position_get(i);
        double tv[2] = {0.0, 0.0};
        Servo_temp_and_v_get(i, tv);
        cache_put_pos(i, p);
        cache_put_tv(i, tv[0], tv[1]);
        osDelay(30); /* 别把舵机总线打爆 */
    }
}

```

在两个已有读取函数里各加一行

```

uint16_t Servo_position_get(uint8_t id) {
    ...
    if(servo_send_recv(cmd, len, 50) > 0) {
        char *p = strchr((char*)servo_rx_buf, 'P');
        if (p != NULL) {
            position = (uint16_t)atoi(p + 1);
            cache_put_pos(id, position); /* ★ 加这一行 */
        }
    }
    ...
}

void Servo_temp_and_v_get(uint8_t id, double _temp_v[2]) {
    ...
    if(servo_send_recv(cmd, len, 50) > 0) {
        ...
        cache_put_tv(id, _temp_v[0], _temp_v[1]); /* ★ 加这一行 */
    }
}

```

freertos.c 的 UART_task

```

void UART_task(void *argument)
{
    /* USER CODE BEGIN UART_task */
    ServoMsg_t *st = malloc(sizeof(ServoMsg_t));
    CmdMsg_t cmd;
    double _temp_v[2];
    memset(_temp_v, 0, sizeof(_temp_v));

    static uint32_t quiet_until = 0; /* ★ 静默截止时刻 (FreeRTOS tick) */

    /* ★★★ 放在这里，不要放 main.c 的 osKernelStart() 之前! ★★★
     * 原因见下面 10.4 的坑。 */
    Servo_cache_init();

    /* Infinite loop */
    for(;;)
    {
        if(osMessageQueueGet(CMD_QueueHandle, &cmd, NULL, portMAX_DELAY) == osOK) {
            switch(cmd.ask) {

            case ASKFORSETPOS: {
                Servo_position_set(cmd.id, cmd.target_pos, cmd.time);
                /* ★ move 之后 300ms 内不碰 UART2 */
                quiet_until = osKernelGetTickCount() + 300U;
                break;
            }

            case ASKFORSTATUS: {
                st->id = cmd.id;
                if ((int32_t)(quiet_until - osKernelGetTickCount()) > 0) {
                    /* ★ 静默期内：回缓存，不做任何 UART 收发 */
                    uint16_t p = 0;

```

```

        Servo_cached_get(cmd.id, _temp_v, &p);
        st->temp    = _temp_v[0];
        st->volt    = _temp_v[1];
        st->cur_pos = p;
    } else {
        /* 线路空闲：正常查询真实值 */
        Servo_temp_and_v_get(cmd.id, _temp_v);
        st->temp    = _temp_v[0];
        st->volt    = _temp_v[1];
        st->cur_pos = Servo_position_get(cmd.id);
    }
    osMessageQueuePut(Status_QueueHandle, st, 0, 0);
    break;
}

case ASKFORALL: {
    Servo_position_set(cmd.id, cmd.target_pos, cmd.time);
    quiet_until = osKernelGetTickCount() + 300U;    /* ★ 同样置静默 */
    {
        uint16_t p = 0;
        Servo_cached_get(cmd.id, _temp_v, &p);
        st->temp    = _temp_v[0];
        st->volt    = _temp_v[1];
        st->cur_pos = p;
    }
    osMessageQueuePut(Status_QueueHandle, st, 0, 0);
    break;
}

default : break;
}
osDelay(1);
continue;
}
osDelay(1);
}
/* USER CODE END UART_task */
}

```

10.3 改动二：Servo_position_get 失败时回缓存而不是回 0

这是坑 12 在固件侧的根本修法。

```

uint16_t Servo_position_get(uint8_t id) {
    if(id > SEVR0_NUMBER) return 1;
    char cmd[Servo_CMD_LEN];
    uint16_t position = 0;
    int len = snprintf(cmd, sizeof(cmd), "%03dPRAD!", id);
    if(servo_send_recv(cmd, len, 50) > 0) {
        char *p = strchr((char*)servo_rx_buf, 'P');
        if (p != NULL) {
            position = (uint16_t)atoi(p + 1);
            cache_put_pos(id, position);
            return position;    /* ★ 成功：返回并更新缓存 */
        }
    }
    return g_cache[id].pos;    /* ★ 失败：回缓存，不回 0 */
}

```

为什么重要：0 远在舵机标定范围（500~2500）之外。ROS 侧拿到 0 会映射成越界的关节角 → 状态突跳 → 容差失败。

ROS 侧也做了防御（见坑 12 的 validReading），但固件侧改掉更根本。

10.4 ※ 一个非常容易再犯的坑：osDelay 在调度器启动前不返回

现象（我实际踩到过）

改了固件烧录后，整块 STM32 像死机一样：

```

can0: state ERROR-PASSIVE (berr-counter tx 0 rx 0)
      error-warn  error-pass
        1          1          ← 进过 error-warning / error-passive

candump 4 秒 → 总线上什么都没有

```

cansend can0 ... → No buffer space available (没有节点 ACK)

根因

main.c 的调用顺序:

```
robot_arm_init();
HAL_Delay(2000);
/* USER CODE END 2 */

osKernelInitialize();
MX_FREERTOS_Init();
osKernelStart();      ← ★ 调度器在这里才启动 ★
```

如果把 Servo_cache_init() 加在 osKernelStart() 之前:

```
Servo_cache_init()
↓
Servo_position_get()
↓
servo_send_recv()      // robot_arm.c 第 41 行
↓
for (uint32_t t = 0; t < time_ms; t++) { osDelay(1); ... }
↑
          调度器没启动, osDelay 不返回
↓
main() 卡死在 osKernelStart() 之前
↓
★ CAN 不初始化收发、UART_task 不运行、总线上什么都没有 ★
```

正确做法

把 Servo_cache_init() 放在 UART_task 的第一件事 (此时调度器已经在跑):

```
void UART_task(void *argument)
{
    ...
    Servo_cache_init();    /* ★ 放这里 */
    for(;;) { ... }
}
```

main.c 保持原样, 不要加任何东西。

烧录前自检清单

检查	期望
main.c 里没有 Servo_cache_init()	sqrt
UART_task 开头有 Servo_cache_init()	sqrt
上电后 运行指示灯在闪 (led_running 任务)	sqrt 闪 = 调度器在跑
上电后 huart3 输出 Init OK	sqrt

如果烧完指示灯不闪 → 还是卡在初始化。先把 Servo_cache_init() 注释掉确认系统能起来, 再逐行加回去定位。

10.5 建议的改进 (可选)

ASKFORSTATUS 一次调用两个 servo_send_recv, 各等 50ms → 最长阻塞 UART_task 100ms。

```
20Hz 轮询 → 100ms × 20 = 2000ms/s > 100%
↓
CMD_Queue (16 槽) 必然填满
↓
if (osMessageQueuePut(...) != osOK) { //cmd_drop++; } ← 静默丢掉!
```

建议: 温度电压低频采集

```
/* 文件作用域 */
```

```

static uint8_t temp_div = 0;
static double last_temp[SERVONUMBER + 1] = {0};
static double last_volt[SERVONUMBER + 1] = {0};

case ASKFORSTATUS: {
    st->id = cmd.id;
    if (静默期) {
        uint16_t p = 0;
        Servo_cached_get(cmd.id, _temp_v, &p);
        st->cur_pos = p; st->temp = _temp_v[0]; st->volt = _temp_v[1];
    } else {
        st->cur_pos = Servo_position_get(cmd.id); /* 只这一次 UART, ~5ms */

        if (++temp_div >= 20) { /* ★ 每 20 次才读温度电压 */
            temp_div = 0;
            Servo_temp_and_v_get(cmd.id, _temp_v);
            last_temp[cmd.id] = _temp_v[0];
            last_volt[cmd.id] = _temp_v[1];
        }
        st->temp = last_temp[cmd.id];
        st->volt = last_volt[cmd.id];
    }
    osMessageQueuePut(Status_QueueHandle, st, 0, 0);
    break;
}

```

一条状态查询从 100ms → 5ms, 队列压力降 20 倍。

第 11 章 性能、精度与限制

11.1 时间开销（RK3568，4× Cortex-A55）

环节	耗时
IK 单次调用（124 个候选 + 自校验）	< 1 ms
move_group 启动（加载 URDF/SRDF/pipeline）	~4 s
一次三维坐标规划（RRTConnect，10mm 容差）	1.2 ~ 2.3 s
一次三维坐标执行（0.2 rad 小动作）	~1.5 s
一次大幅运动规划（joint1 ±1.30 rad）	2.4 ~ 6.2 s

内存：move_group 300~500 MB，控制层 ~50 MB。3.8 GB 的板子够用（但要关掉 VS Code Server）。

11.2 精度

小幅运动（0.15~0.20 rad 关节偏移）

规划 10/10	执行 10/10	容差失败 0 次	末端误差 平均 11.9 mm，最大 19.7 mm
----------	----------	----------	----------------------------

大幅运动（joint1 ±1.30 rad = ±75°，joint5 ±2.5）

规划 8/8	执行 8/8	末端误差 平均 21.5 mm，最大 38.4 mm
--------	--------	----------------------------

★ 关键发现：误差与重力力矩成正比

把大幅运动的结果按「末端水平伸出距离」排序：

目标	水平伸出	误差
① 前伸水平	0.260 m	31.0 mm
② 前伸下压	0.199 m	28.4 mm
⑦ 复合-腕自转	0.031 m	20.8 mm
⑥ 复合+腕自转	0.031 m	18.4 mm
⑤ 后收上举	0.005 m	16.1 mm
④ 右侧大幅	0.050 m	7.3 mm
⑧ 回归 ready	0.026 m	6.4 mm
③ 左侧大幅	0.050 m	5.6 mm

规律极清晰：伸得越远，误差越大（5.6 mm → 31 mm，5.5 倍）。

根因

舵机是位置控制，【没有力矩反馈，也没有重力补偿】
↓
机械臂伸出 → joint2/joint3 承受的重重力矩增大
↓
舵机在死区附近下垂
↓
末端位置偏低/偏内

31 mm / 260 mm ~ 12% —— 这就是这套舵机的刚度极限，软件无法消除。

但重复定位精度很好

回归 ready 姿态：本次 6.4 mm，上一次 6.5 mm → 相差 0.1 mm

同一目标的两次独立到达只差 0.1 mm —— 说明：

- 重复定位精度 ~0.1 mm 级
- 误差是系统性的（重力下垂），不是随机的
- 所以理论上可以标定补偿：在 ArmSystemHardware 里加一张「按 (joint2, joint3) 查表的重力补偿表」，把系统性偏差抵消掉

误差来源汇总

#	来源	量级	能否软件消除
1	重力力矩导致的舵机下垂	5 ~ 38 mm (随臂展)	× 只能标定补偿
2	状态反馈延迟	~120~360 ms 对应的位置滞后	部分 (减小 poll_div)
3	舵机分辨率	~0.26 mm (2000 counts / 90°)	— 可忽略
4	机械间隙	未单独测量	×

11.3 已知限制

限制	说明	可能的改进
奇异位形附近规划不稳	机械臂完全伸直时 ($r \sim 0$) 是奇异点，附近目标规划/执行会失败	加「远离奇异」的代价函数；先走到 ready 姿态再动
只保证位置，不管姿态	5-DOF 无法同时控制 6 个位姿自由度；插件只解位置， q_5 沿用种子	若要控姿态，需利用 q_5 的冗余
运动期间没有真实反馈	STM32 在静默期内只回缓存值	换支持「自动上报位置」的舵机
每次上电要设 CAN	芯片不保存 bitrate	已用 <code>/etc/network/interfaces.d/can0</code> 解决
板子无显示屏	不能跑 RViz	用 headless launch；可视化在 PC 上通过 DDS 连过来
精度受姿态影响大	5.6 ~ 38 mm	标定补偿表

第 12 章 运维与排错速查

12.1 日常启动流程

```
# —— 板子上电后 ——
sudo ifup can0          # 配过 interfaces.d 之后只需这一条
ip -details link show can0  # 确认 state UP + ERROR-ACTIVE

# —— 终端 1: 控制层 ——
source /opt/ros/jazzy/setup.bash
source ~/ros2/ros_ws/install/setup.bash
ros2 launch CanServer arm_control.launch.py

# —— 终端 2: 规划层 ——
source /opt/ros/jazzy/setup.bash
source ~/ros2/ros_ws/install/setup.bash
ros2 launch my_robot_arm_moveit_config arm_moveit_headless.launch.py

# —— 终端 3: 发目标 ——
python3 ~/ros2/ros_ws/test/moveit_exec2.py
```

12.2 完整排错表

症状	大概率原因	怎么查	解决
Write : fail to write! / CanERROR!	can0 没 UP	ip -details link show can0	sudo ifup can0
/joint_states 全是 $\pm 1.571/\pm 3.142$	CAN 没通, 读到无效数据映射	同上	同上
cansend 报 No buffer space available	没有节点 ACK (STM32 没跑)	candump can0	检查 STM32 供电/固件/LED
can state ERROR-PASSIVE	同上	ip -details -statistics link show can0	同上
file 'xxx.launch.py' was not found in share	CMakeLists 缺 install(DIRECTORY ...)	grep -c "install(" CMakeLists.txt	加回并重编
XML or text declaration not at start of entity	xacro 第 1 行有空格	head -c 8 x.xacro \ xxd -p	sed -i '1s/^[:space:]*//'
move_group 直接 abort (std::terminate)	ompl_planning.yaml 用了折叠字符串	看堆栈里有没有 as_string_array	改成 YAML 序列
pilz_cartesian_limits.yaml doesn't exist	load_all 默认 True	看 launch 里有没有 load_all=False	加上
No action namespace specified for controller	controller_names 里列了没定义的	对比名单和定义	名单里的每个都要有完整定义
IK plugin ... relies on deprecated API	调了基类 KinematicsBase::initialize	看插件源码	改成自己设 node_ + storeValues()
could not be initialized for group	插件 initialize 返回 false	看插件自己的日志	逐行定位
Segmentation fault ... get_logger	initialize 里提前用了 node_	看堆栈	基类调用前用参数 node
Unable to sample any valid states for goal tree	IK 解不出来	kinematics.yaml 指向哪个插件	用 arm_ik_plugin/ ArmAnalyticIKPlugin
Goal was rejected by server	控制器关节集合 != 规划组	对比两个 yaml 的 joints	按 SRDF 的 group 拆控制器
State tolerances failed for joint N	状态突跳 (缓存过期 / 无效读数)	grep "State tolerances" / tmp/rc.log	见坑 12、坑 13

PATH_TOLERANCE_VIOLATED(-4)	舵机跟不上 / 命令被查询冲掉	裸 CAN 做对照实验	见坑 10、坑 11
规划成功但机械臂不动	执行链路没接上	ros2 action list \ grep execute_trajectory	查 moveit_controllers.yaml
机械臂——卡——卡	三条：下发太密 / 查询打断 / 缓存过期	python3 test/can_smooth.py direct 6 看 CV	见坑 10、11、12
规划永远失败且无报错	SRDF 缺 disable_collisions	看 srdf	补齐所有相邻连杆对

12.3 一键自查脚本

```
cat > ~/check_all.sh <<'EOF'
#!/bin/bash
echo "—— CAN ——"
ip -details link show can0 | head -3
echo
echo "—— 通信 ——"
timeout 3 candump can0 > /tmp/_d.txt 2>&1 &
sleep 0.5; cansend can0 104#0002000000000000 2>&1; sleep 1.5
grep -c "200" /tmp/_d.txt | xargs echo " 收到回复帧数:"
echo
echo "—— 进程 ——"
for p in ros2_control_node move_group robot_state_publisher; do
  printf " %-24s %s\n" "$p" "$(pgrep -fc $p)"
done
echo
echo "—— 控制器 ——"
source /opt/ros/jazzy/setup.bash 2>/dev/null
source ~/ros2/ros2_ws/install/setup.bash 2>/dev/null
timeout 10 ros2 control list_controllers 2>/dev/null
echo
echo "—— 日志里的错误 ——"
grep -iE "State tolerances failed|Aborted due to|rejected" /tmp/rc.log 2>/dev/null | tail -5 || echo " 无"
EOF
chmod +x ~/check_all.sh
~/check_all.sh
```

12.4 代码同步（双份源码）

CanServer 仓库里 can_server/ 是 git 跟踪的，root 那层是你开发的：

```
# 开发：直接改 root (ros2_ws/src/CanServer 就是软链接到 root)
# 推送：push.sh 把 root rsync 到 can_server/ 再 git push
cd ~/Project/CanServer && ./push.sh
```

※ push.sh 的 SRC 数组要包含所有需要的文件（比如 arm_system_hardware.xml）：

```
SRC=(CMakeLists.txt package.xml arm_system_hardware.xml main.cpp src include launch config)
```

附录 A · 命令速查

```
# —— 环境 ——
source /opt/ros/jazzy/setup.bash
source ~/ros2/ros2_ws/install/setup.bash

# —— CAN ——
sudo ifup can0
ip -details link show can0
candump can0
cansend can0 104#0002000000000000 # 查舵机 0 状态
cansend can0 104#0001DC05B80B0000 # 让舵机 0 在 3000ms 内走到 1500

# —— 编译 ——
cd ~/ros2/ros2_ws
colcon build --symlink-install
colcon build --packages-select CanServer --symlink-install
colcon list

# —— 启动 ——
ros2 launch CanServer arm_control.launch.py
ros2 launch my_robot_arm_moveit_config arm_moveit_headless.launch.py

# —— 验证 ——
ros2 control list_controllers
ros2 control list_hardware_interfaces
ros2 topic echo /joint_states --field position
ros2 topic hz /joint_states
ros2 node list | grep move_group
ros2 action list | grep -E "arm_controller|execute"

# —— 测试脚本 ——
python3 test/kin_check.py # 校验运动学模型
python3 test/send_traj.py --limits # 关节限位表
python3 test/send_traj.py 0.5 - - - - 2 # 只动 joint1 (- = 保持不动)
python3 test/cart_probe.py # 三维坐标规划 3 种约束对比
python3 test/moveit_test.py cart # FK/IK/关节规划/三维坐标规划
python3 test/moveit_exec2.py # ★ 三维坐标 → 规划 → 执行 → 实测误差
python3 test/moveit_big.py # 大幅运动
python3 test/can_smooth.py direct 6 # ★ 裸 CAN 测真实曲线 (需先停 ros2_control)
python3 test/can_smooth.py home 4 # 归位到关节中点
python3 test/smooth_check.py 0.8 6 # 通过 ROS 侧量顺滑度

# —— 清理重编 ——
kill -f ros2_control_node; pkill -f move_group
rm -rf build/CanServer install/CanServer
colcon build --packages-select CanServer --symlink-install
```

项目名

5 自由度机械臂 · RK3568 + STM32 异构控制系统

一句话

基于 ROS 2 与 MoveIt2 实现 5 自由度机械臂的运动规划与执行，自研解析逆解插件解决 KDL 无法求解的问题，并通过裸 CAN 对照实验定位并根治了「运动不流畅」的底层时序缺陷。

技术栈

上位机 RK3568 (4×Cortex-A55, Ubuntu 24.04 aarch64) + ROS 2 Jazzy
规划层 MoveIt2 (OMPL RRTConnect + Ruckig 加加速度受限平滑)
控制层 ros2_control (自定义 SystemInterface 硬件插件)
通信 CAN 2.0B @250kbps (自定义协议)
下位机 STM32F407 + FreeRTOS
执行 6× 总线舵机 (ZX20S + ZX361S), 半双工 UART
开发 C++17 / Python / CMake / pluginlib / xacro / URDF / SRDF

核心工作（可直接写进简历）

- 为 5 自由度机械臂实现 MoveIt2 运动规划 因机械臂自由度为 5、KDL (6-DOF 数值解) 无法从随机种子收敛，基于「底座 yaw + 平面 2R」闭式解自研了解析逆解 KinematicsBase 插件；插件内实现腕部冗余角扫描 + 肘部双构型枚举 + 关节限位筛选 + 正解自校验 (1e-6 容差，该机制在开发中捕获了一处余弦定理公式错误)，并对碰撞回调遍历全部解析解，使 10mm 位置容差的规划成功率从 0 提升到 100%。
- 基于 ros2_control 实现自定义硬件接口插件 把 CAN 总线舵机接入 ROS 2 标准控制框架，实现 SystemInterface 的 read() / write() / 生命周期回调，复用已有的 SocketCAN 驱动库 (不依赖 ROS)；按 SRDF 规划组拆分 arm_controller / gripper_controller，支持 joint_trajectory_controller 轨迹执行。
- 定位并根治「运动不流畅」的底层时序缺陷 现象是机械臂能运动但一顿一顿，而直接串口发送单条命令却很流畅。绕开上层框架用裸 SocketCAN + 20Hz 高频采样做对照实验，定位出三处缺陷：① 50Hz 流式下发导致舵机行程被反复打断；② 半双工总线舵机的状态查询会打断正在执行的运动命令 (需 300ms 线路安静窗口)；③ 运动停止后状态缓存过期导致反馈跳变。分别在 ROS 侧 (降频下发、运动期不查询、停后全量刷新) 与 STM32 侧 (300ms 静默窗口 + 状态缓存) 修复，速度变异系数 CV 从 0.586 降到 0.133，move 完成度从 0.0% 提升到 99.5%。
- 端到端验收 ★ 三维坐标 → 规划 → 执行：规划 10/10，执行 10/10，零容差失败；大幅运动 (joint1 ±75°)：规划 8/8，执行 8/8；末端定位误差平均 11.9mm (臂展 330mm)；并识别出误差与重力力矩正相关 (5.6~38mm) 的舵机刚度瓶颈，提出基于关节角查表的标定补偿方案。

面试可能被问到的点

问题	回答要点
为什么不用 KDL?	5-DOF vs 6-DOF 数值解，雅可比 6×5 欠定，从随机种子不收敛
/compute_ik 不是成功了吗?	那是假象 —— 给了精确种子。真实场景是随机种子
怎么保证 IK 公式没错?	正解自校验 + FK 逐点比对 (10 组随机配置，误差 0.000000)
5-DOF 的冗余怎么处理?	位置由 joint1~4 决定，a4 有 1 个冗余自由度，扫描枚举
ros2_control 的 read/write 什么时候调?	每周期，read 在前 write 在后
MoveIt2 和 ros2_control 怎么衔接?	FollowJointTrajectory action, moveit_controllers.yaml 配置
「一卡一卡」怎么定位的?	绕开框架，裸 CAN + 20Hz 采样做对照实验

为什么查询会打断运动?	半双工总线, move 命令需要 ~300ms 线路安静窗口才能被舵机锁定
定位精度受什么影响?	主要是重力力矩导致的舵机下垂 (无重力补偿), 5.6~38mm
为什么能标定补偿?	重复定位精度 0.1mm 级, 误差是系统性的

附录 C · 测试脚本全文

下面 7 个脚本都在 `~/ros2/ros2_ws/test/`（工作区里也存了一份在 `demo/`）。每个脚本都是独立可跑的，按第 8 章的分层顺序使用。

C.1 `kin_check.py` —— L3 校验运动学模型

```
#!/usr/bin/env python3
"""用 /compute_fk 校验解析运动学模型

推导（从 URDF）：
    joint1 绕 z 转      base→j2 轴线距离 Z0 = 0.0645 + 0.008 = 0.0725
    joint2/3/4 绕 y 转  L1 = 0.0985 (j2→j3)
                        L2 = 0.0950 (j3→j4)
    link5 原点          L3 = 0.0685 (j4→link5)

平面内（q1=0 时）：
    a2 = q2
    a3 = q2 + q3
    a4 = q2 + q3 + q4
    x = L1*sin(a2) + L2*sin(a3) + L3*sin(a4)
    z = Z0 + L1*cos(a2) + L2*cos(a3) + L3*cos(a4)
绕 joint1 旋转 q1 后：
    X = x*cos(q1)
    Y = ±x*sin(q1)      ← 符号待定
"""

import math
import random
import sys
import time

import rclpy
from rclpy.node import Node
from moveit_msgs.srv import GetPositionFK
from moveit_msgs.msg import RobotState, MoveItErrorCodes

L1, L2, L3, Z0 = 0.0985, 0.0950, 0.0685, 0.0725
BASE, TIP = 'base_link', 'link5'
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
LIM = [1.5708, 1.5708, 1.5708, 1.5708, 3.1416]

def model(q, ysign):
    q1, q2, q3, q4, _ = q
    a2, a3, a4 = q2, q2 + q3, q2 + q3 + q4
    x = L1 * math.sin(a2) + L2 * math.sin(a3) + L3 * math.sin(a4)
    z = Z0 + L1 * math.cos(a2) + L2 * math.cos(a3) + L3 * math.cos(a4)
    return x * math.cos(q1), ysign * x * math.sin(q1), z

def main():
    rclpy.init()
    node = Node('kin_check')
    cli = node.create_client(GetPositionFK, '/compute_fk')
    if not cli.wait_for_service(timeout_sec=20.0):
        print('FAIL /compute_fk 不可用')
        return 1

    random.seed(7)
    cases = [[0.0, 0.0, 0.0, 0.0, 0.0],
              [0.0, 0.4, -0.6, 0.3, 0.0],
              [0.5, 0.4, -0.6, 0.3, 0.0],
              [-0.5, 0.4, -0.6, 0.3, 0.0]]
    for _ in range(6):
        cases.append([random.uniform(-l * 0.8, l * 0.8) for l in LIM])

    print(' %-40s %-26s %-26s %s' % ('关节角', 'FK 实测', '模型(ysign=+1)', '误差'))
    print(' ' + '-' * 110)

    worst = {'+1': 0.0, '-1': 0.0}
    for q in cases:
        req = GetPositionFK.Request()
```

```

req.header.frame_id = BASE
req.fk_link_names = [TIP]
rs = RobotState()
rs.joint_state.name = JOINTS
rs.joint_state.position = list(q)
req.robot_state = rs
fut = cli.call_async(req)
rclpy.spin_until_future_complete(node, fut, timeout_sec=10.0)
res = fut.result()
if res is None or res.error_code.val != MoveItErrorCodes.SUCCESS:
    print(' FK 失败: %s' % q)
    continue
p = res.pose_stamped[0].pose.position
errs = {}
for s, tag in ((1, '+1'), (-1, '-1')):
    mx, my, mz = model(q, s)
    errs[tag] = math.sqrt((mx - p.x) ** 2 + (my - p.y) ** 2 + (mz - p.z) ** 2)
    worst[tag] = max(worst[tag], errs[tag])
m1 = model(q, 1)
print(' %-40s (%+.4f, %+.4f, %+.4f) (%+.4f, %+.4f, %+.4f) +1: %.5f -1: %.5f'
      % ([ '%+.2f' % v for v in q], p.x, p.y, p.z, m1[0], m1[1], m1[2],
        errs['+1'], errs['-1']))

print('')
print(' 最大误差: ysign=+1 → %.6f m   ysign=-1 → %.6f m' % (worst['+1'], worst['-1']))
if worst['+1'] < 1e-4:
    print(' sqrt 模型正确, joint1 用 Y = +x*sin(q1)')
elif worst['-1'] < 1e-4:
    print(' sqrt 模型正确, joint1 用 Y = -x*sin(q1)')
else:
    print(' × 模型不对, 需要重新推导')

node.destroy_node()
rclpy.shutdown()
return 0

if __name__ == '__main__':
    sys.exit(main())

```

C.2 send_traj.py —— L4 手发关节轨迹（带范围自检）

```

#!/usr/bin/env python3
"""发关节轨迹到 arm_controller（带范围自检）

用法:
./send_traj.py j1 j2 j3 j4 j5 gripper [时长秒]
./send_traj.py --limits                打印关节限位表
./send_traj.py --home [时长秒]         全部回中位（限位中点）

某个关节写 - 表示【保持当前位置不动】

示例:
./send_traj.py 0.5 0 0 0 0 2          joint1 → 0.5 rad, 2 秒
./send_traj.py 0.5 - - - - 2          只动 joint1, 其余保持
./send_traj.py 0 0 0 0 0 0.015 1      夹爪 → 0.015（范围 0~0.02）
./send_traj.py --home 3                全部回中位
"""

import sys

# ※ ROS 的 import 放在 main() 里（惰性）——
# 这样 --help / --limits / 范围自检 不需要 source ROS 也能用。

# -----
# 关节定义 ※ 改 xacro 的 pos_min/pos_max/dir 时这里要同步
# -----
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5',
          'gripper_finger_left_joint']

LIMITS = [
    (-1.5708, 1.5708),    # joint1
    (-1.5708, 1.5708),    # joint2
    (-1.5708, 1.5708),    # joint3
    (-1.5708, 1.5708),    # joint4
    (-3.1416, 3.1416),    # joint5
    (0.0000, 0.0200),     # gripper_finger_left_joint

```

```

]

DIRS = [+1, +1, -1, -1, +1, -1]      # 和 xacro 的 <param name="dir"> 一致

ACTION = '/arm_controller/follow_joint_trajectory'

ERRORS = {
    0: 'SUCCESSFUL',
    -1: 'INVALID_GOAL',
    -2: 'INVALID_JOINTS',
    -3: 'OLD_HEADER_TIMESTAMP',
    -4: 'PATH_TOLERANCE_VIOLATED'  ← 路径中偏离容差（命令超范围/舵机跟不上）,
    -5: 'GOAL_TOLERANCE_VIOLATED' ← 终点没进容差',
}

def print_limits():
    print()
    print(f"   {'#':<3} {'关节':<28} {'最小':>10} {'最大':>10} {'方向':>6}")
    print(" " + "-" * 58)
    for i, (name, (lo, hi), d) in enumerate(zip(JOINTS, LIMITS, DIRS)):
        print(f"   {i:<3} {name:<28} {lo:>10.4f} {hi:>10.4f} {d:>+6d}")
    print()
    print("  提示：方向 dir=-1 的关节，命令正值时实时机往反方向转")
    print()

def parse_args(argv):
    """返回 (target, dur); 出错时返回 (None, None)"""
    if not argv or argv[0] in ('-h', '--help'):
        print(__doc__)
        return None, None

    if argv[0] == '--limits':
        print_limits()
        return None, None

    if argv[0] == '--home':
        dur = float(argv[1]) if len(argv) > 1 else 3.0
        target = [(lo + hi) / 2.0 for lo, hi in LIMITS]
        return target, dur

    if len(argv) < 6:
        print(f"× 需要 6 个关节角，只给了 {len(argv)} 个")
        print(__doc__)
        return None, None

    # '-' 表示保持当前位置（下面用 cur 替换）
    target = []
    for i, x in enumerate(argv[:6]):
        if x in ('-', '~', 'x', 'X'):
            target.append(None)
            continue
        try:
            target.append(float(x))
        except ValueError:
            print(f"× 第 {i + 1} 个参数 {x!r} 不是数字（用 - 表示保持不动）")
            return None, None

    dur = float(argv[6]) if len(argv) > 6 else 3.0
    if dur <= 0:
        print(f"× 时长必须 > 0，给了 {dur}")
        return None, None
    return target, dur

def check_limits(target, cur=None):
    """范围自检（target 里的 None 表示保持当前位置，跳过检查）。
    返回 True 表示全部合法"""
    bad = []
    for i, (name, v, (lo, hi)) in enumerate(zip(JOINTS, target, LIMITS)):
        if v is None:
            continue
        if not (lo <= v <= hi):
            bad.append((i, name, v, lo, hi))

    if not bad:
        return True

```

```

print()
print(" × 命令超出关节限位, 已中止 (没有发送)")
print()
print(f" {'关节':<28} {'你给的':>12} {'合法范围':>24} {'超出':>12}")
print(" " + "-" * 78)
for i, name, v, lo, hi in bad:
    over = v - hi if v > hi else v - lo
    print(f" {name:<28} {v:>12.4f} {f'[{lo:.4f}, {hi:.4f}']':>24} {over:>+12.4f}")
print()
print(" 用 ./send_traj.py --limits 查看全部限位")
print()
return False

def main():
    target, dur = parse_args(sys.argv[1:])
    if target is None:
        sys.exit(0)

    # ★ 发之前先范围自检 (不需要 ROS)
    if not check_limits(target):
        sys.exit(1)

    # —— 到这里才需要 ROS ——
    import rclpy
    from rclpy.node import Node
    from rclpy.action import ActionClient
    from sensor_msgs.msg import JointState
    from control_msgs.action import FollowJointTrajectory
    from trajectory_msgs.msg import JointTrajectoryPoint
    from builtin_interfaces.msg import Duration

    rclpy.init()
    node = Node('send_traj')

    # —— ① 读当前位置 ——
    cur = None

    def cb(msg):
        nonlocal cur
        try:
            cur = [msg.position[msg.name.index(j)] for j in JOINTS]
        except (ValueError, IndexError):
            pass

    node.create_subscription(JointState, '/joint_states', cb, 10)
    deadline = node.get_clock().now().nanoseconds + int(5e9)
    while cur is None and node.get_clock().now().nanoseconds < deadline:
        rclpy.spin_once(node, timeout_sec=0.5)

    if cur is None:
        print(" × 5 秒内没收到 /joint_states —— ros2_control 起来了没? ")
        node.destroy_node()
        rclpy.shutdown()
        sys.exit(1)

    # ★ '-' 的地方用当前位置填充 (保持不动)
    kept = [JOINTS[i] for i, v in enumerate(target) if v is None]
    target = [c if v is None else v for v, c in zip(target, cur)]
    if kept:
        print(f" ({', '.join(kept)} 保持当前位置)")

    # —— ② 打印计划 ——
    print()
    print(f" {'关节':<28} {'当前':>10} {'目标':>10} {'移动':>10} {'速度':>12}")
    print(" " + "-" * 72)
    for name, c, t in zip(JOINTS, cur, target):
        d = t - c
        print(f" {name:<28} {c:>10.4f} {t:>10.4f} {d:>+10.4f} {d / dur:>+10.3f} rad/s")
    print()
    print(f" 时长 {dur}s")

    # —— ③ 发轨迹 ——
    cli = ActionClient(node, FollowJointTrajectory, ACTION)
    if not cli.wait_for_server(timeout_sec=5.0):
        print(f" × 找不到 action server: {ACTION}")
        print(" ros2_control 起来了没? ros2 control list_controllers")
        node.destroy_node()
        rclpy.shutdown()

```

```

sys.exit(1)

goal = FollowJointTrajectory.Goal()
goal.trajectory.joint_names = JOINTS
goal.trajectory.points = [
    JointTrajectoryPoint(positions=cur,
                          time_from_start=Duration(sec=0, nanosec=0)),
    JointTrajectoryPoint(positions=target,
                          time_from_start=Duration(
                              sec=int(dur),
                              nanosec=int((dur % 1) * 1e9))),
]

fut = cli.send_goal_async(goal)
rclpy.spin_until_future_complete(node, fut)
gh = fut.result()
if not gh.accepted:
    print("× 目标被拒绝 (joint_trajectory_controller 不接受)")
    node.destroy_node()
    rclpy.shutdown()
    sys.exit(1)

print("  sqrt 已接受, 执行中...")

res_fut = gh.get_result_async()
rclpy.spin_until_future_complete(node, res_fut)
code = res_fut.result().result.error_code

print()
if code == 0:
    print(f"  sqrt error_code = 0  ({ERRORS[0]})")
else:
    print(f"  × error_code = {code}")
    print(f"      {ERRORS.get(code, '未知错误码')}")
print()

node.destroy_node()
rclpy.shutdown()
sys.exit(0 if code == 0 else 1)

if __name__ == '__main__':
    main()

```

C.3 `cart_probe.py` —— L5 规划失败时定位

```

#!/usr/bin/env python3
"""找出「三维坐标规划」为什么失败 —— 一次跑 3 种目标约束做对比

python3 cart_probe.py

变体:
V0 参考状态合法性 (/check_state_validity)
V1 纯位置约束, 容差 10mm
V2 纯位置约束, 容差 50mm    ← 如果这个能过, 说明是采样太稀
V3 位置 + 姿态约束          ← 如果这个能过, 说明是姿态采样的问题
"""

import sys
import time

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose, PoseStamped
from shape_msgs.msg import SolidPrimitive
from moveit_msgs.srv import (
    GetPositionFK, GetPositionIK, GetMotionPlan, GetStateValidity,
)
from moveit_msgs.msg import (
    RobotState, MoveItErrorCodes, Constraints, MotionPlanRequest,
    PositionConstraint, OrientationConstraint, BoundingVolume,
)

GROUP, TIP, BASE = 'arm', 'link5', 'base_link'
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
Q_REF = [0.4, 0.6, -0.8, 0.3, 0.2]

```

```

ERR = {v: k for k, v in vars(MoveItErrorCodes).items()
       if isinstance(v, int) and not k.startswith('_')}

def ename(c):
    return ERR.get(c, 'code=%s' % c)

def rstate(q):
    rs = RobotState()
    rs.joint_state.name = JOINTS
    rs.joint_state.position = list(q)
    return rs

def call(node, cli, req, t=30.0):
    fut = cli.call_async(req)
    rclpy.spin_until_future_complete(node, fut, timeout_sec=t)
    return fut.result()

def read_js(node, timeout=6.0):
    from sensor_msgs.msg import JointState
    box = {'m': None}

    def cb(m):
        box['m'] = m

    node.create_subscription(JointState, '/joint_states', cb, 10)
    t0 = time.time()
    while box['m'] is None and time.time() - t0 < timeout:
        rclpy.spin_once(node, timeout_sec=0.3)
    if box['m'] is None:
        return None
    d = dict(zip(box['m'].name, box['m'].position))
    return [d.get(j, 0.0) for j in JOINTS]

def fk(node, cli, q):
    r = GetPositionFK.Request()
    r.header.frame_id = BASE
    r.fk_link_names = [TIP]
    r.robot_state = rstate(q)
    res = call(node, cli, r, 10.0)
    if res is None or res.error_code.val != MoveItErrorCodes.SUCCESS:
        return None
    return res.pose_stamped[0].pose

def pos_c(xyz, tol):
    pc = PositionConstraint()
    pc.header.frame_id = BASE
    pc.link_name = TIP
    pc.weight = 1.0
    s = SolidPrimitive()
    s.type = SolidPrimitive.SPHERE
    s.dimensions = [tol]
    p = Pose()
    p.position.x, p.position.y, p.position.z = xyz
    p.orientation.w = 1.0
    bv = BoundingVolume()
    bv.primitives = [s]
    bv.primitive_poses = [p]
    pc.constraint_region = bv
    c = Constraints()
    c.position_constraints = [pc]
    return c

def ori_c(qpose, tol):
    oc = OrientationConstraint()
    oc.header.frame_id = BASE
    oc.link_name = TIP
    oc.orientation = qpose.orientation
    oc.absolute_x_axis_tolerance = tol
    oc.absolute_y_axis_tolerance = tol
    oc.absolute_z_axis_tolerance = tol
    oc.weight = 1.0
    return oc

```

```

def tryplan(node, cli, start, cons, label, t=8.0):
    req = GetMotionPlan.Request()
    mr = MotionPlanRequest()
    mr.group_name = GROUP
    mr.num_planning_attempts = 3
    mr.allowed_planning_time = t
    mr.start_state = rstate(start)
    mr.max_velocity_scaling_factor = 0.3
    mr.max_acceleration_scaling_factor = 0.3
    mr.goal_constraints = [cons]
    req.motion_plan_request = mr
    res = call(node, cli, req, t + 20)
    if res is None:
        print(' %-34s 超时' % label)
        return False
    code = res.motion_plan_response.error_code.val
    if code != MoveItErrorCodes.SUCCESS:
        print(' %-34s × %s' % (label, ename(code)))
        return False
    tr = res.motion_plan_response.trajectory.joint_trajectory
    dur = tr.points[-1].time_from_start
    print(' %-34s sqrt %d 点 %.2fs' % (label, len(tr.points),
                                     dur.sec + dur.nanosec * 1e-9))

    return True

def main():
    rclpy.init()
    node = Node('cart_probe')
    fk_cli = node.create_client(GetPositionFK, '/compute_fk')
    mp_cli = node.create_client(GetMotionPlan, '/plan_kinematic_path')
    sv_cli = node.create_client(GetStateValidity, '/check_state_validity')
    for n, c in (('compute_fk', fk_cli), ('plan_kinematic_path', mp_cli)):
        if not c.wait_for_service(timeout_sec=15.0):
            print('FAIL %s 不可用' % n)
            return 1

    start = read_js(node) or [0.0] * 5
    print('实际当前状态: %s' % ([ '%+.4f' % v for v in start],))
    print('')

    ref = fk(node, fk_cli, Q_REF)
    if ref is None:
        print('FAIL 参考 FK 失败')
        return 1
    xyz = (ref.position.x, ref.position.y, ref.position.z)
    print('参考关节角 %s' % (Q_REF,))
    print(' → 三维坐标 x=%+.4f y=%+.4f z=%+.4f' % xyz)
    print('')

    # —— V0 参考状态合法性 ——
    print('V0 参考状态合法性')
    if sv_cli.wait_for_service(timeout_sec=10.0):
        r = GetStateValidity.Request()
        r.robot_state = rstate(Q_REF)
        r.group_name = GROUP
        res = call(node, sv_cli, r, 10.0)
        if res is None:
            print(' × 服务超时')
        else:
            print(' valid=%s contacts=%d %s'
                  % (res.valid, len(res.contacts),
                     ' ' if res.valid else ' ← 参考姿态本身就自碰撞!'))
    else:
        print(' (/check_state_validity 不可用, 跳过)')

    print('')
    print('V1/V2/V3 规划对比')

    # —— V1 纯位置 10mm ——
    tryplan(node, mp_cli, start, pos_c(xyz, 0.010), 'V1 纯位置 10mm')

    # —— V2 纯位置 50mm ——
    tryplan(node, mp_cli, start, pos_c(xyz, 0.050), 'V2 纯位置 50mm')

    # —— V3 位置 + 姿态 (姿态来自参考 FK) ——
    c = pos_c(xyz, 0.010)

```

```

c.orientation_constraints = [ori_c(ref, 0.10)]
tryplan(node, mp_cli, start, c, 'V3 位置10mm + 姿态±0.1')

node.destroy_node()
rcipy.shutdown()
return 0

if __name__ == '__main__':
    sys.exit(main())

```

C.4 moveit_exec2.py —— L6 主验收（三维坐标 → 规划 → 执行）

```

#!/usr/bin/env python3
"""Moveit2 三维坐标 → 规划 → 执行（修正版）

上一版的 bug: `start` 只在开头读了一次，但执行后机械臂已经移位，
    下一条轨迹的起点就是错的 → JTC 报 PATH_TOLERANCE_VIOLATED。
本版：每次规划前都重读 /joint_states。

另外：机械臂完全伸直时（r~0）是奇异位形，附近的目标规划/执行都不稳，
    所以先走到一个「弯曲的 ready 姿态」，再在它附近扫描目标。

python3 moveit_exec2.py          扫描约 8 个目标
python3 moveit_exec2.py dry       只规划不执行
python3 moveit_exec2.py sweep    在 ready 姿态周围更密的扫描
"""
import math
import sys
import time

import rcipy
from rcipy.node import Node
from rcipy.action import ActionClient
from sensor_msgs.msg import JointState
from geometry_msgs.msg import Pose
from shape_msgs.msg import SolidPrimitive
from moveit_msgs.srv import GetPositionFK, GetMotionPlan
from moveit_msgs.action import ExecuteTrajectory
from moveit_msgs.msg import (
    RobotState, MoveitErrorCodes, Constraints, JointConstraint,
    MotionPlanRequest, PositionConstraint, BoundingVolume,
)

GROUP, TIP, BASE = 'arm', 'link5', 'base_link'
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']

# 远离奇异的 ready 姿态（弯曲的臂）
READY = [0.0, 0.45, -0.70, 0.35, 0.0]

# 在 READY 附近的关节偏移，每个做一次 FK 得到三维坐标
DELTAS = [
    ('j1 +0.20', [0.20, 0.00, 0.00, 0.00, 0.0]),
    ('j1 -0.20', [-0.20, 0.00, 0.00, 0.00, 0.0]),
    ('j2 +0.15', [0.00, 0.15, 0.00, 0.00, 0.0]),
    ('j2 -0.15', [0.00, -0.15, 0.00, 0.00, 0.0]),
    ('j3 +0.15', [0.00, 0.00, 0.15, 0.00, 0.0]),
    ('j3 -0.15', [0.00, 0.00, -0.15, 0.00, 0.0]),
    ('j4 +0.20', [0.00, 0.00, 0.00, 0.20, 0.0]),
    ('j1+j2 复合', [0.15, 0.10, 0.00, 0.00, 0.0]),
    ('j2+j3 复合', [0.00, 0.12, -0.12, 0.00, 0.0]),
    ('大复合', [0.18, 0.15, -0.20, 0.15, 0.0]),
]

SWEEP = [
    ('%s j1 %+0.2f' % ('', d), [d, 0.0, 0.0, 0.0, 0.0])
    for d in (-0.30, -0.20, -0.10, 0.10, 0.20, 0.30)
] + [
    ('j2+j3 %+0.2f' % d, [0.0, d, -d, 0.0, 0.0])
    for d in (-0.20, -0.12, 0.12, 0.20)
]

def rstate(q):
    rs = RobotState()
    rs.joint_state.name = JOINTS

```

```

rs.joint_state.position = list(q)
return rs

def call(node, cli, req, t=30.0):
    fut = cli.call_async(req)
    rclpy.spin_until_future_complete(node, fut, timeout_sec=t)
    return fut.result()

def read_js(node, timeout=5.0):
    box = {'m': None}

    def cb(m):
        box['m'] = m

    sub = node.create_subscription(JointState, '/joint_states', cb, 10)
    t0 = time.time()
    while box['m'] is None and time.time() - t0 < timeout:
        rclpy.spin_once(node, timeout_sec=0.3)
    node.destroy_subscription(sub)
    if box['m'] is None:
        return None
    d = dict(zip(box['m'].name, box['m'].position))
    return [d.get(j, 0.0) for j in JOINTS]

def fk(node, cli, q):
    r = GetPositionFK.Request()
    r.header.frame_id = BASE
    r.fk_link_names = [TIP]
    r.robot_state = rstate(q)
    res = call(node, cli, r, 10.0)
    if res is None or res.error_code.val != MoveItErrorCodes.SUCCESS:
        return None
    return res.pose_stamped[0].pose

def pos_c(xyz, tol):
    pc = PositionConstraint()
    pc.header.frame_id = BASE
    pc.link_name = TIP
    pc.weight = 1.0
    s = SolidPrimitive()
    s.type = SolidPrimitive.SPHERE
    s.dimensions = [tol]
    p = Pose()
    p.position.x, p.position.y, p.position.z = xyz
    p.orientation.w = 1.0
    bv = BoundingVolume()
    bv.primitives = [s]
    bv.primitive_poses = [p]
    pc.constraint_region = bv
    c = Constraints()
    c.position_constraints = [pc]
    return c

def joint_c(target):
    c = Constraints()
    for name, val in zip(JOINTS, target):
        jc = JointConstraint()
        jc.joint_name = name
        jc.position = val
        jc.tolerance_above = 0.02
        jc.tolerance_below = 0.02
        jc.weight = 1.0
        c.joint_constraints.append(jc)
    return c

def plan(node, cli, start, cons, t=6.0):
    req = GetMotionPlan.Request()
    mr = MotionPlanRequest()
    mr.group_name = GROUP
    mr.num_planning_attempts = 5
    mr.allowed_planning_time = t
    mr.start_state = rstate(start)
    # ★ 降速跑: 舵机 + 卫星式轮询反馈跟不紧高动态轨迹

```

```

mr.max_velocity_scaling_factor = 0.2
mr.max_acceleration_scaling_factor = 0.2
mr.goal_constraints = [cons]
req.motion_plan_request = mr
res = call(node, cli, req, t + 25)
if res is None:
    return None, 'timeout'
code = res.motion_plan_response.error_code.val
if code != MoveItErrorCodes.SUCCESS:
    return None, 'code=%d' % code
return res.motion_plan_response.trajectory, None

def execute(node, cli, traj, t=30.0):
    goal = ExecuteTrajectory.Goal()
    goal.trajectory = traj
    fut = cli.send_goal_async(goal)
    rclpy.spin_until_future_complete(node, fut, timeout_sec=t)
    gh = fut.result()
    if gh is None or not gh.accepted:
        return None
    rf = gh.get_result_async()
    rclpy.spin_until_future_complete(node, rf, timeout_sec=t)
    r = rf.result()
    return r.result.error_code.val if r else None

def settle(node, want, tol=0.05, timeout=8.0):
    """等机械臂走完，返回稳定后的关节角"""
    t0 = time.time()
    q = None
    while time.time() - t0 < timeout:
        q = read_js(node)
        if q and all(abs(a - b) < tol for a, b in zip(q, want)):
            return q
        time.sleep(0.4)
    return q

def main():
    argv = sys.argv[1:]
    dry = 'dry' in argv
    cases = SWEEP if 'sweep' in argv else DELTAS

    rclpy.init()
    node = Node('moveit_exec2')
    fk_cli = node.create_client(GetPositionFK, '/compute_fk')
    mp_cli = node.create_client(GetMotionPlan, '/plan_kinematic_path')
    for n, c in (('compute_fk', fk_cli), ('plan_kinematic_path', mp_cli)):
        if not c.wait_for_service(timeout_sec=15.0):
            print('FAIL %s 不可用' % n)
            return 1
    ex_cli = ActionClient(node, ExecuteTrajectory, '/execute_trajectory')
    if not dry and not ex_cli.wait_for_server(timeout_sec=15.0):
        print('FAIL 找不到 /execute_trajectory')
        return 1

    # —— ① 先走到 ready 姿态 (远离奇异) ——
    cur = read_js(node)
    if cur is None:
        print('FAIL 读不到 /joint_states')
        return 1
    p = fk(node, fk_cli, cur)
    print('起始关节角: %s' % ([v%.3f for v in cur],))
    if p:
        print('起始末端: x=%+.4f y=%+.4f z=%+.4f' % (p.position.x, p.position.y, p.position.z))

    if not dry:
        print('')
        print('→ 先走到 ready 姿态 %s (远离奇异)' % ([v%.2f for v in READY],))
        traj, e = plan(node, mp_cli, cur, joint_c(READY))
        if traj is None:
            print('FAIL 规划: %s' % e)
            return 1
        code = execute(node, ex_cli, traj)
        print('执行 code=%s' % code)
        if code != MoveItErrorCodes.SUCCESS:
            return 1
        cur = settle(node, READY)

```

```

p = fk(node, fk_cli, cur)
print(' ready 末端: x=%.4f y=%.4f z=%.4f'
      % (p.position.x, p.position.y, p.position.z))

print('')
hdr = '%-18s %-24s %-7s %-7s %s' % ('目标', '三维坐标', '规划', '执行', '实测误差')
print(hdr)
print('-' * len(hdr))

n_ok = n_plan = 0
errs = []

for label, delta in cases:
    # ★ 每次都【最新的】当前状态做起点
    start = read_js(node)
    if start is None:
        print('%-18s 读不到 /joint_states' % label)
        continue

    q_ref = [s + d for s, d in zip(start, delta)]
    ref = fk(node, fk_cli, q_ref)
    if ref is None:
        print('%-18s FK 失败' % label)
        continue
    xyz = (ref.position.x, ref.position.y, ref.position.z)

    traj, e = plan(node, mp_cli, start, pos_c(xyz, 0.010))
    if traj is None:
        print('%-18s (%+.3f, %+.3f, %+.3f) FAIL %s'
              % (label, xyz[0], xyz[1], xyz[2], e))
        continue
    n_plan += 1

    if dry:
        print('%-18s (%+.3f, %+.3f, %+.3f) OK - -'
              % (label, xyz[0], xyz[1], xyz[2]))
        continue

    code = execute(node, ex_cli, traj)
    if code != MoveItErrorCodes.SUCCESS:
        print('%-18s (%+.3f, %+.3f, %+.3f) OK FAIL %s'
              % (label, xyz[0], xyz[1], xyz[2], code))
        time.sleep(1.0)
        continue

    # ★ 舵机走完后还要稳一下; read() 的轮询周期是 240ms,
    # 太早读会读到「还在路上」的值, 误差虚高。
    time.sleep(3.0)
    now = read_js(node)
    pe = fk(node, fk_cli, now) if now else None
    if pe is None:
        print('%-18s (%+.3f, %+.3f, %+.3f) OK OK 读不到'
              % (label, xyz[0], xyz[1], xyz[2]))
        n_ok += 1
        continue

    d = math.sqrt((pe.position.x - xyz[0])**2 +
                  (pe.position.y - xyz[1])**2 +
                  (pe.position.z - xyz[2])**2)
    errs.append(d)
    n_ok += 1
    print('%-18s (%+.3f, %+.3f, %+.3f) OK OK %5.1f mm %s'
          % (label, xyz[0], xyz[1], xyz[2], d * 1000,
            'OK' if d < 0.010 else 'OUT'))

print('')
print('规划 %d/%d 执行 %d/%d' % (n_plan, len(cases), n_ok, len(cases)))
if errs:
    print('末端误差: 平均 %.1f mm 最大 %.1f mm' % (
        sum(errs) / len(errs) * 1000, max(errs) * 1000))

node.destroy_node()
rcipy.shutdown()
return 0

if __name__ == '__main__':
    sys.exit(main())

```

```
#!/usr/bin/env python3
"""大幅运动测试：三维坐标 → MoveIt2 规划 → 执行（大幅度）

python3 moveit_big.py          执行大幅序列
python3 moveit_big.py dry      只规划不执行（先看能不能规划）

安全设计：
• 所有目标都由 FK 从合法关节角生成 → 保证【可达】
• 速度缩放 0.3，慢走
• 每个目标单独判成败，失败就跳过，不中断
• 每个目标移动后等机械臂停稳再读回，报实测误差
• 开头先走到 ready（远离奇异）
"""

import math
import sys
import time

import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from sensor_msgs.msg import JointState
from geometry_msgs.msg import Pose
from shape_msgs.msg import SolidPrimitive
from moveit_msgs.srv import GetPositionFK, GetMotionPlan
from moveit_msgs.action import ExecuteTrajectory
from moveit_msgs.msg import (
    RobotState, MoveItErrorCodes, Constraints, JointConstraint,
    MotionPlanRequest, PositionConstraint, BoundingVolume,
)

GROUP, TIP, BASE = 'arm', 'link5', 'base_link'
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
READY = [0.0, 0.45, -0.70, 0.35, 0.0]

# 大幅度目标（关节角都在限位内：joint1~4 ±1.5708, joint5 ±3.1416）
CONFIGS = [
    ('① 前伸水平', [0.00, 1.45, 0.00, 0.00, 0.0]),
    ('② 前伸下压', [0.00, 1.20, -0.60, 0.30, 0.0]),
    ('③ 左侧大幅', [1.30, 1.00, -0.40, 0.20, 0.0]),
    ('④ 右侧大幅', [-1.30, 1.00, -0.40, 0.20, 0.0]),
    ('⑤ 后收上举', [0.00, -0.60, 1.20, -0.50, 0.0]),
    ('⑥ 复合+腕自转', [1.20, 0.90, -1.10, 0.60, 2.5]),
    ('⑦ 复合-腕自转', [-1.20, 0.90, -1.10, 0.60, -2.5]),
    ('⑧ 回归 ready', [0.00, 0.45, -0.70, 0.35, 0.0]),
]

TOL = 0.020      # 三维坐标容差 20mm（大幅运动放宽）
SCALE = 0.3      # 速度缩放

def rstate(q):
    rs = RobotState()
    rs.joint_state.name = JOINTS
    rs.joint_state.position = list(q)
    return rs

def call(node, cli, req, t=30.0):
    fut = cli.call_async(req)
    rclpy.spin_until_future_complete(node, fut, timeout_sec=t)
    return fut.result()

def read_js(node, timeout=5.0):
    box = {'m': None}

    def cb(m):
        box['m'] = m

    sub = node.create_subscription(JointState, '/joint_states', cb, 10)
    t0 = time.time()
    while box['m'] is None and time.time() - t0 < timeout:
        rclpy.spin_once(node, timeout_sec=0.3)
    node.destroy_subscription(sub)
    if box['m'] is None:
```

```

        return None
    d = dict(zip(box['m'].name, box['m'].position))
    return [d.get(j, 0.0) for j in JOINTS]

def fk(node, cli, q):
    r = GetPositionFK.Request()
    r.header.frame_id = BASE
    r.fk_link_names = [TIP]
    r.robot_state = rstate(q)
    res = call(node, cli, r, 10.0)
    if res is None or res.error_code.val != MoveItErrorCodes.SUCCESS:
        return None
    return res.pose_stamped[0].pose

def pos_c(xyz, tol):
    pc = PositionConstraint()
    pc.header.frame_id = BASE
    pc.link_name = TIP
    pc.weight = 1.0
    s = SolidPrimitive()
    s.type = SolidPrimitive.SPHERE
    s.dimensions = [tol]
    p = Pose()
    p.position.x, p.position.y, p.position.z = xyz
    p.orientation.w = 1.0
    bv = BoundingVolume()
    bv.primitives = [s]
    bv.primitive_poses = [p]
    pc.constraint_region = bv
    c = Constraints()
    c.position_constraints = [pc]
    return c

def joint_c(target):
    c = Constraints()
    for name, val in zip(JOINTS, target):
        jc = JointConstraint()
        jc.joint_name = name
        jc.position = val
        jc.tolerance_above = 0.02
        jc.tolerance_below = 0.02
        jc.weight = 1.0
        c.joint_constraints.append(jc)
    return c

def plan(node, cli, start, cons, t=8.0):
    req = GetMotionPlan.Request()
    mr = MotionPlanRequest()
    mr.group_name = GROUP
    mr.num_planning_attempts = 5
    mr.allowed_planning_time = t
    mr.start_state = rstate(start)
    mr.max_velocity_scaling_factor = SCALE
    mr.max_acceleration_scaling_factor = SCALE
    mr.goal_constraints = [cons]
    req.motion_plan_request = mr
    res = call(node, cli, req, t + 30)
    if res is None:
        return None, 'timeout'
    code = res.motion_plan_response.error_code.val
    if code != MoveItErrorCodes.SUCCESS:
        return None, 'code=%d' % code
    return res.motion_plan_response.trajectory, None

def execute(node, cli, traj, t=45.0):
    goal = ExecuteTrajectory.Goal()
    goal.trajectory = traj
    fut = cli.send_goal_async(goal)
    rclpy.spin_until_future_complete(node, fut, timeout_sec=t)
    gh = fut.result()
    if gh is None or not gh.accepted:
        return None
    rf = gh.get_result_async()
    rclpy.spin_until_future_complete(node, rf, timeout_sec=t)

```

```

r = rf.result()
return r.result.error_code.val if r else None

def traj_duration(traj):
    # traj 是 RobotTrajectory, 点在里面那层 joint_trajectory 里
    pts = traj.joint_trajectory.points
    if not pts:
        return 0.0
    d = pts[-1].time_from_start
    return d.sec + d.nanosec * 1e-9

def main():
    dry = 'dry' in sys.argv[1:]

    rclpy.init()
    node = Node('moveit_big')
    fk_cli = node.create_client(GetPositionFK, '/compute_fk')
    mp_cli = node.create_client(GetMotionPlan, '/plan_kinematic_path')
    for n, c in (('compute_fk', fk_cli), ('plan_kinematic_path', mp_cli)):
        if not c.wait_for_service(timeout_sec=15.0):
            print('FAIL %s 不可用 —— move_group 起了吗? ' % n)
            return 1
    ex_cli = ActionClient(node, ExecuteTrajectory, '/execute_trajectory')
    if not dry and not ex_cli.wait_for_server(timeout_sec=15.0):
        print('FAIL 找不到 /execute_trajectory —— ros2_control 起了吗? ')
        return 1

    cur = read_js(node)
    if cur is None:
        print('FAIL 读不到 /joint_states')
        return 1
    p = fk(node, fk_cli, cur)
    print('起始关节角: %s' % ([ '%+.3f' % v for v in cur ],))
    if p:
        print('起始末端: x=%+.4f y=%+.4f z=%+.4f'
              % (p.position.x, p.position.y, p.position.z))
    print('幅度: joint1 ±1.30 joint2 ~1.45 joint3 ~-1.10 joint5 ±2.50')
    print('容差 %0f mm, 速度缩放 %1f, %s' % (TOL * 1000, SCALE,
                                              '只规划' if dry else '规划+执行'))
    print('')

    # —— 先走到 ready ——
    if not dry:
        print('→ 先回到 ready 姿态')
        traj, e = plan(node, mp_cli, cur, joint_c(READY))
        if traj is None:
            print(' FAIL 规划: %s' % e)
            return 1
        print(' 轨迹时长 %.1fs, 执行...' % traj_duration(traj))
        code = execute(node, ex_cli, traj)
        if code != MoveItErrorCodes.SUCCESS:
            print(' FAIL 执行 code=%s' % code)
            return 1
        time.sleep(3.0)
        cur = read_js(node)
        p = fk(node, fk_cli, cur)
        print(' ready 末端: x=%+.4f y=%+.4f z=%+.4f'
              % (p.position.x, p.position.y, p.position.z))
        print('')

    hdr = '%-16s %-26s %-9s %-7s %s' % ('目标', '三维坐标', '规划', '执行', '实测误差')
    print(hdr)
    print('-' * len(hdr))

    n_plan = n_exec = 0
    errs = []

    for label, cfg in CONFIGS:
        start = read_js(node)
        if start is None:
            print('%-16s 读不到 /joint_states' % label)
            continue

        ref = fk(node, fk_cli, cfg)
        if ref is None:
            print('%-16s FK 失败' % label)
            continue

```

```

xyz = (ref.position.x, ref.position.y, ref.position.z)

traj, e = plan(node, mp_cli, start, pos_c(xyz, TOL))
if traj is None:
    print('%-16s (%+.3f, %+.3f, %+.3f) FAIL %s'
          % (label, xyz[0], xyz[1], xyz[2], e))
    time.sleep(0.5)
    continue
n_plan += 1
dur = traj_duration(traj)

if dry:
    print('%-16s (%+.3f, %+.3f, %+.3f) OK %.1fs - -'
          % (label, xyz[0], xyz[1], xyz[2], dur))
    continue

code = execute(node, ex_cli, traj)
if code != MoveItErrorCodes.SUCCESS:
    print('%-16s (%+.3f, %+.3f, %+.3f) OK %.1fs FAIL %s'
          % (label, xyz[0], xyz[1], xyz[2], dur, code))
    time.sleep(1.0)
    continue
n_exec += 1

# 等机械臂真正停稳 (轨迹时长 + 余量)
time.sleep(max(2.0, dur * 0.3 + 2.0))
now = read_js(node)
pe = fk(node, fk_cli, now) if now else None
if pe is None:
    print('%-16s (%+.3f, %+.3f, %+.3f) OK %.1fs OK 读不到'
          % (label, xyz[0], xyz[1], xyz[2], dur))
    continue

d = math.sqrt((pe.position.x - xyz[0])**2 +
              (pe.position.y - xyz[1])**2 +
              (pe.position.z - xyz[2])**2)
errs.append(d)
print('%-16s (%+.3f, %+.3f, %+.3f) OK %.1fs OK %5.1f mm %s'
      % (label, xyz[0], xyz[1], xyz[2], dur, d * 1000,
        'OK' if d < TOL else 'OUT'))

print('')
print('规划 %d/%d 执行 %d/%d' % (n_plan, len(CONFIGS), n_exec, len(CONFIGS)))
if errs:
    print('末端误差: 平均 %.1f mm 最大 %.1f mm'
          % (sum(errs) / len(errs) * 1000, max(errs) * 1000))

node.destroy_node()
rcipy.shutdown()
return 0

if __name__ == '__main__':
    sys.exit(main())

```

C.6 can_smooth.py — L7 裸 CAN 测真实曲线 + 顺滑度 (★ 最有用的诊断工具)

```

#!/usr/bin/env python3
"""顺滑度对比: 纯串口(裸 CAN) vs MoveIt2 流式下发

用 raw SocketCAN 直接和 STM32 说话, 20Hz 高频查询舵机位置,
拿到真实运动曲线, 再算速度抖动指标。

python3 can_smooth.py direct      纯串口: 一条 T=6s 的命令
python3 can_smooth.py direct 3.0  改时长

对比时:
① 先停掉 ros2_control, 跑 direct → 这是"你说的那个丝滑基线"
② 再起 ros2_control, 跑 jtc → 这是 MoveIt2 那条路
③ 比较两者的 CV (变异系数, 越小越顺滑)
"""
import math
import socket
import statistics
import struct
import sys

```

```

import time

CAN_IF = 'can0'
ID_DOWN = 0x104      # 本机 → STM32
ID_UP = 0x200        # STM32 → 本机
ASK_SETPOS = 1
ASK_STATUS = 2

# joint1 的标定 (和 xacro 一致)
J1_ID = 0
J1_SERVO_MIN, J1_SERVO_MAX = 500, 2500
J1_POS_MIN, J1_POS_MAX = -1.5708, 1.5708
J1_DIR = +1

def angle_to_servo(a):
    sp = J1_SERVO_MAX - J1_SERVO_MIN
    ap = J1_POS_MAX - J1_POS_MIN
    t = (a - J1_POS_MIN) / ap
    if J1_DIR < 0:
        t = 1.0 - t
    t = max(0.0, min(1.0, t))
    return int(J1_SERVO_MIN + t * sp)

def servo_to_angle(p):
    sp = J1_SERVO_MAX - J1_SERVO_MIN
    ap = J1_POS_MAX - J1_POS_MIN
    t = (p - J1_SERVO_MIN) / sp
    if J1_DIR < 0:
        t = 1.0 - t
    t = max(0.0, min(1.0, t))
    return J1_POS_MIN + t * ap

def can_open():
    s = socket.socket(socket.AF_CAN, socket.SOCK_RAW, socket.CAN_RAW)
    s.bind((CAN_IF,))
    s.settimeout(0.0)
    return s

def send(s, can_id, data):
    pad = bytes(data) + b'\x00' * (8 - len(data))
    s.send(struct.pack('=IB3x8s', can_id, len(data), pad))

def drain(s, timeout=0.15):
    """收掉所有在途帧, 返回 {id: position}"""
    got = {}
    t0 = time.time()
    while time.time() - t0 < timeout:
        try:
            f = s.recv(16)
        except BlockingIOError:
            time.sleep(0.002)
            continue
        if len(f) < 16:
            continue
        can_id, dlc = struct.unpack('=IB3x', f[:8])
        d = f[8:16]
        if (can_id & 0x1FFFFFFF) == ID_UP and dlc >= 3:
            sid = d[0]
            pos = d[1] | (d[2] << 8)
            got[sid] = pos
    return got

def query(s, sid):
    send(s, ID_DOWN, [sid, ASK_STATUS, 0, 0, 0, 0, 0, 0])

def read_pos(s, sid, tries=3):
    for _ in range(tries):
        query(s, sid)
        got = drain(s, 0.035)
        if sid in got:
            return got[sid]
    return None

```

```

def metrics(track, label, k=8):
    """track: [(t, angle)]。用大小为 k 的窗口做局部线性拟合估速度，
    避免采样间隔不均造成的假抖动。"""
    if len(track) < 2 * k + 2:
        print(' %s: 样本太少 (%d)' % (label, len(track)))
        return None

    t0 = track[0][0]
    ts = [t - t0 for t, _ in track]
    xs = [a for _, a in track]

    vels = []
    for i in range(k, len(track) - k):
        # 局部窗口最小二乘斜率
        n = 2 * k + 1
        tw = ts[i - k:i + k + 1]
        xw = xs[i - k:i + k + 1]
        mt = sum(tw) / n
        mx = sum(xw) / n
        num = sum((a - mt) * (b - mx) for a, b in zip(tw, xw))
        den = sum((a - mt) ** 2 for a in tw)
        if den > 1e-12:
            vels.append((ts[i], num / den))

    if not vels:
        print(' %s: 算不出速度' % label)
        return None

    total = abs(xs[-1] - xs[0])
    vmax = max(abs(v) for _, v in vels)
    moving = [(t, v) for t, v in vels if abs(v) > 0.15 * vmax]

    if len(moving) < 3:
        print(' %s: 运动段太短' % label)
        return None

    vs = [v for _, v in moving]
    mean = statistics.mean(vs)
    sd = statistics.pstdev(vs)
    cv = sd / abs(mean) if mean else 0.0
    stalls = sum(1 for v in vs if abs(v) < 0.15 * vmax)
    rev = sum(1 for a, b in zip(vs, vs[1:]) if a * b < 0)

    print(' %14s 位移 %.3f rad 样本 %3d 运动段 %3d' % (label, total, len(track), len(moving)))
    print(' %14s 平均速度 %.3f 峰值 %.5f 标准差 %.3f' % ('', mean, vmax, sd))
    print(' %14s ★ CV %.3f 停顿占比 %.1f%% 速度反向 %d 次' % ('', cv, 100.0 * stalls / len(vs), rev))
    return {'cv': cv, 'stall': 100.0 * stalls / len(vs), 'rev': rev,
            'mean': mean, 'vmax': vmax, 'n': len(track)}

def run_home(target_a=0.0, dur=3.0):
    """把 joint1 慢慢挪到一个安全的中点，作为基线起点"""
    s = can_open()
    p0 = read_pos(s, J1_ID)
    if p0 is None:
        print('FAIL 读不到舵机位置')
        return 1
    a0 = servo_to_angle(p0)
    pT = angle_to_servo(target_a)
    print('———— 归位: joint1 %.4f → %.4f rad (servo %d → %d) ————' % (a0, target_a, p0, pT))
    send(s, ID_DOWN, [J1_ID, ASK_SETPOS,
                      pT & 0xFF, (pT >> 8) & 0xFF,
                      int(dur * 1000) & 0xFF, (int(dur * 1000) >> 8) & 0xFF, 0, 0])

    t0 = time.time()
    last = p0
    while time.time() - t0 < dur + 2.0:
        p = read_pos(s, J1_ID, tries=1)
        if p is not None:
            last = p
        time.sleep(0.1)
    print(' 到位 servo=%d → %.4f rad' % (last, servo_to_angle(last)))
    print('')
    return 0

```

```

def run_direct(dur):
    s = can_open()
    p0 = read_pos(s, J1_ID)
    if p0 is None:
        print('FAIL 读不到舵机 %d 的位置 —— CAN 通吗?' % J1_ID)
        return 1
    a0 = servo_to_angle(p0)
    # ★ 方向自适应: +0.8 会撞限位就往反方向走
    delta = 0.8 if (a0 + 0.8) <= J1_POS_MAX - 0.05 else -0.8
    aT = a0 + delta
    pT = angle_to_servo(aT)
    t_ms = int(dur * 1000)

    print('———— 纯串口基线 (完全绕开 ros2_control) ————')
    print(' 当前 servo=%d → %.4f rad' % (p0, a0))
    print(' 目标 servo=%d → %.4f rad' % (pT, aT))
    print(' 方向 %.1f rad' % delta)
    print(' 一条命令: 104#%02X%02X%02X%02X%02X%02X0000 (T=%d ms)'
          % (J1_ID, ASK_SETPOS, pT & 0xFF, (pT >> 8) & 0xFF,
             t_ms & 0xFF, (t_ms >> 8) & 0xFF, t_ms))
    print('')

    # 发一条长 T 命令 (这就是「直接通过串口发送坐标」)
    send(s, ID_DOWN, [J1_ID, ASK_SETPOS,
                      pT & 0xFF, (pT >> 8) & 0xFF,
                      t_ms & 0xFF, (t_ms >> 8) & 0xFF, 0, 0])

    track = []
    t0 = time.time()
    while time.time() - t0 < dur + 1.5:
        p = read_pos(s, J1_ID, tries=1)
        if p is not None:
            track.append((time.time() - t0, servo_to_angle(p)))
        time.sleep(0.008) # 目标 20Hz 采样

    print('———— 结果 ————')
    r = metrics(track, '纯串口')
    return 0

def run_jtc(dur):
    """MoveIt2/ros2_control 那条路: 发一条 FollowJointTrajectory,
    同时用裸 CAN 高频采样"""
    import rclpy
    from rclpy.node import Node
    from rclpy.action import ActionClient
    from control_msgs.action import FollowJointTrajectory
    from trajectory_msgs.msg import JointTrajectoryPoint
    from builtin_interfaces.msg import Duration

    s = can_open()
    p0 = read_pos(s, J1_ID)
    if p0 is None:
        print('FAIL 读不到舵机位置')
        return 1
    a0 = servo_to_angle(p0)
    delta = 0.8 if (a0 + 0.8) <= J1_POS_MAX - 0.05 else -0.8
    aT = a0 + delta

    rclpy.init()
    node = Node('can_smooth')
    cli = ActionClient(node, FollowJointTrajectory,
                      '/arm_controller/follow_joint_trajectory')
    if not cli.wait_for_server(timeout_sec=8.0):
        print('FAIL 找不到 arm_controller —— ros2_control 起了吗?')
        return 1

    J = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
    start = [a0, 0.0, 0.0, 0.0, 0.0]
    target = [aT, 0.0, 0.0, 0.0, 0.0]

    goal = FollowJointTrajectory.Goal()
    goal.trajectory.joint_names = J
    goal.trajectory.points = [
        JointTrajectoryPoint(positions=start, time_from_start=Duration(sec=0)),
        JointTrajectoryPoint(positions=target,
                              time_from_start=Duration(sec=int(dur),
                                                            nanosec=int((dur % 1) * 1e9))),
    ]

```

```

print('——— ros2_control + MoveIt2 链路 ——')
print(' joint1 %.4f → %.4f rad, 轨迹时长 %.1fs' % (a0, aT, dur))
print('')

fut = cli.send_goal_async(goal)
rclpy.spin_until_future_complete(node, fut, timeout_sec=10)
gh = fut.result()
if gh is None or not gh.accepted:
    print('FAIL 目标被拒绝')
    return 1

track = []
t0 = time.time()
rf = gh.get_result_async()
while time.time() - t0 < dur + 1.5:
    rclpy.spin_once(node, timeout_sec=0.0)
    p = read_pos(s, J1_ID, tries=1)
    if p is not None:
        track.append((time.time() - t0, servo_to_angle(p)))
    time.sleep(0.008)

print('——— 结果 ——')
r = metrics(track, 'ros2_control')
node.destroy_node()
rclpy.shutdown()
return 0

def main():
    mode = sys.argv[1] if len(sys.argv) > 1 else 'direct'
    dur = float(sys.argv[2]) if len(sys.argv) > 2 else 6.0

    if mode == 'home':
        return run_home(0.0, dur)
    if mode == 'direct':
        return run_direct(dur)
    if mode == 'jtc':
        return run_jtc(dur)
    print(__doc__)
    return 1

if __name__ == '__main__':
    sys.exit(main())

```

C.7 smooth_check.py —— L7 通过 ROS 侧量顺滑度

```

#!/usr/bin/env python3
"""顺滑度对比测试：同一条慢速轨迹，走多次，量测「运动是否连续」

判断顺滑的客观指标：在轨迹执行期间，以 20Hz 采样 /joint_states，
看【关节速度】的抖动。一卡一卡时速度会忽大忽小（走走停停）。

python3 smooth_check.py          默认测 joint1 转 0.8 rad / 6s
python3 smooth_check.py 0.5 4    转 0.5 rad / 4s
"""
import statistics
import sys
import time

import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from sensor_msgs.msg import JointState
from control_msgs.action import FollowJointTrajectory
from trajectory_msgs.msg import JointTrajectoryPoint
from builtin_interfaces.msg import Duration

# ★ arm_controller 只管这 5 个关节（gripper 是另一个控制器）
JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
ACTION = '/arm_controller/follow_joint_trajectory'

def main():
    argv = sys.argv[1:]

```

```

delta = float(argv[0]) if argv else 0.8
dur = float(argv[1]) if len(argv) > 1 else 6.0

rclpy.init()
node = Node('smooth_check')

samples = []
cur = {'q': None}

def cb(m):
    try:
        q = [m.position[m.name.index(j)] for j in JOINTS]
    except (ValueError, IndexError):
        return
    cur['q'] = q
    samples.append((time.time(), q))

node.create_subscription(JointState, '/joint_states', cb, 50)
while cur['q'] is None:
    rclpy.spin_once(node, timeout_sec=0.5)

start = list(cur['q'])
target = list(start)
target[0] += delta
print('起始: %s' % ([ '%.3f' % v for v in start],))
print('目标: %s 时长 %.1fs' % ([ '%.3f' % v for v in target], dur))
print('')

cli = ActionClient(node, FollowJointTrajectory, ACTION)
if not cli.wait_for_server(timeout_sec=8.0):
    print('FAIL 找不到 %s' % ACTION)
    return 1

goal = FollowJointTrajectory.Goal()
goal.trajectory.joint_names = JOINTS
goal.trajectory.points = [
    JointTrajectoryPoint(positions=start,
                          time_from_start=Duration(sec=0, nanosec=0)),
    JointTrajectoryPoint(positions=target,
                          time_from_start=Duration(sec=int(dur),
                                                  nanosec=int((dur % 1) * 1e9))),
]

samples.clear()
t0 = time.time()
fut = cli.send_goal_async(goal)
rclpy.spin_until_future_complete(node, fut, timeout_sec=10)
gh = fut.result()
if gh is None or not gh.accepted:
    print('FAIL 目标被拒绝')
    return 1
rf = gh.get_result_async()
rclpy.spin_until_future_complete(node, rf, timeout_sec=int(dur) + 20)
code = rf.result().result.error_code

# 边转边采样
while time.time() - t0 < dur + 2.5:
    rclpy.spin_once(node, timeout_sec=0.05)

print('error_code = %s' % code)
print('到位后: %s' % ([ '%.3f' % v for v in cur['q']],))
print('')

# —— 用 joint1 的位移序列算「速度抖动」 ——
traj = [(t, q[0]) for t, q in samples if t >= t0 - 0.5]
if len(traj) < 10:
    print('采样太少 (%d), 无法评估' % len(traj))
    return 0

vels = []
for (t1, p1), (t2, p2) in zip(traj, traj[1:]):
    dt = t2 - t1
    if dt > 1e-6:
        vels.append((t2, (p2 - p1) / dt))

# 只统计运动区间 (速度明显非零的部分)
moving = [(t, v) for t, v in vels if abs(v) > 0.005]
if len(moving) < 5:
    print('几乎没有运动, 无法评估 (采样 %d, 速度点 %d)' % (len(traj), len(vels)))

```

```

    return 0

    vs = [v for _, v in moving]
    mean = statistics.mean(vs)
    sd = statistics.pstdev(vs)
    cv = sd / abs(mean) if mean else 0.0

    # 反向检验：速度变号的次数（走走停停会让符号反复跳）
    sign_changes = sum(1 for a, b in zip(vs, vs[1:]) if a * b < 0)
    # 速度接近 0 的样本占比（停顿）
    stalls = sum(1 for v in vs if abs(v) < 0.02)

    print('———— 顺滑度指标 (joint1) ————')
    print('  采样点数          %d' % len(traj))
    print('  运动区间样本        %d' % len(moving))
    print('  平均速度              %+.4f rad/s' % mean)
    print('  速度标准差            %.4f rad/s' % sd)
    print('  ★ 变异系数 CV          %3f          (越小越顺滑)' % cv)
    print('  ★ 停顿样本占比          %.1f%%          (速度 < 0.02 rad/s)' % (100.0 * stalls / len(vs)))
    print('  速度反向次数            %d          (走走停停会反复跳)' % sign_changes)
    print('')
    print('  参考: CV < 0.3 算顺滑, > 0.6 就是明显的走走停停')

    node.destroy_node()
    rclpy.shutdown()
    return 0

if __name__ == '__main__':
    sys.exit(main())

```